

介绍一下你的XX项目？

介绍是什么？

- 校园论坛是一个在线论坛交流网站，实现**用户发帖交流评论**，实现了**富文本发帖、回帖以及楼中楼回复、点赞收藏、消息通知**等功能
- 服务器监控运维系统是一个实时在线监控服务器运行状态以及详细信息的网站，能够做到**服务器状态和信息监控、一键SSH远程登录、子用户分配具体服务器**等操作

介绍主体功能？

校园论坛

- 进入系统前的**注册 / 登录**功能（JWT / rabbitmq发验证码 / redis存验证码）
- 最主要的**帖子相关功能**：富文本发帖 / 评论帖子 / 楼中楼回复 / 收藏点赞帖子 / 隐私展示 / 评论提醒
- 工具类 接口限流的**基于Redis的固定窗口计数器限流**

服务器监控运维系统

- 介绍三个模块：**服务端、客户端、网页端**
- 服务端：生成token展示在网页端，接收来自客户端的数据并发给网页端展示（并用influxdb存储起来）
- 客户端：安装在被监控主机上，携带token向服务端注册后定时自动进行数据收集并上报
- 网页端：展示实时最新数据以及历史一小时的折线图数据等等，并且可以进行SSH连接客户端
- 另外：管理员可以在网页端进行子用户创建并为其分配可操作的服务器

★项目有什么难点？

论坛交互内容（点赞和收藏）的数据存储问题

由于论坛交互数据(如点赞、收藏等)的更新会非常频繁，更新信息实时到MySQL不太现实，所以需要Redis做缓冲并在合适的时机一次性入库一段时间内的全部数据

核心点：**线程池、Redis 哈希结构、定时任务、同步锁、数据同步 / 一致性**

- **获取交互信息**：从Redis中获取交互信息，如果Redis中没有则从MySQL中获取
- **更新交互信息**：**加锁**写入Redis，并且开启定时的入库任务（如果已经开启了，那么仅更新Redis不开启新的）
- **存储交互信息**：采用Redis的Hash，key为 交互的类型(点赞/收藏)，hashkey为 用户ID+交互帖子ID，hashvalue即为 交互的目标状态(false/true)，并使用线程池往Redis中写入数据
- **同步交互信息（入库）**：设定每3s进行一次Redis中的数据同步到MySQL（同样需要**加锁**写入），**入库后清除**相关交互信息的缓存
- **☆☆☆为什么更新Redis和入库MySQL需要加锁？**

这里加锁加的是 `synchronized(type.intern())` type是交互类型（点赞/收藏）

为了防止发生：入库MySQL的同时来了新的交互数据更新，此时会发生丢失这段新的数据

因此需要加锁来让入库的时候，新的交互数据等待入库完毕再写入Redis

- **☆☆如何保障Redis和MySQL的数据一致性？**
 - 仅从Redis中获取数据，只有不存在的时候才去MySQL中取
 - 入库和更新不会同时进行（加锁）、入库后删除所有交互数据

接口限流问题

- 先介绍主流的几种限流方式：
 - **固定窗口计数器**，优点是实现简单、易于理解和部署，缺点是可能存在时间不对称以及难以应对突发流量
 - **滑动窗口计数器**，优点是能更好应对突发流量，缺点是实现复杂、需要维护窗口的状态
 - **令牌桶算法**，优点是能够很好地平滑流量，允许一定程度的突发流量，缺点是需要更多资源来维护令牌桶状态、不能处理长时间的突发流量

- **漏桶算法**，优点是可以按照恒定的速率限制请求且算法相对简单，缺点是不够灵活、无法应对突发流量、可能导致请求延迟
- 再说（我采用的是**固定窗口计数器限流**）
 - 加了一个继承自HttpFilter的限流过滤器在CORS过滤器之后
 - 采用filter对来自同个IP的请求计数（每次请求前加HTTP过滤器）
 - 限流工具类的Redis的key是ip地址，value是访问次数
 - 编写了多个逻辑的方法以供不同目的的限流，如
 - 针对单次频率限制，请求成功后一段时间不得再发起请求（如点击请求邮件或者手机验证码）
 - 针对一段时间内多次请求的频率限制（如3秒内最多允许20次请求，例如论坛的评论）
 - 针对于需要采取阶段封禁的频率限制（如防止用户频繁申请JWT令牌，如果已经提示无法登录而用户还在请求，那么就封禁更长时间）

★项目八股

★校园论坛

springboot3，相比2有什么不同？

1. **最低Java环境**。SpringBoot3使用Java17作为最低版本，且基于Spring6，而spring boot2最低版本是Java8且基于Spring5
2. **GraalVM支持**。相比 SpringBoot2，SpringBoot3 的 Spring Native 也是升级的一个重大特性，支持使用 GraalVM 将 Spring 的应用程序编译成本地可执行的镜像文件，提升启动速度、峰值性能以及减少内存使用

Java8-17有什么新特性？

Lambda表达式

匿名内部类，当且仅当接口内部仅有一个抽象方法（可以有多个，但是其他方法必须有默认实现）

写成 `() -> {}`

Optional类

更“优雅”地处理空指针问题，使用Optional类可以避免显式的null值判断

```
String str = null;
if(str != null){    //不使用optional
    list.add(str);
}

//使用Optional
String str = null;
Optional<String> optional = Optional.ofNullable(str);    //转换为Optional
optional.ifPresent(System.out::println);                //str非空时再执行方法
```

文本块

```
String str = """
    你好
    nihao
    hallo
    """

// 可以更好的书写HTML或者SQL语句了，无需写很多转义字符
```

记录类型record

相当于是官方出版的翻版“lombok”

```
public record Account(String username, String password) { // 字段直接写在括号里
    // 帮我们像lombok一样写好了 全参构造方法 getter hashCode equals toString等方法
}
```

密封类型sealed

“人如其名”，就是密封了一个类，显式写明只有某些类可以继承这个密封类，其他类都不可以继承

```
public sealed class A permits B{
    // 在class关键字前添加sealed关键字，表示此类为密封类型
    // permits后面跟上允许继承的类型，多个子类使用逗号隔开
    // 表明只有类B可以继承类A，并且必须有类继承A
    // 继承的子类必须显式标记为final、sealed或是non-sealed类型(非密封类，代表可以继承了)
}
```

🌟介绍一下 JWT? 和session比起来有什么优劣?

jwt (json web token) 由三部分**组成**:

- 标头(header): 包含一些元数据 例如JWT使用的加密算法 / 类型
- 有效载荷(payload): 包括用户名称、令牌发布时间 / 过期时间、JWT ID等，自定义的字段也存在于这里
- 签名(signature): 根据服务端的密钥，再按照标头中指定的算法对标头和载荷加密后再进行base64编码得到 (用于验证此token是否被篡改过)

jwt的**优点**:

- **可扩展性好**，分布式部署的情况下，session需要做多机数据共享，通常存在数据库或者redis，而jwt不需要

- **无状态**，jwt不在服务端存储任何状态，且jwt的载荷中可以存储一些常用的用户信息用于交换，有效使用jwt还可以降低服务器查询数据库的次数

jwt的缺点：

- **安全性低**：jwt的payload使用base64编码，并没有加密，因此jwt中不能存储敏感信息
- **性能问题**：由于是无状态使用jwt，所有的数据都放在jwt中，如果还有自定义字段，载荷会更大，cookie很可能放不下，所以一般都放在local storage里面，并且用户每一次http请求都需要在header里携带jwt，因此使用jwt请求比使用session的开销大
- **无法即时失效**：一旦签发一个jwt，到期之前始终有效，无法中途废弃。用户注销时或者jwt载荷里的用户信息需要更新时，签发了新的jwt，但旧的jwt还没过期，依然可以登录或者服务端拿到的载荷信息是过时的

（为了解决这个问题，我的项目在后端编写了额外的逻辑，**黑名单机制**：一旦签发了新的jwt，就把旧的加入黑名单(比如存到redis里面)，避免被非法再次使用)

- **续签麻烦**：传统的cookie续签方案一般是框架自带，但jwt到期就需要签发新的jwt，因此需要前端需要配置JWT自动续签，在发起请求时如果令牌即将到期，那么就向后端发起续签请求得到一个新的JWT令牌

如果要统计一天之内论坛登录过的人数怎么做？

答：基于**Redis**或者**数据库** 存储登录过的用户ID

- 基于Redis
 - **BitSet**（位集合，精准 但是占用空间更大） setbit 添加用户 bitcount 统计用户
 - **HyperLogLog**（更适合，但是有一些误差） pfadd 添加 pfcount 统计
- 数据库简单存储一张表 每天定时清空再存新的即可

“楼中楼”评论是如何做的？

答：**数据库中额外的引用字段**

- 数据库comment表中有一个quote字段，如果是一级评论（也就是直接评论帖子）则此字段值为-1（无效值）
- 如果是评论的评论，那么子评论的quote字段的值为父评论的ID

delta是什么存储格式？为什么用它？

- delta是quill（强大的开源富文本编辑器）存储文本内容及其样式的一种数据格式，本质上是一种JSON
- delta相比html没有歧义和复杂（没有html的各种标签），缺点是前端从数据库中拿到帖子内容后需要解析
- XSS，跨站脚本攻击（Cross-Site Scripting）（简称与css重合 因此叫xss）
简单来说就是：想办法“教唆”用户的浏览器去执行一些这个网站中不存在的JS代码
- delta可以降低xss攻击的风险，就是因为无法直接在前端页面中展示，需要经过额外处理，处理时可以筛掉非法的标签例如 <script> 等

★为什么要用RabbitMQ？

- ★★任何相关MQ的问题，优先答**MQ的三大作用：异步、解耦、削峰**
- 相比多线程
 - 消息持久化，提高了系统的稳定性，系统down了，但消息存储在外部，不会丢失
 - 削峰填谷，当系统压力较大时，消息可以缓存起来逐渐处理，从而避免系统崩溃或过载
 - 解耦和异步通信，消息队列将不同模块间的通信解耦，降低系统复杂度，且能处理高并发请求
- 对比其他主流消息队列
 - 简单易用：RabbitMQ 提供了简单易用的API，易于上手和部署
 - 灵活的路由和交换机机制：可以根据消息的特性和路由规则进行高效的消息分发
 - 自带的管理界面简单方便，社区活跃度高（文档齐全丰富）
 - 缺点：用erlang开发，难以进行二次开发和维护，吞吐量相比rocketmq/kafka低

为什么选用MinIO存储图片？

- MinIO是一个开源免费的**对象存储**服务器，核心功能是对对象存储，并提供了多种API和SDK访问这些对象
- 参考官方文档就可以使用Minio官方提供的Java SDK进行操作，进行文件的存储
- 如何对接MySQL？

- MySQL中有一张表，表中存储上传图片的用户ID，和图片的存储路径、存储时间等
- 往MinIO里存储图片的同时，往表中插入数据，以便管理员端可以按需依靠索引快速访问图片数据
- 🌟🌟🌟为什么不用MySQL直接存储图片？而非要用Minio？
 - 存储图片（或者任何大型二进制数据）时，使用MySQL直接存储会遇到以下挑战和限制：
 - **性能不佳**：由于图片等二进制数据体积较大，MySQL对于大型二进制数据的存储和检索性能不佳
 - **难以水平扩展**：随着数据量增加，难以水平扩展
 - **难以备份和恢复**：备份和恢复非常耗时，且需要大量存储空间
 - 相比来说，使用对象存储服务（如MinIO）存储图片等大型二进制数据，有以下优势：
 - 上面三条相反，性能好，可扩展，备份和恢复
 - **数据管理**：对象存储服务通常提供了丰富的管理功能，如数据版本控制、访问控制、生命周期管理等，可以帮助用户更好地管理和保护数据

日志 雪花算法

- 雪花算法是一种分布式 ID 生成方案，它可以生成一个长度为 **64 位的唯一ID**，其中包含了**时间戳**、**数据中心 ID** 和**机器 ID** 等信息
- 编写请求日志过滤器，在每个HTTP请求之前，获取请求的接口URL、参数、远程IP地址、用户角色、ID等

服务器监控运维系统

客户端用于定时任务调度的SpringQuartz是什么？怎么用的？

- 是基于Spring的一个功能强大、开源的**任务调度框架**，核心要素主要为：
 - **Job**：定时任务的主体，也是写业务逻辑的地方，如，我们在这里写收集客户端的运行信息
 - **JobDetail**：任务的详细信息，包含了任务的标识ID、任务描述、任务存储方式等等

- Trigger: 触发器, 用来定义任务的执行规则, 如几点执行, 几点结束, 多长时间执行一次等等
- Scheduler: 用来调度任务的, 一个Scheduler就是一个调度容器, 可以调度多个JobDetail和Trigger
- 在项目中, 我主要使用SpringQuartz实现客户端每十秒钟向服务端报告自身信息

客户端的oshi框架是什么? 都采集了什么信息?

- Oshi 框架是一个用于**获取系统信息**的 Java 库
- 它允许开发人员查询和获取有关计算机硬件和操作系统的各种信息, 它和Java自带的包相比起来, 可以获取**更丰富的信息**, 且具有**跨平台性**, 并且使用**简单**
- 我利用Oshi框架获取了客户端的**操作系统名称和架构以及版本**、**CPU名称和核心数** (以及**CPU使用率**)、**内存大小** (包括已用)、**硬盘大小** (包括已用)、**IP地址**, 以及网络的**上传 / 下载速度**, **磁盘读写速度**等

怎么实现的前端shell操作? 简要介绍一下websocket?

- 采用**Xterm.js**作为前端伪终端实现, 然后基于websocket连接实现前端shell操作
- **★WebSocket** 是一种在单个 TCP 连接上进行全双工通信的协议, 它是**应用层**协议
 - **全双工通信**: 客户端和服务端之间可以同时发送和接收数据, 实现真正的双向通信
 - **低延迟**: 使用单个 TCP 连接, 减少了建立连接和发送数据的延迟, 实现更快的数据传输速度
 - **持久连接**: WebSocket 连接是持久性的, 与 HTTP 连接不同, 它不需要在每次通信时重新建立连接, 可以在连接保持的情况下连续发送数据
 - **与HTTP兼容**: WebSocket 握手过程与 HTTP 兼容, 可以通过标准的 HTTP 端口 (通常是 80 和 443) 进行通信, 并且可以在现有的基础设施上轻松部署
- 与WebSocket不同, **Socket套接字**, 是位于**应用层和传输层**之间的一组**接口**, 它是为了方便使用TCP/UDP而抽象出来的, 相当于使用了设计模式中的**外观模式**把复杂的TCP/IP协议族隐藏, 用它即可
- 总的来说, **WebSocket** 是一种高层次的协议, 专门用于实时通信的 **Web 应用程序**, 而 **Socket** 则是一种通用的底层网络通信接口, 可以用于各种网络通信场景。WebSocket 可以看作是在 Socket 的基础上实现的一种高层次的应用层协议

为什么有mysql还要用influxdb存储信息？

- **场景适合：**InfluxDB 是一种专门用于存储和处理时序数据的数据库，非常适合这种物联网存储历史信息场景
- **特殊优化：**它针对时间序列数据进行了优化，提供了高效的存储和查询方式，对于需要处理大量时间序列数据的场景比MySQL更合适
- **按ID以及字段存取方便：**相比MySQL关系型数据库采取表格存储，它使用标签(tag)和字段(field)来存储数据，这种存储方式适合存储大量根据时间且需要根据不同主键的数据，在我的项目中，标签即可代表服务器ID、字段代表每台服务器不同类型的信息

你的管理员用户 子用户 权限校验是怎么做的？

- 在SecurityConfig中配置，任何操作子用户的接口都需要admin角色
- 并且，在数据库的用户表中加入角色和可操作的服务器列表
- 当用户对服务器进行增删改查的时候，对用户进行鉴权，查看其是否拥有此服务器的操作权限

JavaSE

基础

JDK、JRE、JVM有什么区别？

- JVM（Java虚拟机）是Java程序的运行环境，它负责将Java源代码编译成的字节码在各种操作系统上执行，并且不同平台都有对应的JVM实现，从而实现跨平台。

JVM是用于执行Java程序的核心模块，但是它并不能单独安装到我们的电脑上，我们一般都是下载安装JDK或JRE，它们包含了JVM和一些其他的东西

- JRE（Java运行环境）包含了JVM和Java类库，面向客户和使用者，可以用来运行已经编译完成的Java程序。在不需要进行Java开发的情况下，仅需要安装JRE即可运行Java程序即可。

- JDK（Java开发工具包）是Java开发的核心工具包，面向开发者，它不仅包含了JRE的全部内容，还额外包含了编译器（javac）、调试器（jdb）、Java库（Java API）等适用于我们开发人员使用的相关工具。



🌟字节码

什么是字节码？

Java字节码就是程序编译之后的一种中间代码，它由Java编译器生成，可以在Java虚拟机（JVM）上运行。Java源代码在经过编译后会被转换为字节码文件（以.class为文件扩展名），而不是直接编译成平台相关的机器代码。



使用字节码好处是什么？

便于实现**跨平台**，因为JVM上可以读取并执行字节码，而不同平台都有对应的JVM，这样就可以实现跨平台通用了

Java是如何实现跨平台的？

由于Java程序编译后会变成字节码文件，它有着统一的标准，无论在哪里编译出来都是一样的。同时，不同系统都有对应的JVM实现，无论放在哪个平台上的JVM执行，都是一样的，这样就可以做到一次编译到处运行了

Java是编译型还是解释型语言？

先看什么是编译型什么是解释型：

简单来说：编译型语言在运行前需要整体编译成机器语言或者中间代码，而解释型语言则是在运行时逐行解释并执行源代码

- **编译型语言**：优点是计算机直接运行，性能高；缺点是与平台密切相关，在一种操作系统上编译的程序，无法在其他非同类操作系统上运行，比如Windows下的exe程序在Mac上就无法运行。
- **解释型语言**：只需要通过解释器代为执行即可，不需要进行编译。优点是可以跨平台，缺点是需要依靠解释器解释执行，效率肯定没直接编译成机器指令运行的快，并且会产生额外的资源占用。

Java需要先编译为字节码再由JVM解释为机器指令执行，可见，它同时具备两种类型的特性，因此**Java既是编译型也是解释型语言**

JIT即时编译是什么？

JIT编译器（是JVM的一部分）负责**将频繁执行的字节码转换为本地机器代码**，从而提高程序的执行速度

简单来说就是，没有JIT，JVM就一句一句翻译字节码并执行，有了JIT就可以直接大段的直接进行“预翻译”并直接执行，让程序执行的更快。（**编译和解释并存**）

★面向对象

面向对象三大特性都是什么？

封装 继承 多态

什么是重载什么是重写？

重载是多个同名方法，重写是子类Override重写父类方法

- **重载（Overload）** 是指在同一个类中可以存在多个**同名方法**，但参数列表不同，返回类型可以相同也可以不同的情况（注意，不能是参数相同仅返回值不同）在调用重载方法时，根据传入的参数类型和个数来确定调用哪个重载方法（提高代码复用性和灵活性

- **重写（Override）** 是指子类重新定义父类中已经有的同名、同参数列表和返回类型的方法。在使用多态时，调用子类对象的该方法时会执行子类中的方法而不是父类中的方法。重写可以实现子类对父类方法的个性化定制

★final字段有什么作用？

- 修饰变量：修饰的变量不可变，且必须被显式指定初始值

此处不可变指的是**引用不可变** 如果是对象则不可以更改引用 但其中的值还可以修改

- 修饰方法：不可以被重写
- 修饰类：不可以被继承

访问修饰符？

- **private** - 私有，标记为私有的内容无法被除当前类以外的任何位置访问。
- **什么都不写** - 默认，默认情况下，只能被类本身和同包中的其他类访问。
- **protected** - 受保护，标记为受保护的内容可以被类本身和同包中的其他类访问，也可以被子类访问（子类我们会在下一章介绍）
- **public** - 公共，标记为公共的内容，允许在任何地方被访问。

这四种访问权限，总结如下表：

	当前类	同一个包下的类	不同包下的子类	不同包下的类
public	✓	✓	✓	✓
protected	✓	✓	✓	✗
默认	✓	✓	✗	✗
private	✓	✗	✗	✗

子类会不会继承父类的private属性？

父类的private属性子类无法直接访问，但是依然可以通过间接的形式去访问父类定义的属性，如果这个属性没有被继承，那么子类肯定是访问不到这个属性的，所以private是会被继承的

抽象类和接口？

接口只是具有哪些能力的一个代表而已。而抽象类不同，抽象类再怎么抽象，它本质上依然是一个类，类是可以具有属性的（但是二者都不可以被new实例化）

1. 接口

- 只能包含抽象方法
- 接口可以定义属性 但是默认为public static final的（后面的jdk版本）
- 实现接口必须实现所有方法
- 接口支持多继承（implements）

2. 抽象类

- 可以同时包含抽象方法和普通方法
- 可以有构造方法 但是不能new实例化抽象类的对象
- 不支持多继承（extends）

★★★★== 和 equals方法 的区别?

在Java中， == 操作符和 equals() 方法都是用于比较两个对象，但它们在使用上有着本质区别：

1. 使用 == 运算符

- **基本类型**：当用于基本数据类型比较时（如 int , char 等）， == 比较的是两个值是否相同
- **对象类型**：当用于对象比较时， == 比较的是两个对象引用是否指向堆内存中的同一个位置。即它比较的是对象的内存地址，而非内容

2. 使用 equals() 方法

- equals() 方法用于比较两个对象的内容是否相等，默认情况下（例如在 Object 类中定义）， equals() 方法和 == 操作符的行为相同，都是比较对象的内存地址
- 但是，很多内置类（如 String 、 Integer 等包装类）中预先为我们重写了 equals() 方法，使其比较的是对象的内容而不是它们的引用。因此，即使是两个不同的对象，只要它们的内容相同，使用 equals() 方法比较时就会返回 true

★★★★equals 和 hashCode?

什么是hashCode?

在Java中，`hashCode()` 方法是用来返回对象的哈希码的。哈希码是一个整数，用来表示对象的唯一标识。Java中默认的 `hashCode()` 方法是从对象的内存地址计算出来的

为什么要有hashCode?

哈希码主要用于优化具有哈希结构的集合（如 `HashMap` 、 `HashSet` 等）的性能。当向这些集合中添加或检索元素时，Java会使用对象的哈希码来快速定位该对象可能存储的桶位置

为什么重写 equals 时必须重写 hashCode 方法?

为了保证 `equals()` 方法和 `hashCode()` 方法之间的一致性，尤其是在使用 `HashMap` 、 `HashSet` 等集合的时候

1. **维护相等对象的一致性**：如果两个对象的 `equals()` 方法判定相等，那么它们必须具有相同的哈希码，`hashCode()` 方法默认根据对象自身的内存计算独特值，如果没有重写，那么相同类的两个对象的哈希值无论如何也不可能相等（即便两个对象数据一样）
2. **具有哈希结构的集合的实现依赖于hashCode()方法**：如果两个相等的对象却具有不同的哈希码，那么哈希表将无法正确处理它们，可能会导致元素无法被正确存储或检索

Java是 值传递 还是 引用传递?

Java语言是**值传递**，基本数据类型，还是引用数据类型，都是按照值传递的方式进行传递

- **基本类型**: 当你传递一个基本类型给一个方法时，你实际上传递的是该变量值的拷贝，在方法内部对这个值的任何改变都不会影响原始值
- **引用类型**: 当传递一个引用类型（例如对象）时，传递的是对象引用（或者说指针）而不是对象本身，不过，如果在方法内部改变了引用所指向对象的状态（例如，改变了一个对象的属性）那么肯定原始对象也会被改变

什么是深拷贝 浅拷贝?

- **浅拷贝(Shallow Copy)** 浅拷贝是指创建一个新对象，但只复制原始对象的基本数据类型的数据值，而引用类型数据仍然指向原始对象的内存地址。（根据B复制了A但是B指向A，B修改会影响A）

- **深拷贝(Deep Copy)** 深拷贝同样是指创建一个新对象，但是此时复制的不仅仅是基本数据类型数据，同时引用类型所指向的对象也会被复制，得到一个新的对象并引用（根据B完全创造了A，AB之间互不关联了）

常用类

String类可以被继承吗？

不行。String类在Java中被声明为 **final类**，这意味着它不能被继承

String StringBuilder StringBuffer？

都是用于处理字符串的类，都实现了CharSequence接口

类名	可变性	线程安全性	性能
String	不可变	线程安全	低
StringBuffer	可变	线程安全	低
StringBuilder	可变	非线程安全	高

- String：不可变，故而线程安全，但频繁的字符串拼接操作会产生大量临时对象，影响性能
- StringBuffer：方法都使用 `synchronized` 关键字进行同步保证线程安全，但性能相对较低
- StringBuilder：没有进行同步，不具备线程安全，性能更好，但需要自行保证线程安全

String str1 = new String("abc")和String str2 = "abc" 和 区别？

运行时常量池位于**方法区**，字符串常量池位于**堆**，对象也位于**堆**

- 两个语句都会去**字符串常量池**中检查是否已经存在“abc”，如果有则直接使用，如果没有则会在常量池中创建“abc”对象

- 对于 `String str1 = new String("abc")`，无论字符串常量池中是否已经存在 `"abc"` 字符串，都会在堆内存中创建一个新的对象。因此，`str1` 和任何其他 `"abc"` 字符串对象都不会相等（除非是同一个对象的引用）

String的intern方法有什么作用？

它返回的是字符串常量池中的引用

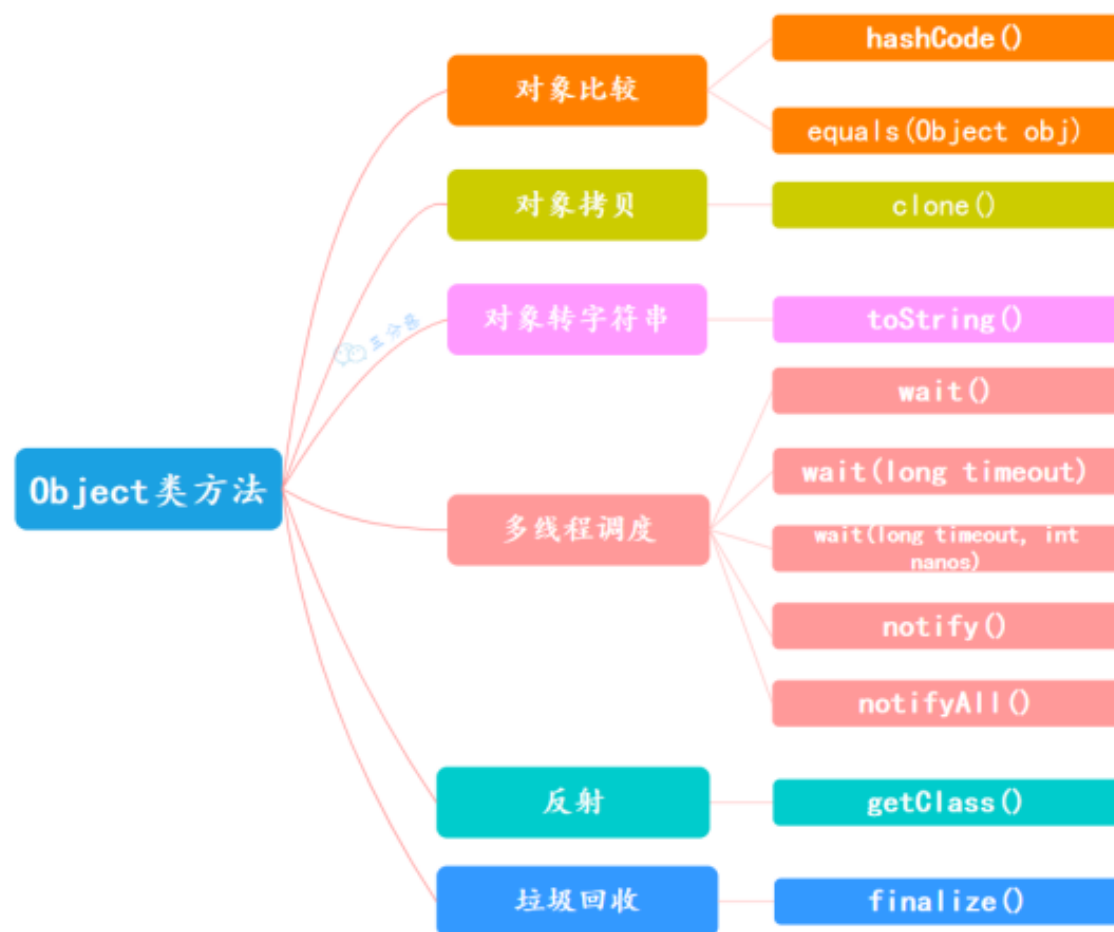
- 如果常量池中已经存在等于此字符串对象的字符串，则返回常量池中的引用
- 否则，将此字符串对象添加到常量池中，并返回该对象的引用

```
String s1 = new String("hello");
String s2 = "hello";

String s3 = s1.intern(); // 返回常量池中的引用
System.out.println(s1 == s2); // 输出 false，因为s1和s2是两个不同的对象
System.out.println(s2 == s3); // 输出 true，因为s3是从常量池中返回的引用，与s2相同
```

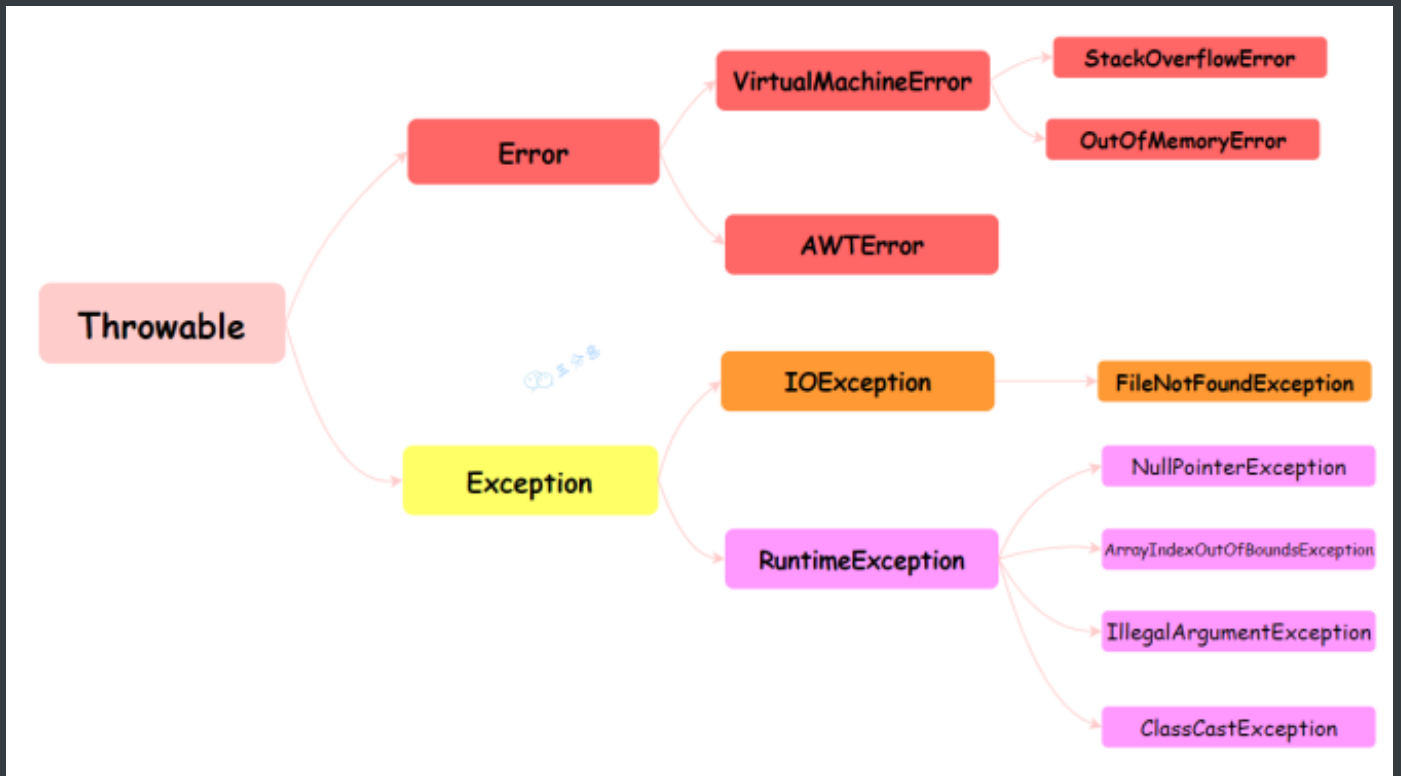
Object类的常见方法？

Object 类是一个特殊的类，是所有类的父类，也就是说所有类都可以调用它的方法。它主要提供了以下 11 个方法，大概可以分为六类：



异常

值得注意的是：即便 try/catch 里面 return 了 还是会执行 finally



序列化

什么是序列化？反序列化？

序列化就是把Java对象转为二进制流，方便存储和传输

所以反序列化就是把二进制流恢复成对象

Serializable接口有什么用？

Serializable接口是Java提供的一个标记接口，用于标识类可以被序列化。实现 Serializable接口的类可以将对象转换为字节流进行传输和存储，实现数据的持久化和 跨平台的数据交换

serialVersionUID 有什么用？

serialVersionUID是Java中用于序列化和反序列化的一个特殊字段。它是一个静态常量，用于标识序列化类的版本号，它是一个64位的long类型数字，具有以下作用：

- 版本控制
- 兼容性

Java 序列化不包含静态变量？

Java序列化只会保存对象的非静态成员变量，不包含静态变量

如果有些变量不想序列化，怎么办？

使用 transient 关键字修饰。

反射

什么是反射？

动态地获取类信息、创建类实例、调用类方法这时候就要用到反射

反射最核心的四个类：



反射的应用场景

像 Spring / Spring Boot、MyBatis 等等框架中都大量使用了反射机制

Spring里的很多注解，它真正的功能实现就是利用反射

反射的实现原理？

基于JVM的类加载机制和类对象模型

在Java 中，每个类都有一个对应的Class对象，该对象包含了类的名称、字段、方法等信息。通过反射，我们可以获取到这些信息，并在运行时动态地操作类和对象

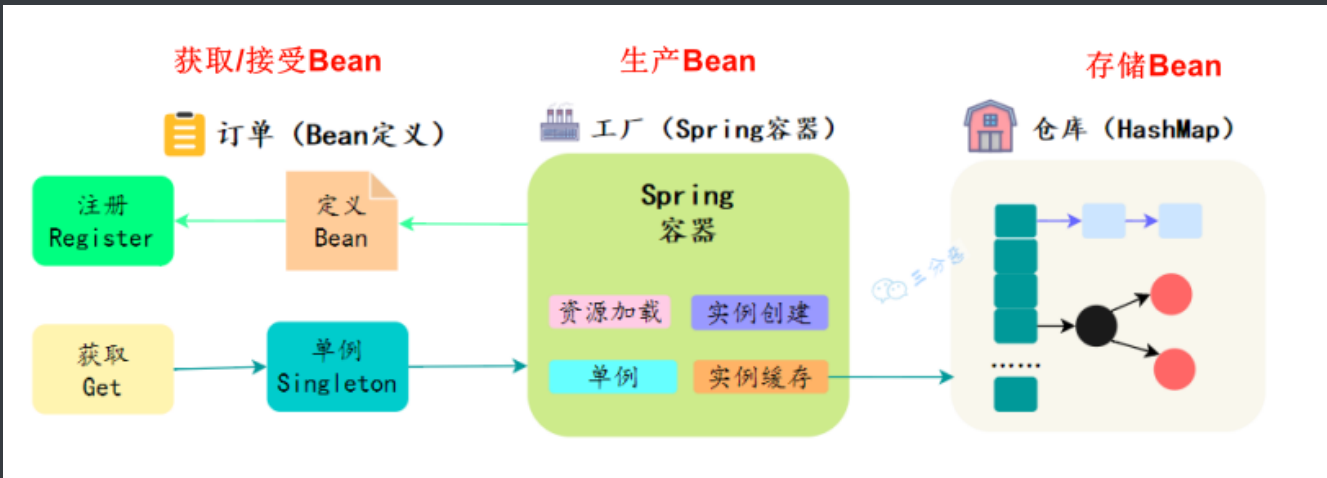
Spring

Spring 是一个轻量级、非侵入式的控制反转 (IoC) 和面向切面 (AOP)的框架

IOC

使用IOC最主要的作用就是解耦，不再关心对象之间的依赖

- IOC(Inversion of Control)：控制反转，控制对象生命周期的不再是引用它的对象，而是容器，由容器来负责控制对象的生命周期和对象间的关系
- 比如当我们需要一个接口的实现时，由它根据配置文件来决定到底给我们哪一个实现类，这样，我们就可以不用再关心我们要去使用哪一个实现类了
- IOC的简单原理：



依赖注入，指的是容器在实例化对象的时候把它依赖的类注入给它

有哪些方法可以依赖注入？

- 构造方法注入
- 属性注入
- 工厂方法注入
 - 非静态工厂
 - 静态工厂

@Autowired和@Resource有什么区别？

特点	@Autowired	@Resource
来源	Spring框架	Java EE
匹配方式	按类型匹配	按名称匹配
使用方式	构造方法、属性、方法、参数上	属性、setter方法上
优先级	按类型匹配，可使用@Qualifier指定名称	按名称匹配，可使用name属性指定名称
兼容性	只能在Spring环境中使用	可在Spring环境和Java EE容器中使用

循环依赖？

简单来说就是自己依赖自己，或者和别的Bean相互依赖（只存在于**单例模式**，原型模式会抛出异常）

注意：Spring可以自动解决：AB均采用属性注入、先构造器注入后属性注入，无法解决均构造器注入

Spring解决循环依赖主要依赖于其内部的“**三级缓存**”机制

- Spring容器初始化Bean的过程：实例化、属性赋值（注入就发生在这一步）、初始化
- “三级缓存”机制
 - **一级缓存**：存放完全初始化好（三个步骤都完成）的单例Bean对象
 - **二级缓存**：存放提前曝光的原始（仅完成了实例化，没有属性赋值）单例Bean对象
 - **三级缓存**：存放能够创建Bean实例的ObjectFactory对象
- 如何用三级缓存解决循环依赖？（举例当A、B两个类都用属性注入发生循环依赖时）
 - 创建A实例，把A对象工厂放入三级缓存，表示A开始实例化了
 - A注入属性时，发现依赖B，但B还没有创建，就去实例化B
 - 同样，B注入属性发现依赖A，就从缓存里找A，依次从一级到三级缓存查询A，从三级缓存通过对象工厂拿到A，但是A不太完善，就把A放入二级缓存，同时删掉三级缓存中的A，此时B完成属性注入，把完全初始化好的B放入一级缓存
 - 接着A继续完成属性赋值，从一级缓存拿到完全初始化的B，删除二级中的A，放入一级
 - 最后一级缓存中保存着完全初始化好的A和B对象

所以，我们就知道为什么Spring能解决属性注入的循环依赖了，因为实例化和属性赋值是分开的，所以里面有操作的空间。如果都是构造器注入，那么都得在实例化这一步完成注入，所以自然是无法支持

AOP

- 简单说，就是把一些业务逻辑中的相同的代码抽取到一个独立的模块中，让业务逻辑更加清爽。
- 例如，每个模块都要有日志记录、数据校验等功能，就可以分离出来，在各个模块合适的地方植入即可
- AOP的核心是**动态代理**
 - Spring在实现AOP时，会根据目标类是否实现接口自动选择使用JDK动态代理或CGLIB代理

- 默认情况下，如果目标类实现了接口，Spring会选择JDK代理；否则，会选择CGLIB代理
- 用户也可以通过配置强制指定使用某种代理方式

JDK动态代理适用于那些已实现接口且对代理类创建速度要求不高的情况，尤其适合遵循面向接口编程原则的项目。而CGLIB代理则适用于需要代理未实现接口的类，或者对代理类创建速度有较高要求的场景

JDK动态代理

基于**接口**实现，目标类必须至少要实现一个接口

▪ 原理

- 定义一个实现 `java.lang.InvocationHandler` 接口的类，该类包含一个 `invoke` 方法，用于处理代理对象上的方法调用，在此方法内，可以编写横切逻辑，并最终调用目标对象的实际方法
- 通过代理对象调用方法时，实际上会触发 `InvocationHandler` 的 `invoke` 方法，从而执行预设的横切逻辑和目标方法
- **优点**：无需额外依赖、稳定性、面向接口有助于解耦
- **缺点**：仅能代理实现了接口的类、性能相较于CGLIB略逊一筹

CGLIB代理

CGLIB并不依赖于目标类是否实现了接口，而是基于**继承**

▪ 原理：

- 基于**ASM字节码操作框架**，能够在运行时动态生成目标类的子类作为代理对象
- 这个子类重写了目标类的部分或全部方法，从而在方法调用时插入额外的操作
- **优点**：无接口限制（适用范围更广）、性能比JDK动态代理快
- **缺点**：第三方依赖、继承局限（无法代理`final`类、无法覆盖`final`方法）

Spring事务传播行为？

存在事务与否，代表着
它的方法是否由另外一个事务的方法调用

Spring事务传播行为

三分四

默认 REQUIRED

存在事务就加入（共享）
不存在就自行创建新的事务

SUPPORTS

存在事务就加入，不存在就非事务执行

MANDATORY

必须存在事务并加入，没有就报错

REQUIRES_NEW

无论如何都创建新事务
且与外部事务完全独立

NOT_SUPPORTED

存在事务就挂起，
先以非事务执行本方法，
且不影响外部事务执行

NEVER

存在事务就报错，
没有事务就以非事务执行

NESTED

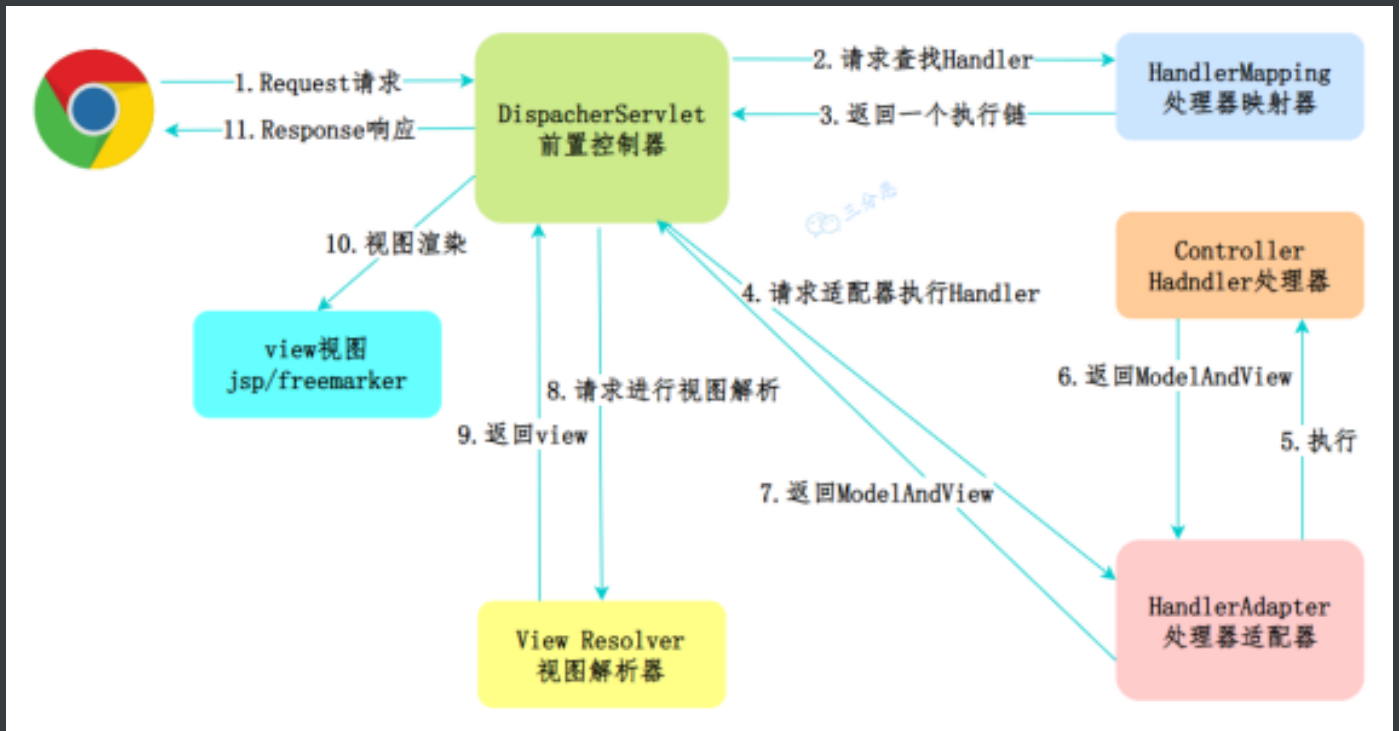
存在事务就创建一个子事务
外部提交内部才能真正提交，
不存在事务就自行创建新的事务

导致Spring声明式事务失效的原因？

- @Transactional 应用在非 public 修饰的方法上
- @Transactional 注解属性 propagation（传播行为）设置错误
- @Transactional 注解属性 rollbackFor（事务回滚的异常类型）设置错误
- 同一个类中方法调用，导致@Transactional失效

Spring MVC

工作流程



Spring Boot?

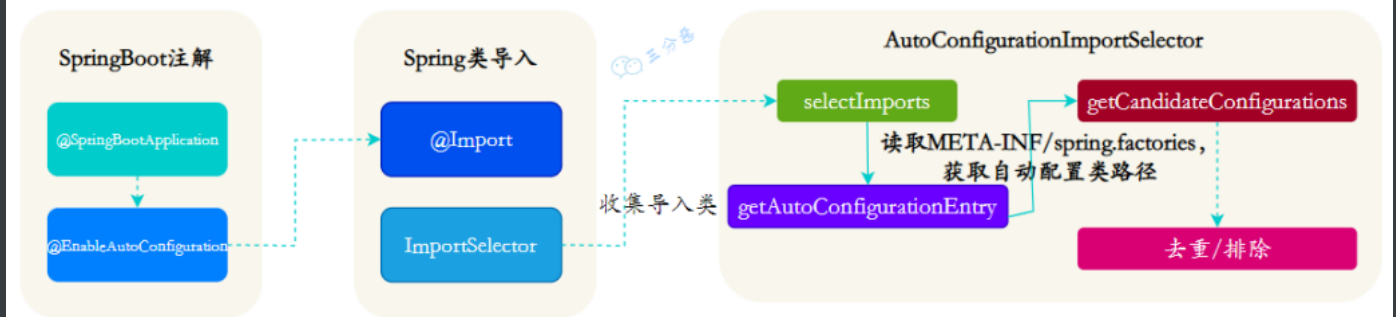
介绍一下SpringBoot，它有哪些优点？

- Spring Boot 以约定大于配置核心思想开展工作，相比Spring具有很多优势
- 内置servlet容器，不需要在服务器部署 tomcat。只需要将项目打成jar包，使用java -jar xxx.jar一键式启动项目
- SpringBoot提供了starter，把常用库聚合在一起，简化复杂的环境配置，快速搭建spring应用环境
- SpringBoot无需再像Spring一样配置一堆繁琐的xml文件
- 可以快速整合常用依赖，创建独立运行的spring项目

SpringBoot自动配置原理了解吗？

SpringBoot开启自动配置的注解是@EnableAutoConfiguration，启动类上的注解@SpringBootApplication是一个复合注解，包含@EnableAutoConfiguration

SpringBoot自动配置原理



starter是什么？如何编写一个自己的starter？

Spring Boot Starter 是 Spring Boot 框架的一个重要特性，它旨在简化应用程序的初始设置和依赖管理。一个 Starter 是一个特殊的模块或库，它包含了实现特定功能所需的所有依赖项、自动配置类和可能的默认配置

1. 创建一个新的 Maven 项目
2. 添加 Spring Boot 相关依赖
3. 创建自动化配置类
4. 定义属性类和默认配置
5. 打包并发布到私有或公共仓库

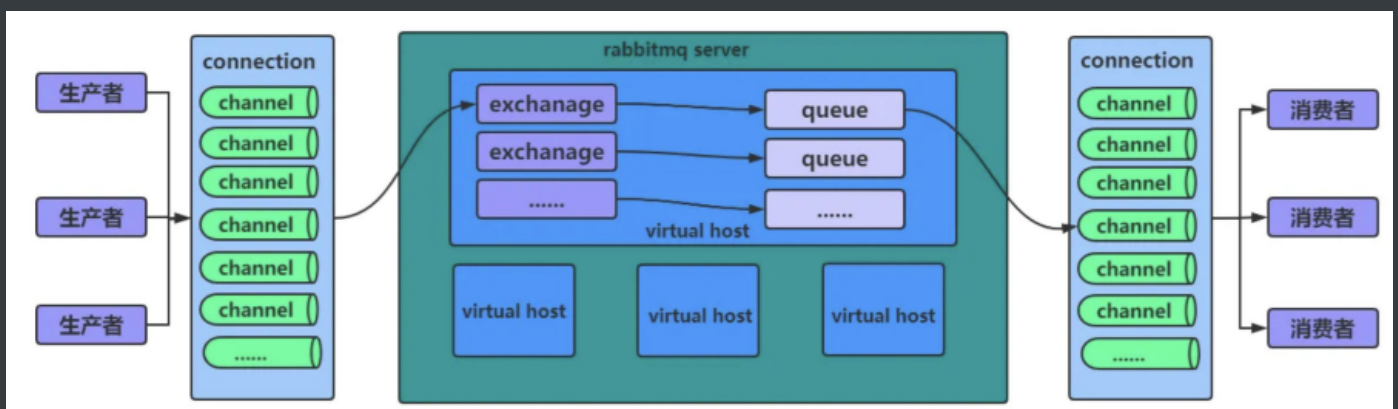
Spring Boot Starter 的主要特点包括：

1. **依赖聚合**：Starter 封装了一组相关的库及其依赖，确保版本兼容性和减少版本冲突。例如，`spring-boot-starter-data-jpa` 不仅包含 Spring Data JPA，还包含了 Hibernate、Spring ORM 等必要依赖
2. **自动配置**：每个 Starter 通常会附带一个或多个 `@Configuration` 类，这些类使用 Spring Boot 的条件化配置机制（`@ConditionalOnClass`，`@ConditionalOnMissingBean` 等注解）来自动配置 Bean 和属性，当满足特定条件（如类路径上有某些库存在）时，自动配置就会生效

3. **默认配置**：Starter 可能会提供默认的配置属性，这些属性可以在 `application.properties` 或 `application.yml` 文件中轻松覆盖。Spring Boot 的 `spring-boot-configuration-processor` 插件还会为这些属性生成元数据，使得 IDE 能够提供智能提示和文档链接

RabbitMQ

介绍一下RabbitMQ?



exchange都有哪些类型?

- **direct**：（默认的）**全文匹配**，按照routingkey发到指定队列
- **topic**：**模糊匹配**，根据通配符可将一个消息发到多个队列
- **fanout**：**消息广播**，直接将消息发到该交换机绑定的所有队列
- **header**：**自定义消息头的key-value来判断是否发送到绑定的队列**（性能很差，一般不用）

什么是死信队列?

消费失败的消息存放的队列

消费失败的原因：

- 消息被拒绝且消息没有重新入队

- 消息超时未被消费（超过设置的TTL）
- 队列满了（达到最大队列长度）

★如何保证消息可靠性（不丢失）？

消息丢失场景：生产者 -> MQ、MQ本身存储的消息、MQ -> 消费者

三个方面来解决：**生产者确认机制**、**消费者手动确认消息**和**消息持久化**

生产者确认机制

- **事务机制**：在一条消息发送之后会使发送端阻塞，等待RabbitMQ的回应，之后才能继续发送下一条消息（性能差）
- **生产者确认机制**：只要消息成功发送到交换机之后，RabbitMQ就会发送一个ack给生产者（即使消息没有Queue接收，也会发送ack），如果消息没有成功发送到交换机，就会发送一条nack消息，提示发送失败

路由不可达消息

生产者确认机制 只能确保**消息正确到达交换机**，对于从交换机路由到queue失败的消息，就会被丢弃，导致消息丢失

对于**不可路由的消息**，有两种处理方式

- **Return消息机制**：提供回调函数，当消息从交换机路由到queue失败时回调这个方法（持久化或者重发）
- **备份交换机**：相当于是一个exchange，当消息没有匹配到queue时就自动转到备份交换机对应的queue，保证消息不丢失

消费者手动消息确认

消费者默认采用自动ack，也就是消费者一收到消息就向MQ确认，MQ就会移除这条消息，但有可能消费者收到消息还没来得及处理时MQ就宕机了，因此采用**消费者手动消息确认**

- 消费者处理完消费逻辑之后再给MQ回复ack，表示消息已经成功消费，可以移除
- 当消费者消费失败时，给MQ回复nack，再根据MQ配置决定消息重新入队还是移除，或者进入死信队列
- （只要没收到消费者的确认，MQ就会一直保存这条消息，但也不会重新入队或者给其他的消费者消费）

消息持久化

默认情况下，RabbitMQ的消息是不持久化的

内存中的消息持久化到硬盘上，即使MQ重启也不会丢失消息

消息持久化需要满足：消息、队列、交换机全部设置了持久化

- 当发布一条消息到交换机上时，Rabbit会**先把消息写入持久化日志**，然后才向生产者发送响应
- 一旦从队列中消费了一条消息的话并且做了确认，RabbitMQ会在持久化日志中移除这条消息
- 在消费消息前，如果RabbitMQ重启的话，服务器会自动重建交换机和队列，加载持久化日志中的消息到相应的队列或者交换机上，保证消息不会丢失

镜像队列（集群）

🌟RabbitMQ如何保证高可用？

RabbitMQ基于**主从**做高可用性，有三种模式：单机、普通集群、镜像集群

普通集群只是提高系统吞吐量，让集群中多个节点服务某个Queue的读写操作

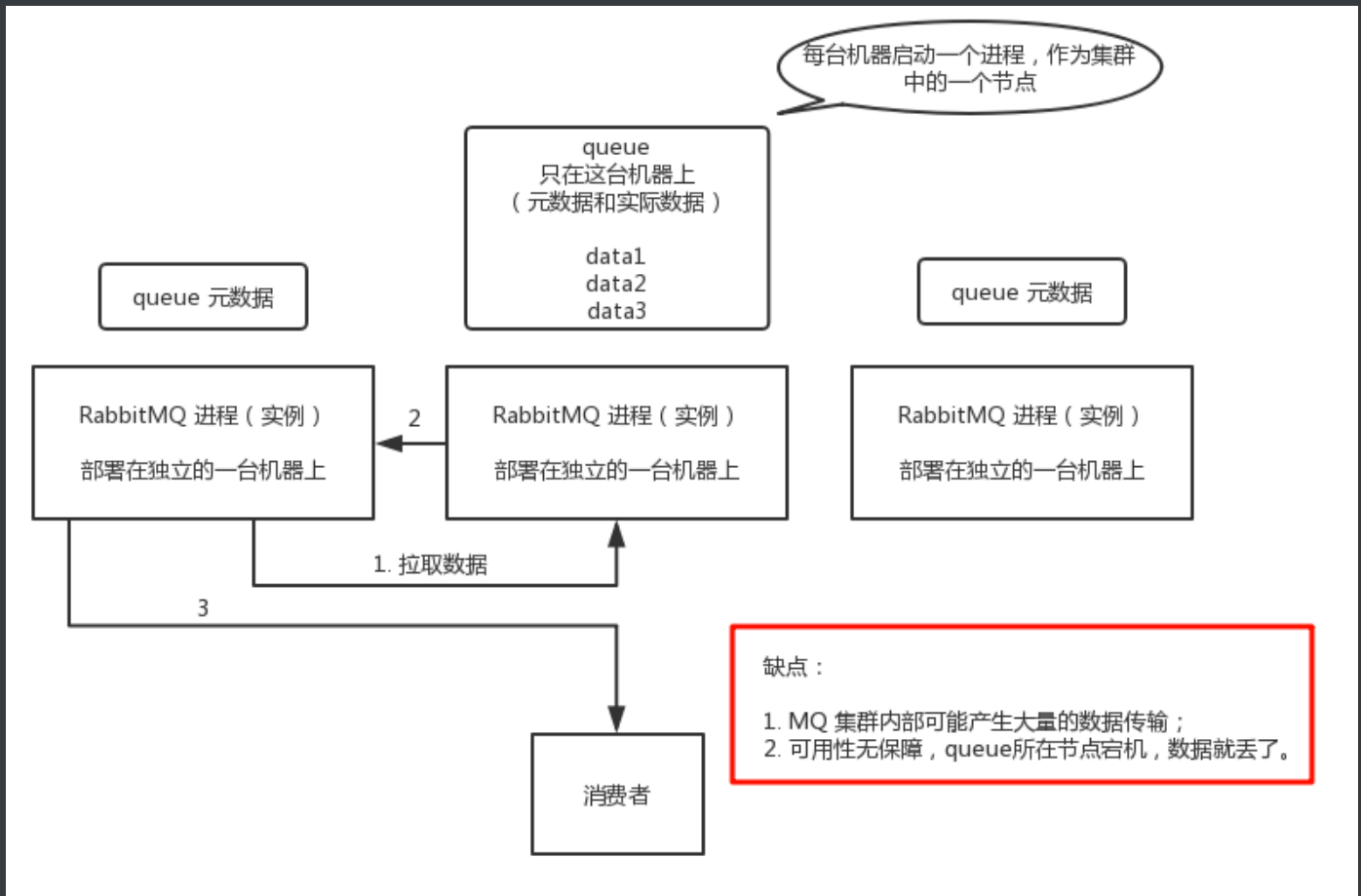
真正实现RabbitMQ高可用的是**镜像集群模式**

普通集群

在多台机器上启动多个RabbitMQ实例，但你创建的 queue 只会放在一个实例上，但是每个实例都会同步这个queue的**元数据**（可以认为是queue的一些配置信息等，通

过元数据可以找到queue所在的mq实例），消费的时候，如果连接到别的实例，就从queue所在实例上拉取消息过来

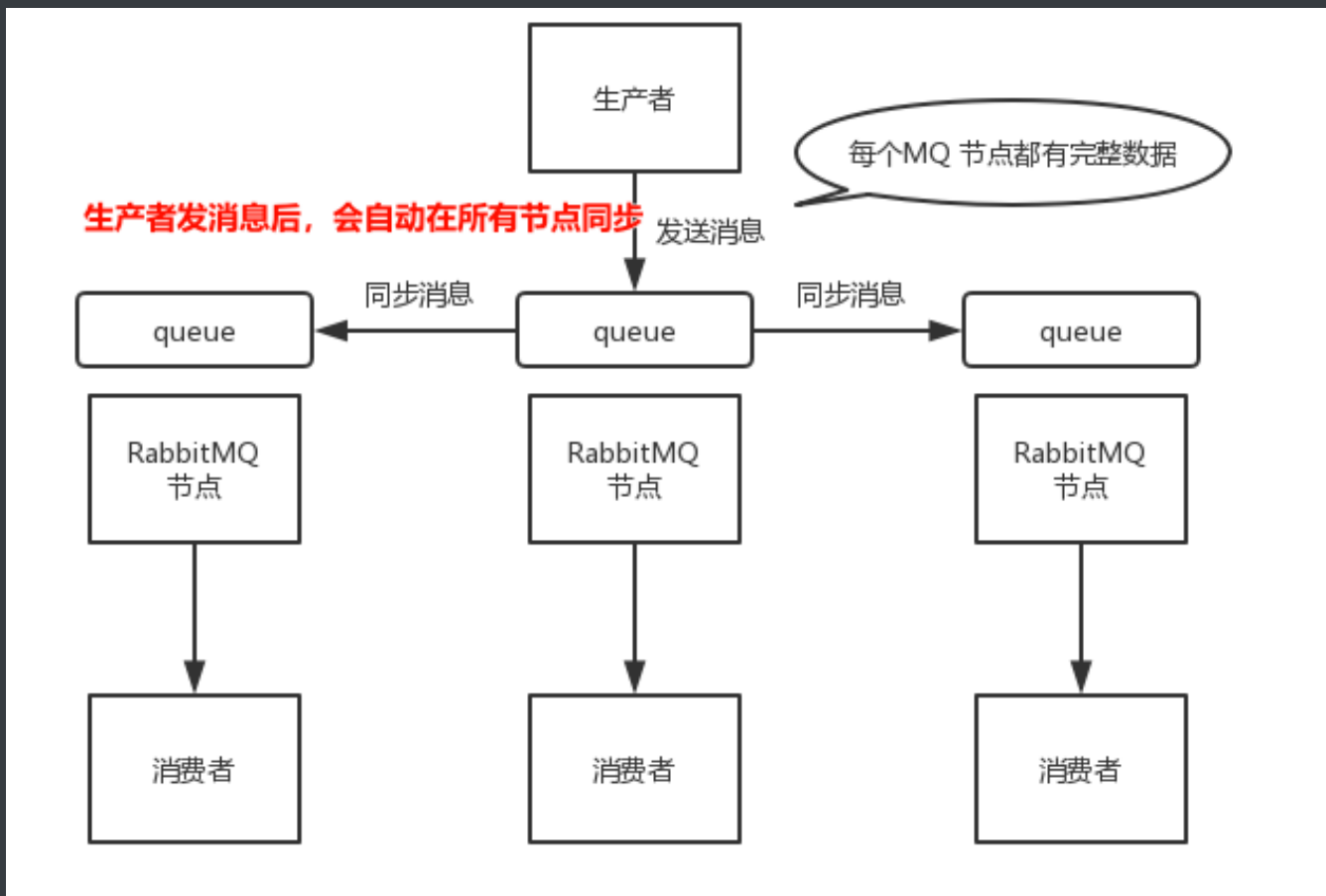
该方案主要是**提高吞吐量**，无法保证高可用



🌟 镜像集群（真正高可用）

与普通集群不一样的是，创建的 queue，无论元数据还是消息都会存在于多个实例上

每次写消息到queue的时候，都会自动把消息同步到多个实例的 queue 上



- 好处
 - 任何一个节点宕机了，其他的节点还保存有 queue 的完整数据
- 坏处
 - 性能消耗太大，所有节点都要进行消息同步
 - 没有扩展性可言，如果一个queue负载很重，就算增加节点，新增节点还是包含所有数据，无法线性扩展

如何保证不重复消费？

消费重复的原因有两个：生产时消息重复、消费时消息重复

解决方法：

- 保证消费逻辑的幂等
- 给消息加全局唯一ID，消费前查询是否消费过（可以存入 redis）

如何实现顺序消息？

1. 传统 / 简单 做法：单一消费者，缺点是并发度较高的时候效率很低

顺序消费和多线程本身就矛盾，多线程意义就是不在乎顺序、所以才多线程，现在既要顺序又要多线程

2. 解决办法：将需要保证顺序的消息放在一块，进行分组然后由单一的消费者消费
相当于给单独需要保证顺序的消息加标识，放在单独的queue里由单一消费者消费，其余queue正常多线程消费

如何实现延迟消息？

1. 传统 / 简单 做法：死信队列，缺点是必须保证所有延迟消息延迟消息一致（如下）



2. 另一种方法：本地存库，然后通过轮询来重新发送一次消息，缺点是性能上有损耗（但是更加灵活，算是取舍）

如何处理消息堆积？

原因（一般就如下两种）：消费者消费的慢、消费出现问题导致nack的消息太多

解决办法：

- 增加消费者（数量）
- 消费过程改为异步或多线程、增加消费者prefetch的数量（增强消费者能力）
- 新开topic，消费不了的发的新topic上，然后再单开一些消费者在这个新topic消费

🌟生产者生产的太多了，如何确保不把rabbitmq服务器挤爆？

1. 首先，先横向扩展，搭建集群
2. 其次，采用限流（如令牌桶、漏桶等）的方式，用redis进行缓存这些消息
3. 最后，再按一定正常的速率分发给rabbitmq

MySQL

数据库引擎

主要就是MyISAM和InnoDB的区别

MyISAM适合查询以及插入为主的应用，InnoDB适合频繁修改以及涉及到安全性较高的应用

1. **事务支持**：MyISAM不支持，InnoDB支持
2. **最小锁粒度**：MyISAM只能添加表级锁，InnoDB支持行级锁
3. **主键必需**：InnoDB必需主键（没有则自己设一个隐藏的），MyISAM无必需
4. **表的具体行数**：MyISAM会在表里自存行数，InnoDB不会
5. **外键支持**：MyISAM不支持，InnoDB支持

★索引

什么是索引?

索引就好像我们**书的目录**，每本书都有一个目录用于我们快速定位我们想要的内容在哪一页，索引也是，通过建立索引，就可以根据索引来快速找到想要的一条记录，大大提高查询效率

索引类型

- 单列索引
 - **NORMAL**：普通的索引类型，完完全全相当于一本书的目录
 - **UNIQUE**：唯一索引。一旦建立唯一索引，那么整个列中将不允许出现重复数据。每个表的主键列，都有一个特殊的唯一索引，叫做 Primary Key，它不仅仅要求不允许出现重复，还要求不能为NULL，它还可以自动递增。每张表可以有多个唯一索引，但是只能有一个Primary索引

- **SPATIAL：空间索引**，空间索引是对空间数据类型的字段建立的索引，MySQL中的空间数据类型有4种，分别是GEOMETRY、POINT、LINESTRING、POLYGON，不是很常用，这里不做介绍
- **FULLTEXT：全文索引**（MySQL 5.6 之后InnoDB才支持），它是**模糊匹配**的一种更好的解决方案，它的效率要比使用 like % 更高，并且它还支持多种匹配方式，灵活性也更加强大。只有字段的数据类型为 char、varchar、text 及其系列才可以建全文索引
- 组合索引
 - 实际上就是将多行捆绑在一起，作为一个索引，它同样支持以上几种索引类型
 - 注意组合索引在进行匹配时，遵循**最左原则**

索引方法

索引方法不是类型，它是实现上述索引类型的具体**底层实现**，主要有 **hash** 和 **b+tree**

explain关键字

分析select查询语句到底有没有通过索引进行匹配

主要关心字段：

- **type**：MySQL决定如何查找对应的记录，效率从高到低：system > const > eq_ref > ref > range > index > all
- **possible_keys**：执行查询时可能会用到的索引
- **key**：实际使用的索引
- **rows**：扫描的行数

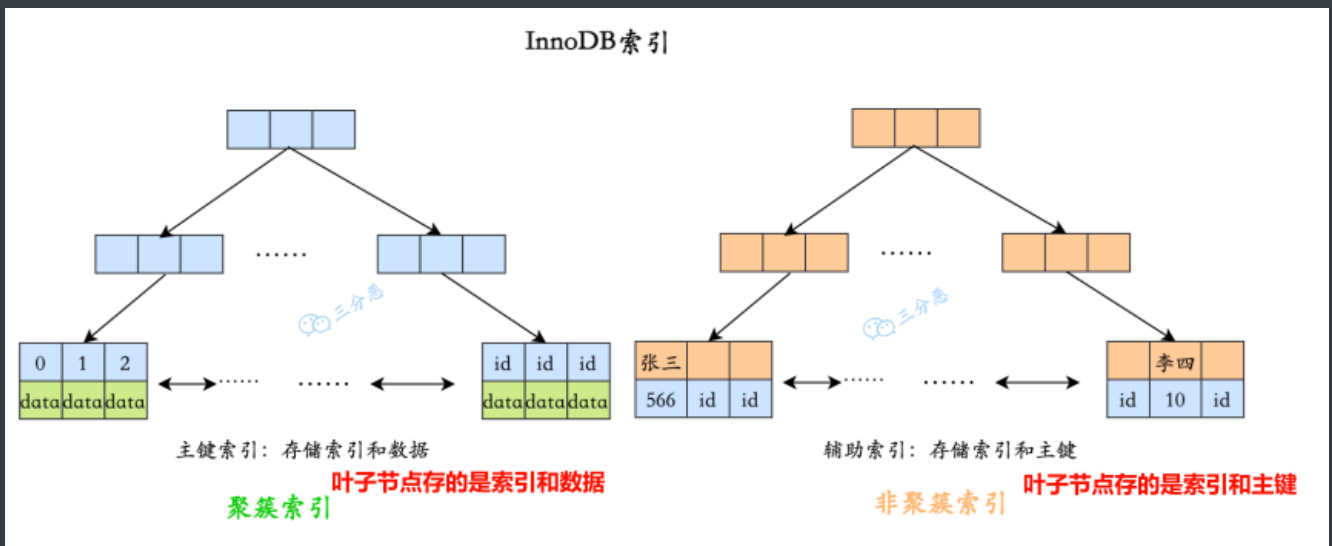
★聚簇索引与非聚簇索引的区别？

聚簇索引不是一种新的索引，而是而是一种数据存储方式

聚簇表示数据行和相邻的键值紧凑地存储在一起

可以这么说：

- 索引的数据结构是树，聚簇索引的索引和数据存储在一棵树上，树的叶子节点就是数据，非聚簇索引索引和数据不在一棵树上



- 聚簇索引：物理存储按照索引排序；非聚簇索引：物理存储不按照索引排序
- 由于上述排序，因此聚簇索引一个表中只能拥有一个聚簇索引，而非聚簇索引一个表可以存在多个

回表是什么？

特指在使用非聚簇索引（辅助索引）执行查询时，需要根据索引查找到的主键值，返回到对应的聚簇索引中去获取完整的数据行的过程，它比基于聚簇索引（主键索引）的查询多扫描了一棵树，这个过程就叫回表

什么时候需要 / 不需要创建索引？（创建索引注意什么？）

- 索引建立在频繁查询，而不是频繁增删改的字段上（索引提高查询效率，但有所降低增删改效率，因为需要动态维护索引）
- 控制索引的数量
- 不要在低区分度的字段上创建索引，且创建组合索引时把区分度高的字段放在前面
- 不使用无序的值作为索引，推荐有唯一性的作为索引

有什么索引优化的方法？（也可以说上面的）

- 前缀索引优化
- 覆盖索引优化（索引包含查询所需全部列）
- 主键最好是自增的
- 防止索引失效（失效情况见下）

索引失效都有哪些情况？

- 使用左或者左右模糊匹配时，也就是 `like %xx`、`like %xx%`（百分号只在右边不会失效）
- 查询条件中对索引列做了计算 / 函数运算
- 联合索引时未遵循最左匹配原则
- `where` 查询时，索引列 `or` 非索引列

为什么MySQL要采用B+树作为默认索引底层结构？（对比B+Tree/B Tree/二叉树/hash）

B+ 树非叶子节点仅存储索引，所有数据都在叶子节点

B 树 每个节点都可以存索引和数据

- **B+Tree vs B Tree：**
 - 存储相同数据量的情况下，B+Tree树高度比B Tree低，磁盘I/O次数更少
 - B+Tree叶子节点用双向链表串起来，适合范围查询，B Tree无法做到
- **B+Tree vs 二叉树**
 - 随着数据量的增加，二叉树高度越来越高，磁盘I/O次数也更多，B+Tree在千万级数据量下，高度依然保持在3-4层
- **B+Tree vs Hash**
 - Hash 等值查询效率巨高（近乎O(1)），但是无法做范围查询

事务

注意：InnoDB支持事务、MyISAM不支持

介绍一下事务四大特效，再说说什么靠什么保证的？

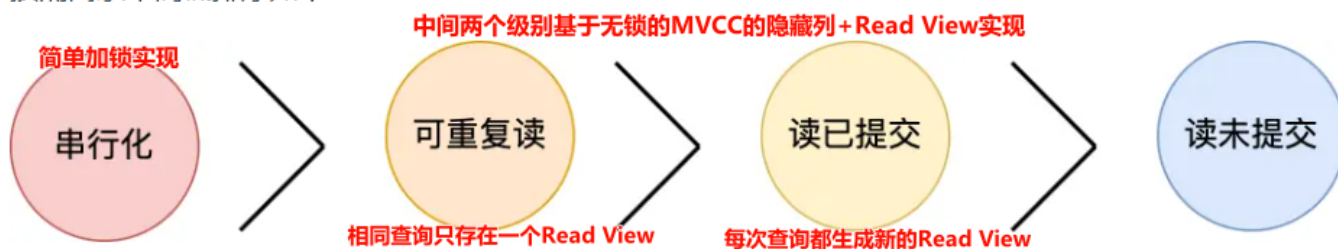
- A原子性：要么都做，要么都不做（通过**undo log(回滚日志)**保证）
- C一致性：事务操作前后，数据库保持一致性状态
- I隔离性：多个事务并发互不影响（通过**MVCC**或者**锁机制**保证）
- D持久性：事务提交后，对数据的修改是永久的（通过**redo log(重做机制)**保证）

并发事务会引发什么问题？

- 脏读：读到其他事务未提交的数据
- 幻读：前后读取的记录数量不一致
 - 针对**快照读**（普通select语句）：通过**MVCC**方式解决幻读
 - 针对**当前读**（select...for update语句）：通过**next-key lock**（记录锁+间隙锁）解决幻读
- 不可重复读：前后读取的数据不一致

事务隔离级别？

按隔离水平高低排序如下：



针对不同的隔离级别，并发事务时可能发生的现象也会不同。



★MVCC

多版本并发控制，用于解决并发访问下的读一致性问题，它通过维护数据的历史版本来实现

几个关键点：隐藏字段、undo日志、版本链、快照读和当前读、Read View

快照读和当前读

- **快照读**，读取的是记录的可见版本 (有可能是历史版本)，不用加锁
- **当前读**，读取的是记录的最新版本，并且，当前读返回的记录，都会加上锁，保证其他事务不会再并发修改这条记录

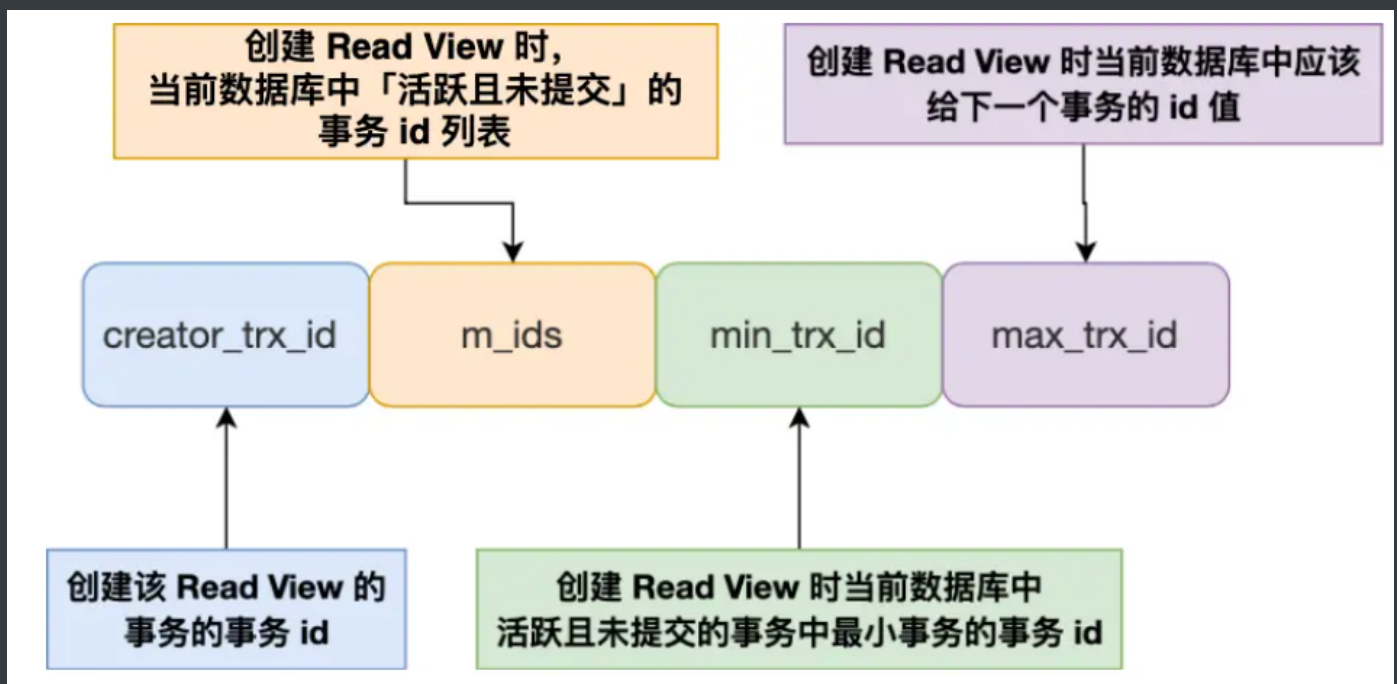
快照读：简单的select操作，属于快照读，不加锁。

- `select * from table where ?;`

当前读：特殊的读操作，插入/更新/删除操作，属于当前读，需要加锁。

- `select * from table where ? lock in share mode;`
- `select * from table where ? for update;`
- `insert into table values (...);`
- `update table set ? where ?;`
- `delete from table where ?;`

Read View



以上四个重要字段：（都是创建该Read View时的数据）

1. 活跃且未提交的事务id列表
2. 活跃且未提交事务中ID最小的事务（也就是最先的）

3. 下一个应该赋的事务ID（也就是最大事务ID+1）
4. 创建该Read View的事务ID

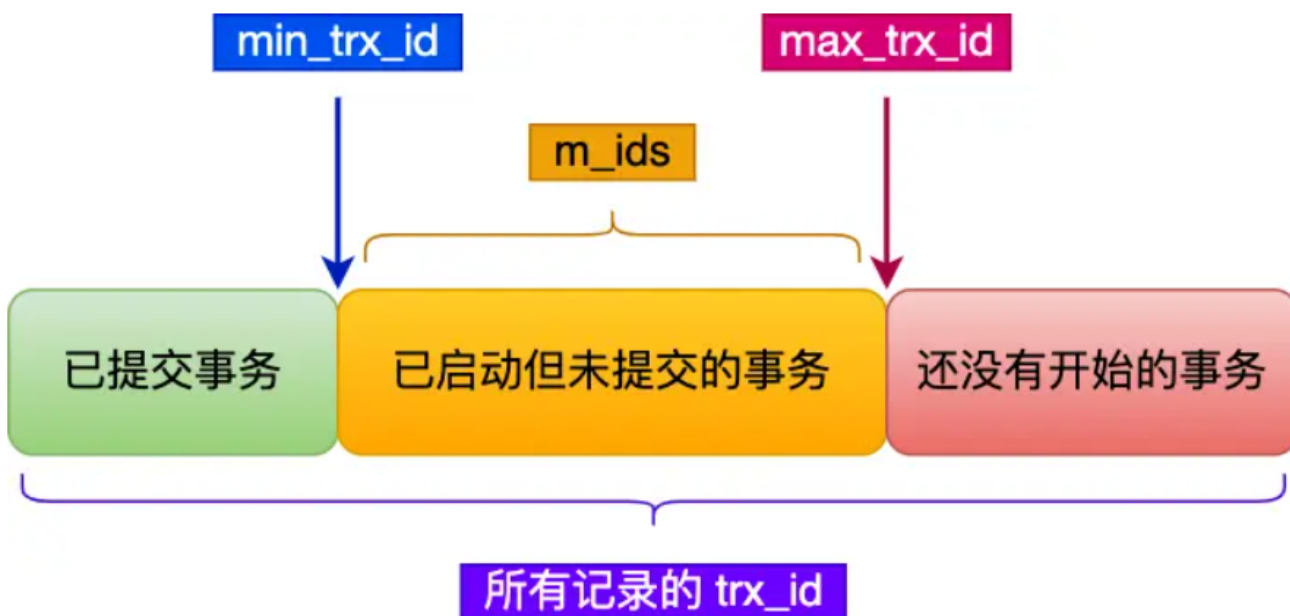
隐藏列

隐藏列包含两个：

1. `trx_id`：操作该行数据的事务ID
2. `roll_ptr`：历史版本信息指针（指向旧版本记录）

实现过程

在创建 Read View 后，我们可以将记录中的 `trx_id` 划分这三种情况：



一个事务去访问记录的时候，除了自己更新的总是可见之外，还有以下情况：

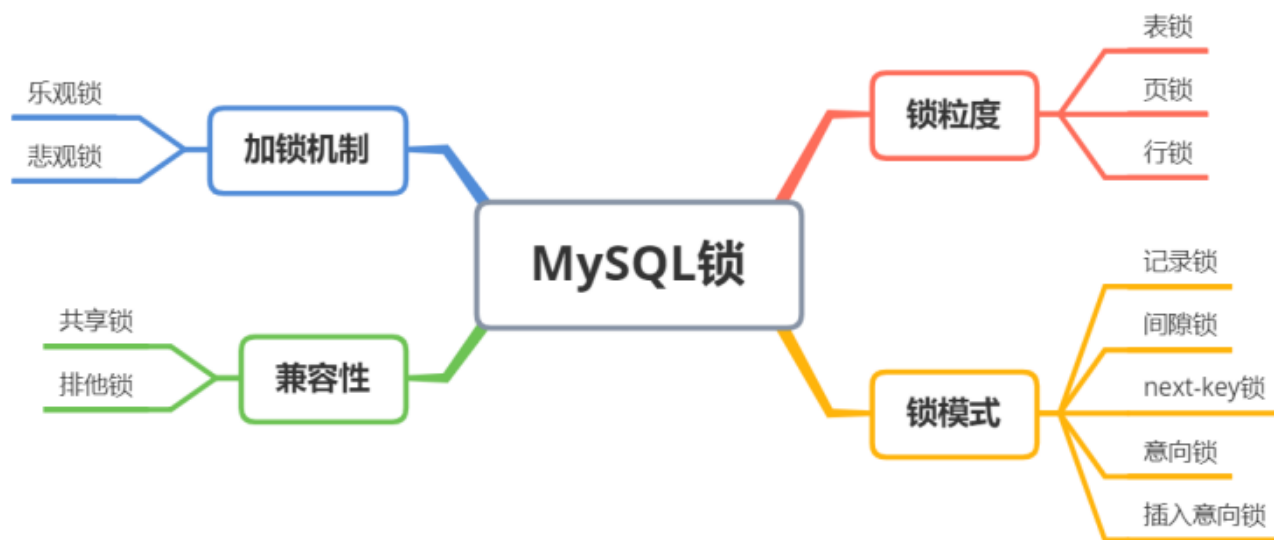
- 如果该记录的事务ID，**小于Read View中的活跃且未提交的最小事务ID**，表示这个版本的记录是在创建Read View之前已经提交的事务生成的，则该版本记录对当前事务**可见**
- 如果该记录的事务ID，**大于Read View中的活跃且未提交的最大事务ID**，表示这个版本的记录是在创建Read View之后才启动的事务生成的，所以该版本记录对当前事务**不可见**
- 如果记录的事务ID，在 `min_trx_id` 和 `max_trx_id` 之间，需要判断该记录id是否在**活跃且未提交事务列表**中

- 如果**存在**，那么表示生成该版本记录的活跃事务仍未提交，所以该版本记录对当前事务**不可见**
- 如果**不在**，那么表示生成该版本记录的活跃事务已经提交，所以该版本记录对当前事务**可见**

这种通过「版本链」来控制并发事务访问同一个记录时的行为就叫 **MVCC**（多版本并发控制）

锁

MySQL里的锁五花八门，按照不同维度划分，有各种不同的锁：



🌟InnoDB里的行锁

InnoDB是支持行级锁的，而MyISAM并不支持行级锁

InnoDB的**行锁是基于索引**的，而不是基于整个表。这意味着**只有在通过索引访问数据时，InnoDB才会使用行级锁**。如果没有合适的索引，InnoDB会使用表级锁

行锁的类型？

共享锁（S锁）——读读共享、读写互斥（`select ... lock in share mode` 对读取的记录加共享锁）

独占锁（X锁）——写写互斥、读写互斥（`select ... for update` 对读取的记录加独占锁）

记录锁

Record Lock，也就是仅仅锁住一条记录，有S锁和X锁之分

- 当一个事务对一条记录加了**S型记录锁**，其他事务可以继续加**S**，但不能加**X**
- 当一个事务对一条记录加了**X型记录锁**，其他事务不能加**S**，也不能加**X**
- 当事务commit之后，所有加的锁都会释放

间隙锁

Gap Lock，只存在于可重复读隔离级别，目的是为了解决可重复读级别下幻读的现象

假设，表中有一个范围 id 为（3，5）间隙锁，那么其他事务就无法插入 id = 4 这条记录了，这样就有效的防止幻读现象的发生

临键锁

Next-Key Lock，是**记录锁+间隙锁**的组合，锁定一个范围，并且锁定记录本身

假设，表中有一个范围 id 为（3，5] 的 next-key lock，那么其他事务即不能插入 id = 4 记录，也不能修改和删除 id = 5 这条记录

next-key lock 是包含间隙锁+记录锁的，如果一个事务获取了 X 型的 next-key lock，那么另外一个事务在获取相同范围的 X 型的 next-key lock 时，是会被阻塞的

MySQL是怎么加行锁的？

总结一句，在能使用记录锁或者间隙锁就能避免幻读现象的场景下，**next-key lock 就会退化成记录锁或间隙锁**

加锁的对象是索引，加锁的基本单位是 **next-key lock**，它是由记录锁和间隙锁组合而成的，**next-key lock** 是前开后闭区间，而间隙锁是前开后开区间

MySQL死锁是怎么回事？如何避免？

死锁的四个经典条件：互斥、请求和保持、不可抢占、循环等待

死锁情况有几种：

- 记录锁：事务A锁了记录a还想锁b，事务B锁了记录b还需要a，此时死锁
- 间隙锁：两个或多个事务对同一区间加间隙锁，其中一个事务需要执行insert，此时陷入死锁

如何解决？

- 直接进入等待，直到超时释放锁
- 回滚死锁链条中的某一个事务（持有最少行锁的事务进行回滚）

日志

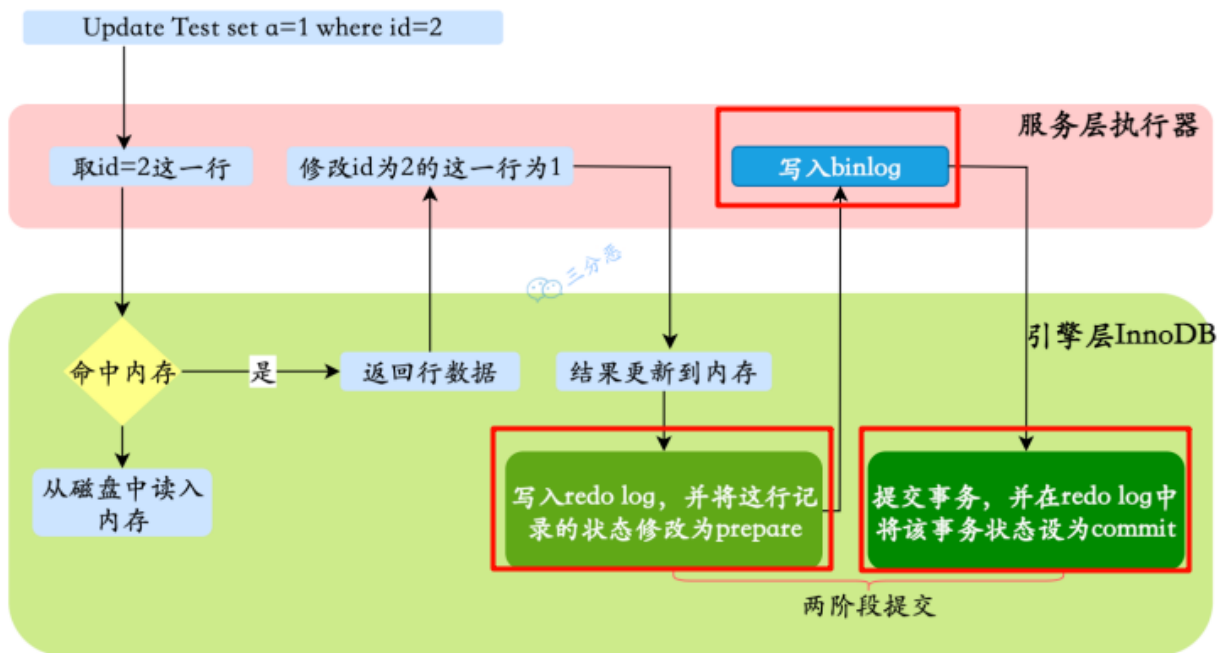
介绍一下都有哪些日志？

- **bin log**：记录数据库的所有DDL语句（数据定义语言）和 DML语句（数据操作语言），但不包括SELECT语句，以二进制格式记录，可以用于数据恢复、主从复制以及数据变更的回放

对于InnoDB存储引擎，还有以下两种特有的日志文件：

- **redo log**（重做日志）：用于保证数据库的事务持久性和崩溃恢复能力。当事务提交时，相关的修改操作会首先被记录到重做日志中，然后才会写入磁盘
- **undo log**（回滚日志）：用于支持事务的回滚操作。在事务执行期间，对数据的修改操作会同时被记录到回滚日志中，以便在事务回滚时恢复数据到修改之前的状态

更新语句的执行过程？



为什么要二阶段提交?

简单说，redo log和bin log都可以用于表示事务的提交状态，而两阶段提交就是让这两个状态保持逻辑上的一致

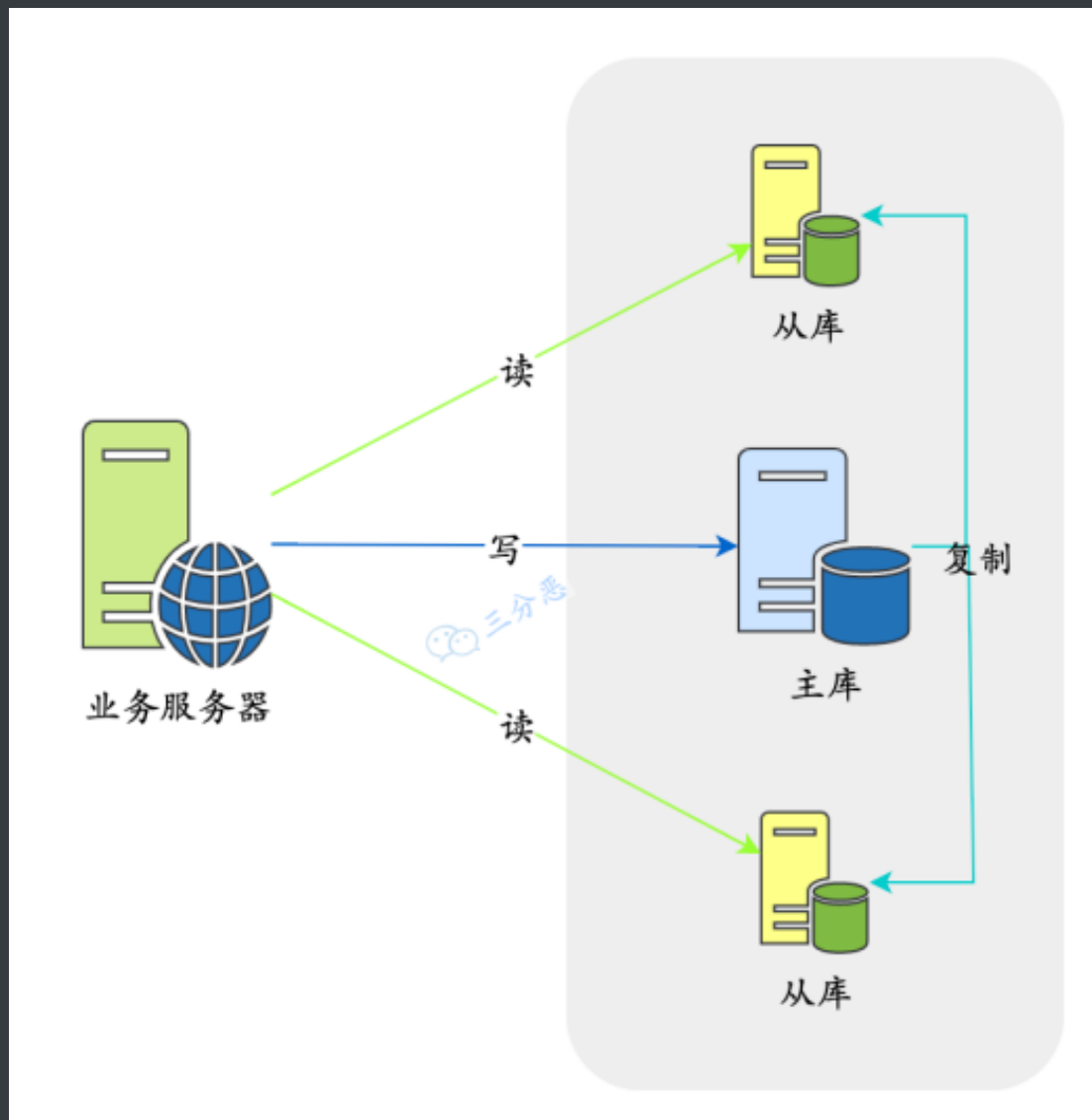
如果采用“单阶段”进行提交，即要么先写入redo log，后写入binlog；要么先写入binlog，后写入redo log。这两种方式的提交都会导致原先数据库的状态和被恢复后的数据库的状态不一致。

高可用

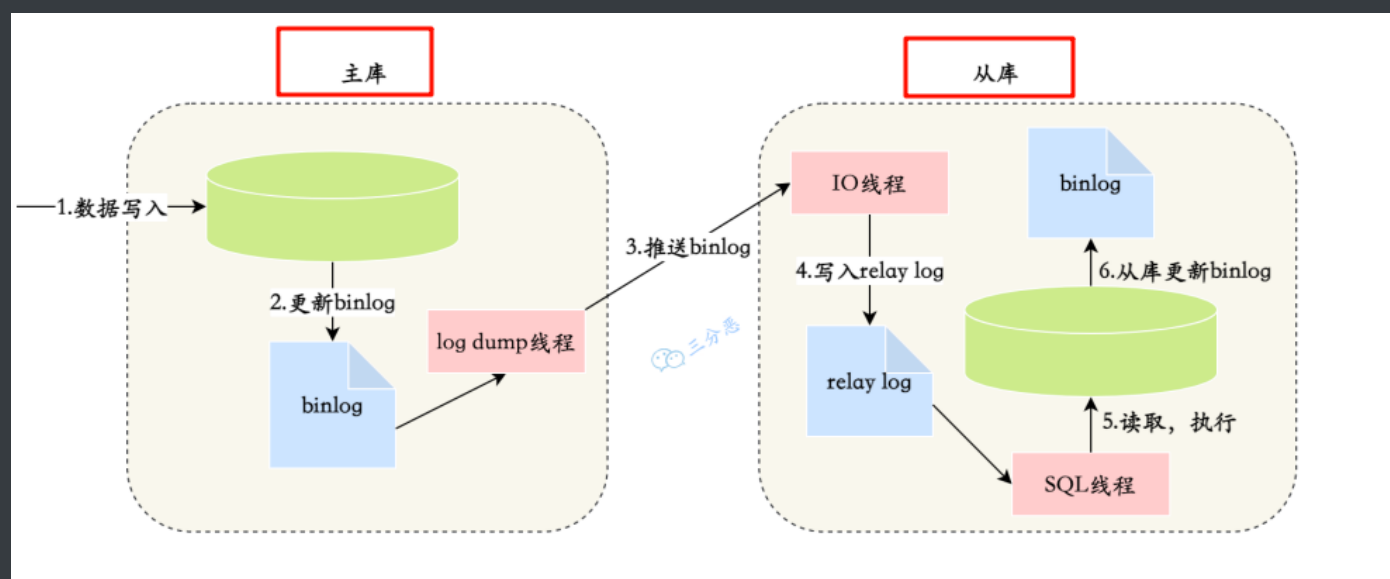
什么是读写分离 / 主从复制?

数据库主机负责读写操作，从机只负责读操作

主机通过复制将数据同步到从机，每个数据库都存储了所有的业务数据



主从复制原理是什么？



如何进行在线转移 / 在线扩容?

1. 在线双写，查询走老库
 - 数据写入旧库的同时，也写入新库
 - 并且同时将旧库中的历史数据迁移到新库
 - 新旧库数据对比 把差异补齐
2. 在线双写，查询走新库
 - 完成了历史数据的同步和校验
 - 把对数据的读切换到新库
3. 旧库下线
 - 旧库不再写入新的数据
 - 经过一段时间后，确定旧库没有请求，下线旧库

Redis

基础

Redis都有什么数据结构?

- string 字符串
- hash 哈希
- list 列表
- set 集合
- zset 有序集合

Redis可以用来干什么?

1.  **缓存**: 最广泛的应用，降低数据库压力 提高响应速度
2. **计数器**: Redis天然支持计数功能，并且性能好 项目里的**固定窗口计数器**限流便是基于Redis

3. 排行榜：Redis提供了列表和有序集合数据结构，合理地使用这些数据结构可以很方便地构建各种排行榜系统
4. 分布式锁
5. 存储验证码：项目里需要验证的手机 / 邮箱 验证码 存储在Redis里 验证后删除

Redis为什么这么快？

- 完全基于内存
- 🌟单线程操作：单线程方式处理客户端请求，避免多线程的上下文切换和竞争的开销
- 非阻塞的I/O多路复用机制：一个线程监听多个客户端连接
- 优化的数据结构：哈希表、跳表、压缩列表等

Redis为什么选择单线程？

核心原因：CPU并不是制约Redis的性能表现的瓶颈所在，更多情况下是受到内存大小和网络I/O的限制

并且使用多线程，也增加了系统复杂度、线程切换的开销等

Redis6.0之后为什么又引入了多线程？

在 Redis 6.0 版本之后，也采用了多个 I/O 线程来处理网络请求，这是因为随着网络硬件的性能提升，Redis 的性能瓶颈有时会出现在网络 I/O 的处理上

但是对于命令的执行，Redis 仍然使用单线程来处理

数据结构

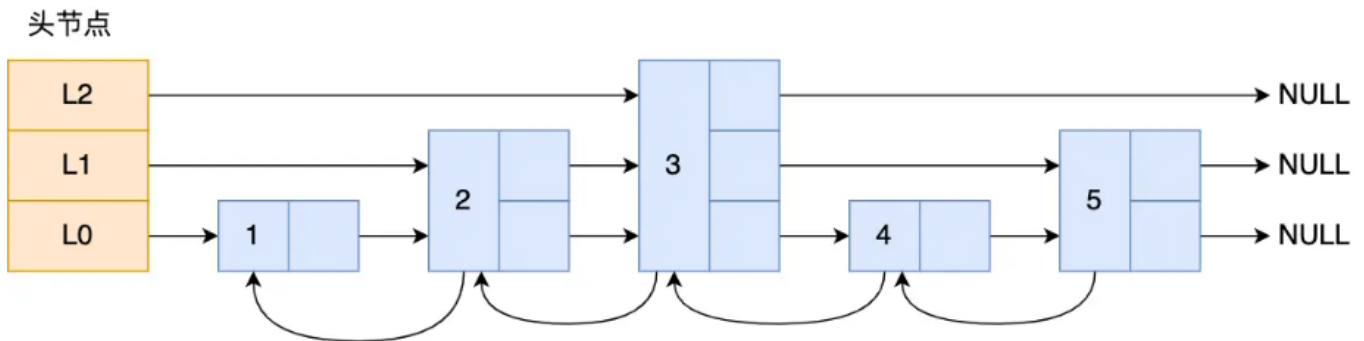
🌟🌟🌟跳跃表是如何实现的？原理？

Redis 只有 Zset 对象的底层实现用到了跳表，跳表的优势是能支持平均 $O(\log N)$ 复杂度的节点查找

zset 结构体里有两个数据结构：一个是跳表，一个是哈希表。这样的好处是既能进行高效的范围查询，也能进行高效单点查询

跳表结构设计

那跳表长什么样呢？我这里举个例子，下图展示了一个层级为 3 的跳表。



问题来了，由谁定义哪个跳表节点是头节点呢？这就介绍「跳表」结构体了，如下所示：

```
typedef struct zskiplist {
    struct zskiplistNode *header, *tail;
    unsigned long length;
    int level;
} zskiplist;
```

跳表结构里包含了：

- 跳表的头尾节点，便于在 $O(1)$ 时间复杂度内访问跳表的头节点和尾节点；
- 跳表的长度，便于在 $O(1)$ 时间复杂度获取跳表节点的数量；
- 跳表的最大层数，便于在 $O(1)$ 时间复杂度获取跳表中层高最大的那个节点的层数量；

跳表节点查询过程

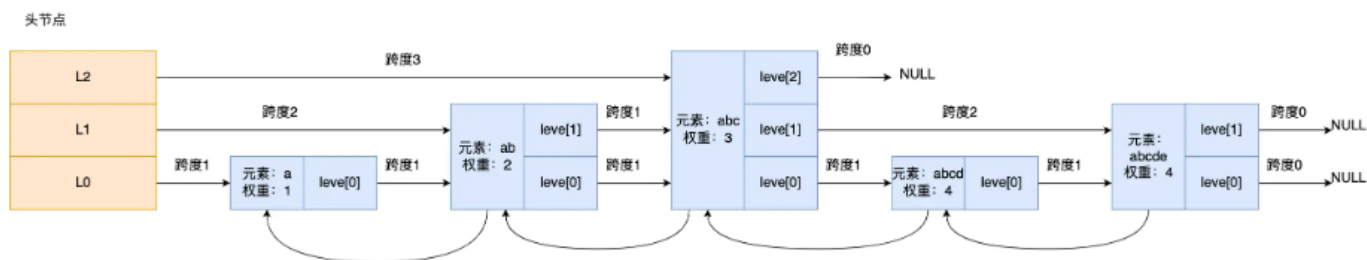
查找一个跳表节点的过程时，跳表会从头节点的最高层开始，逐一遍历每一层。在遍历某一层的跳表节点时，会用跳表节点中的 SDS 类型的元素和元素的权重来进行判断，共有两个判断条件：

- 如果当前节点的权重「小于」要查找的权重时，跳表就会访问该层上的下一个节点

- 如果当前节点的权重「等于」要查找的权重时，并且当前节点的 SDS 类型数据「小于」要查找的数据时，跳表就会访问该层上的下一个节点

如果上面两个条件都不满足，或者下一个节点为空时，跳表就会使用目前遍历到的节点的 level 数组里的下一层指针，然后沿着下一层指针继续查找，这就相当于跳到了下一层接着查找

举个例子，下图有个 3 层级的跳表。



如果要查找「元素：abcd，权重：4」的节点，查找的过程是这样的：

- 先从头节点的最高层开始，L2 指向了「元素：abc，权重：3」节点，这个节点的权重比要查找节点的小，所以要访问该层上的下一个节点；
- 但是该层的下一个节点是空节点（level[2]指向的是空节点），于是就会跳到「元素：abc，权重：3」节点的下一层去找，也就是 level[1]；
- 「元素：abc，权重：3」节点的 level[1] 的下一个指针指向了「元素：abcde，权重：4」的节点，然后将其和要查找的节点比较。虽然「元素：abcde，权重：4」的节点的权重和要查找的权重相同，但是当前节点的 SDS 类型数据「大于」要查找的数据，所以会继续跳到「元素：abc，权重：3」节点的下一层去找，也就是 level[0]；
- 「元素：abc，权重：3」节点的 level[0] 的下一个指针指向了「元素：abcd，权重：4」的节点，该节点正是要查找的节点，查询结束。

跳表节点层数设置

跳表的相邻两层的节点数量最理想的比例是 2:1，查找复杂度可以降低到 $O(\log N)$

为什么用跳表而不用平衡树？

- 从内存占用上来比较，跳表比平衡树更灵活一些
- 在做范围查找的时候，跳表比平衡树操作要简单
- 从算法实现难度上来比较，跳表比平衡树要简单得多

持久化

Redis持久化方式了解吗？

- **RDB快照**：将某一时刻的内存数据，以二进制文件的方式写入磁盘
- **AOF日志**：每执行一条写操作命令，就把该命令以追加的方式写入到一个日志文件里
- **混合持久化**：AOF + RDB

RDB和AOF各自原理？

RDB

RDB文件是一个压缩的**二进制文件**，保存在硬盘上，通过它可以还原某个时刻Redis数据库的状态

- RDB持久化是把当前进程数据生成 **快照** 保存到硬盘的过程，触发RDB持久化分为 手动和 自动
- 手动触发

手动触发分别对应save和bgsave命令：



- 自动触发

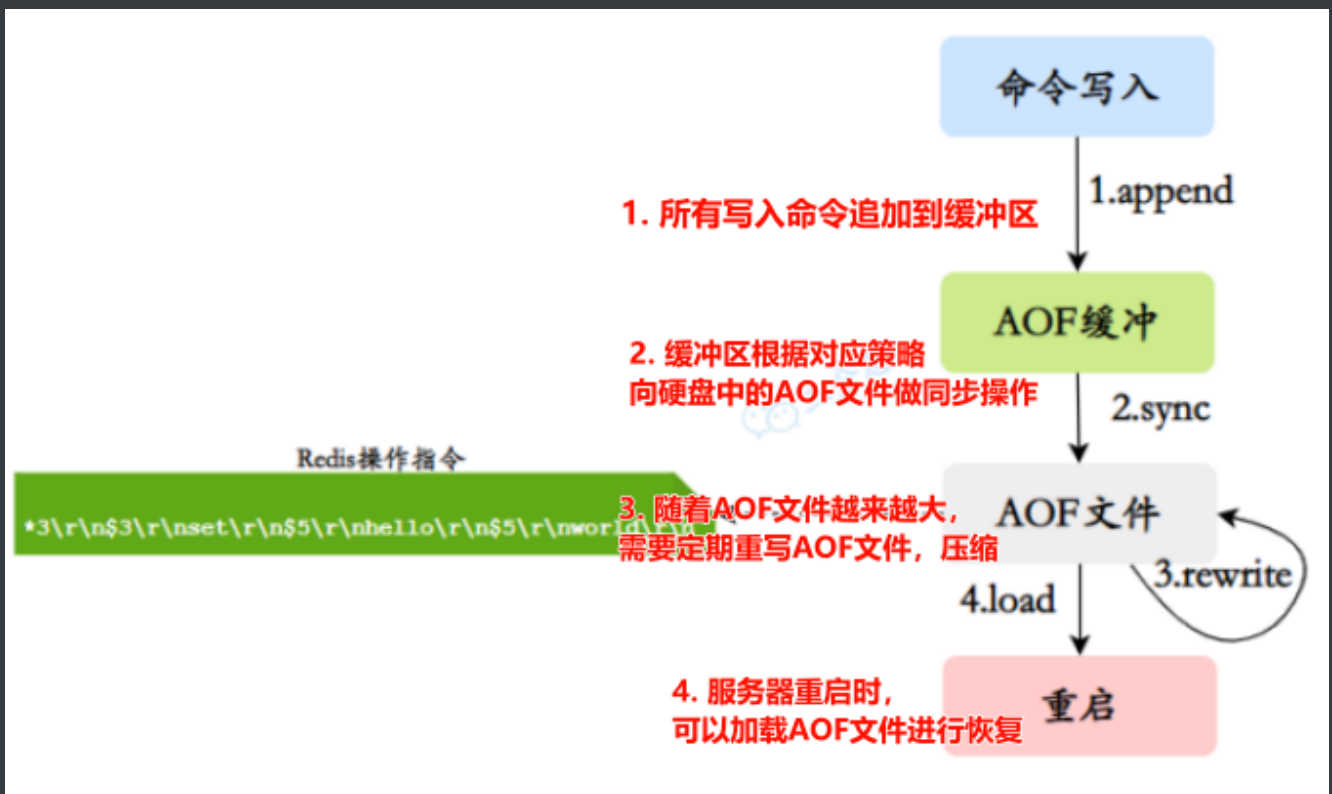
- **save配置**：在Redis的配置文件中可以配置save命令，指定触发RDB持久化的条件
如 `save 900 1` 表示在900s内如果有至少一个键被修改，则自动触发一次**bgsave**
- **从节点执行全量复制**：主节点自动执行**bgsave**命令生成RDB文件发送给从节点
- **关闭Redis服务器**：Redis服务器关闭时，如果没有开启AOF持久化则自动执行**bgsave**

AOF

以独立日志的方式记录**每次写命令**，重启时再重新执行AOF文件中的命令达到恢复数据的目的

解决了Redis数据持久化的**实时性**

- AOF的工作流程操作



RDB和AOF如何选择？（各自优缺点）

RDB

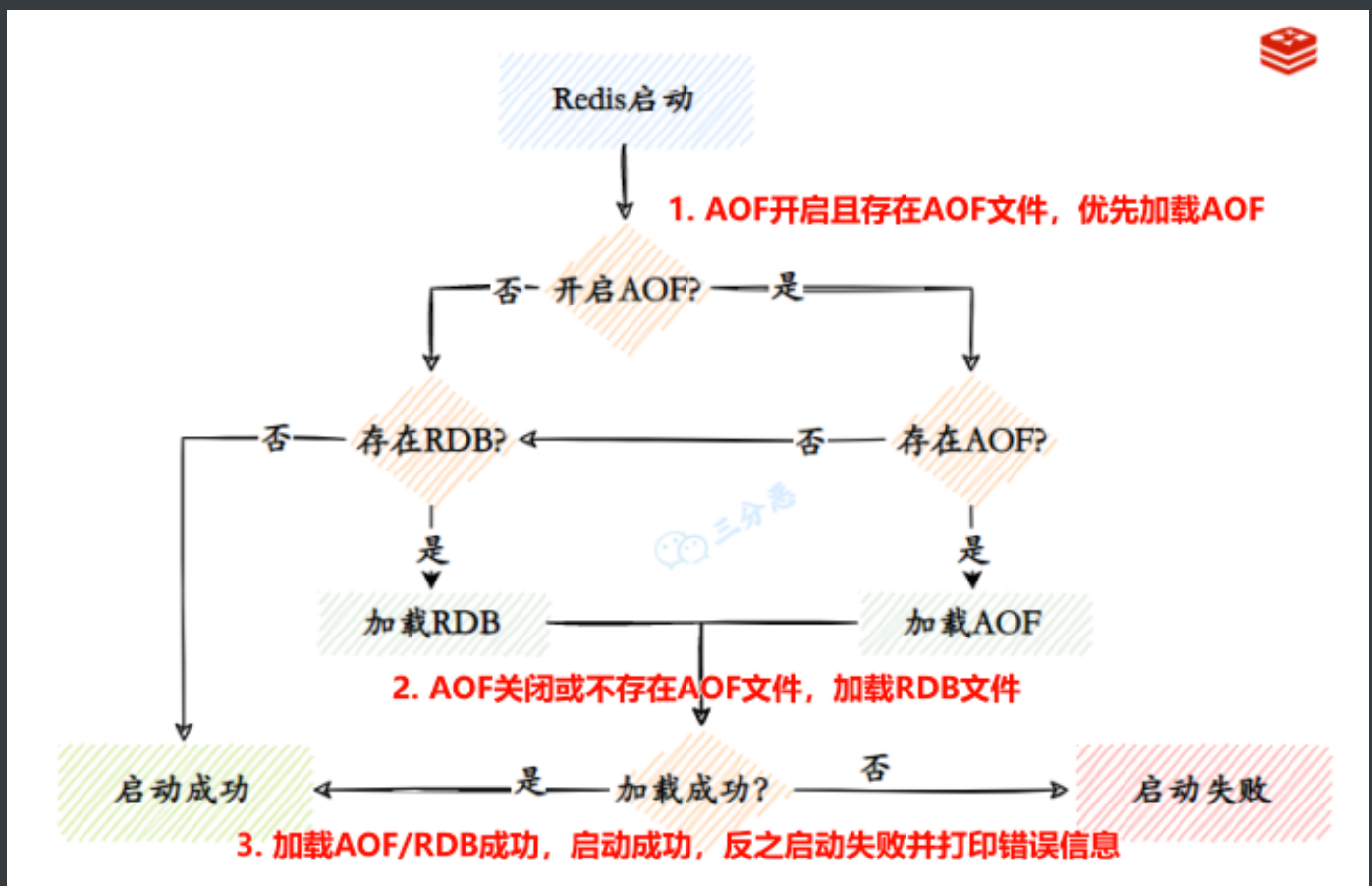
- 优点

- 只有一个紧凑的二进制文件 dump.rdb ， 非常适合备份、全量复制的场景
- 容灾性好，可以把RDB文件拷贝到远端机器或者文件系统中，用于容灾恢复
- 恢复速度快，RDB恢复数据的速度远远快于AOF
- 缺点
 - 实时性低，RDB是间隔一段时间进行，没法做到实时持久化，如果在间隔内发生故障，数据会丢失

AOF

- 优点
 - 实时性好，AOF可以配置来设置，每进行一次命令就记录到AOF文件中一次
 - 通过append模式写文件，即使写文件途中服务器宕机，也可以通过截断最后不完整的命令等手段来解决问题
- 缺点
 - AOF文件比RDB文件大，且恢复速度慢

Redis的数据恢复?

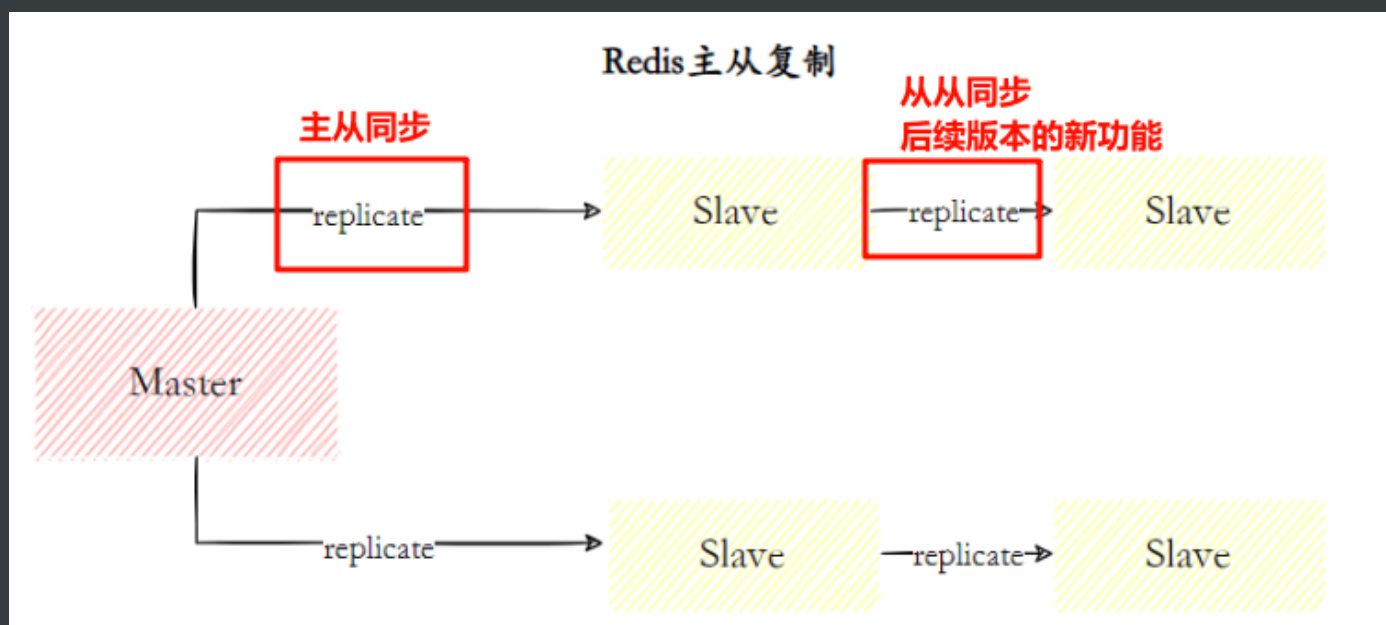


高可用

保证高可用主要有三种方式：主从、哨兵、集群

主从复制

主节点写、从节点读、主从之间数据异步同步



主从复制主要的作用？

- 数据冗余
- 负载均衡
- 高可用的基石

存在问题？

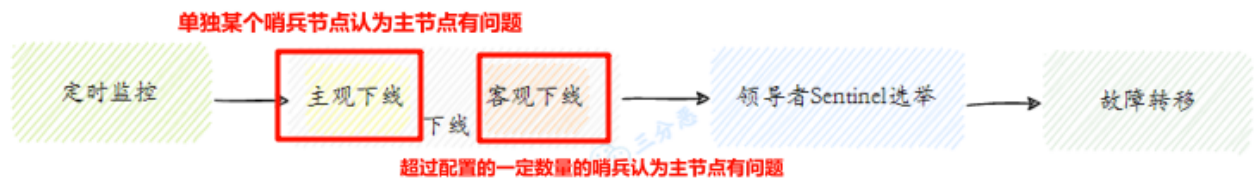
- 主节点故障后 没办法完成自动故障转移（哨兵解决）
- 主节点的写能力和存储能力也都受到单机的限制（集群解决）

哨兵

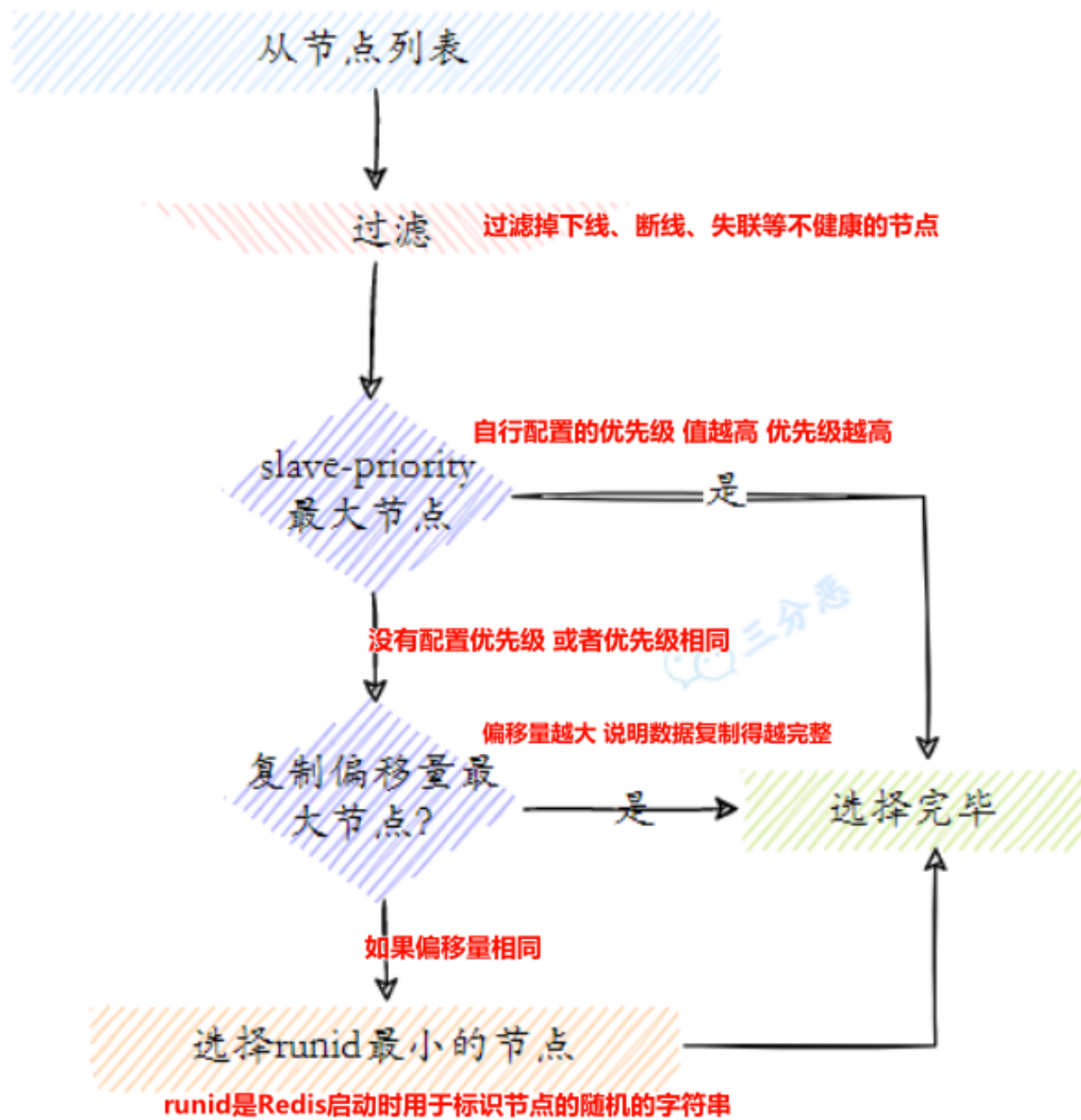
解决上述主从复制的自动故障转移问题

- 监控：哨兵会不断地检查主节点和从节点是否运作正常
- 自动故障转移
- 配置提供者：客户端在初始化时，通过连接哨兵来获得当前 Redis 的主节点地址
- 通知：哨兵可以将故障转移的结果发送给客户端

哨兵模式是通过哨兵节点完成对数据节点的监控、下线、故障转移。

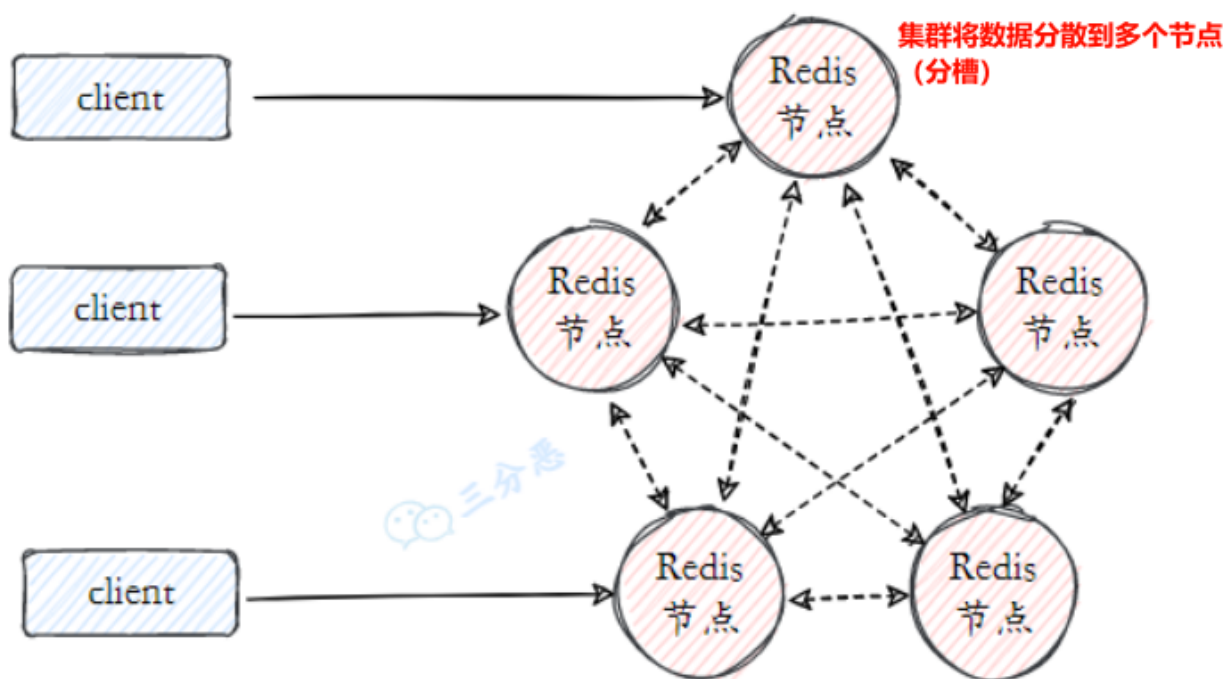


新的主节点是如何被跳出来的？



集群

集群是终极方案，一举解决高可用和分布式问题



集群的原理？

Redis集群通过**数据分区**来实现数据的分布式存储，通过**自动故障转移**实现高可用

集群创建

- 数据分区是在集群创建的时候完成的



故障转移

Redis集群的故障转移和哨兵的故障转移类似，但是Redis集群中**所有的节点都要承担状态维护的任务**

- **故障发现**：集群中每个节点都定期向其他节点发送ping消息，通信失败则标记主观下线，最后当半数以上持有槽的主节点都标记主观下线，则触发客观下线
- **故障恢复**：故障节点变为客观下线后，如果下线节点是持有槽的主节点则需要在它的从节点中选出一个替换它，从而保证集群的高可用（和哨兵模式一样）

缓存设计

缓存三大问题？如何解决？

- **缓存击穿**：热点数据过期（也就是**热点key**问题处理）
 - 互斥锁更新数据
 - 热点数据永不过期
- **缓存穿透**：请求数据既不在缓存，也不在数据库
 - 缓存空值或者默认值（占用内存大）
 - 使用**布隆过滤器**快速判断数据是否存在（实现复杂）
- **缓存雪崩**：大量缓存数据同时过期，或者redis故障
 - 均匀设置过期时间
 - 互斥锁更新数据
- **Redis故障宕机**
 - 服务熔断
 - 请求限流
 - 构建Redis集群

数据库和缓存如何保证一致性

根据 CAP 理论 在保证可用性A和分区容错性P的前提下，无法保证一致性C，所以缓存和数据库的绝对一致是不可能实现的，只能尽可能保存缓存和数据库的**最终一致性**

- 选择合适的缓存更新策略（删除缓存而不是更新缓存、先更数据库再删除缓存）
- 缓存不一致后处理（原因主要有：缓存key删除失败 并发导致缓存写入了脏数据）
 - 消息队列保证缓存key删除（对业务代码有一定侵入性）

- 延时双删防止脏数据（延时的时间设置需要仔细考量）
- 设置缓存过期时间兜底（朴素但有用，缓存过期自然会恢复一致）

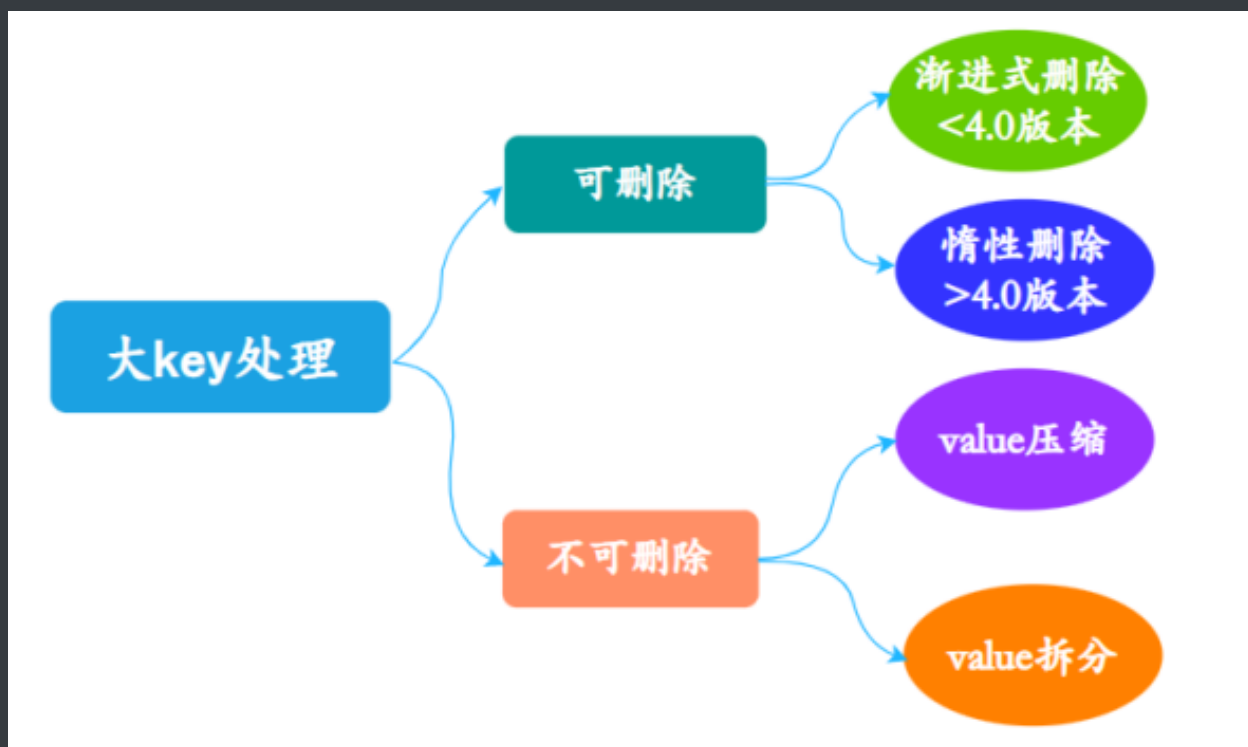
运维

Redis报内存不足怎么处理？

- 纵向扩展：增加单机Redis可用内存
- 修改内存淘汰策略：如 lru/lfu/ttl/random 等
- 横向扩展：使用Redis集群模式进行横向扩容，数据分散存储在多个Redis节点上

大key问题了解吗？

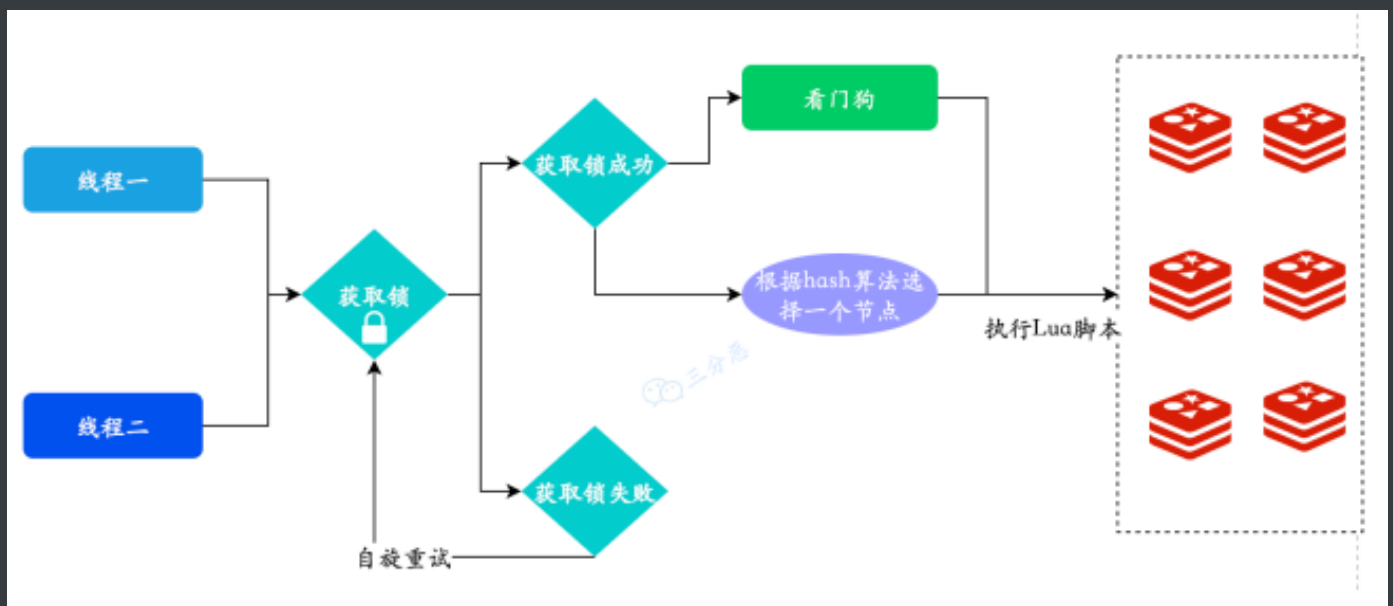
大key：单个简单的key存储的value很大、hash/set/zset/list中存储过多的元素（数以万计）



分布式

Redisson了解吗？

实现分布式锁，加锁过程中用到了 set 指令和 lua 脚本



Redisson加锁大致分为以下几步：

1. 使用Lua脚本执行加锁操作
2. Watch Dog续约锁的过期时间
3. 释放锁

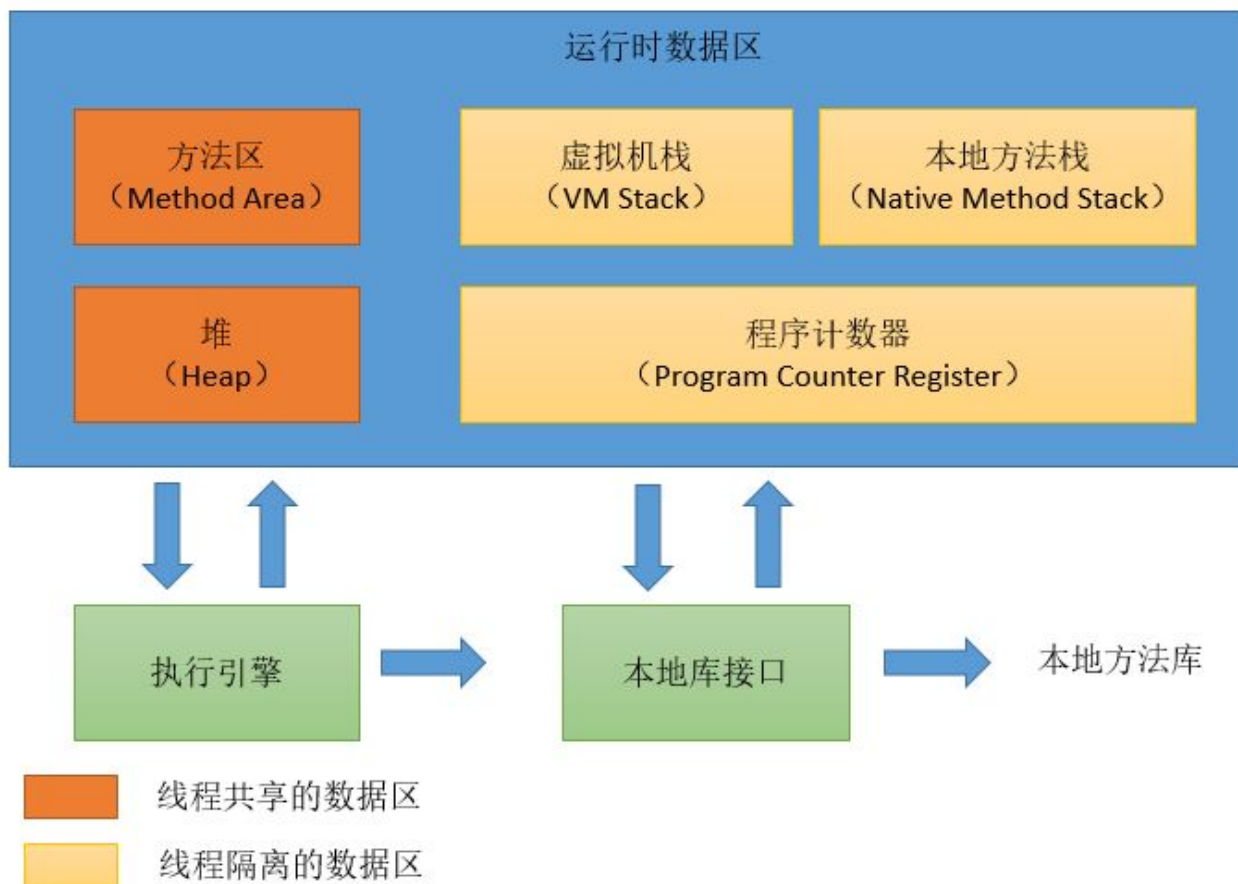
为什么要有 Watch Dog?

- 续约：Watch Dog定期发送续约命令给持有锁的客户端，确保锁在被持有过程中不会过期
- 释放：如果Watch Dog发现持有锁的客户端不再存活或者发生故障，会触发锁的自动释放

JVM

★内存区域

说说JVM的内存区域？



- **程序计数器**：保存当前程序的执行位置
- **虚拟机栈**：通过**栈帧**（存储局部变量、方法参数、操作数栈和动态链接等信息）来维持方法调用顺序，帮助控制程序有序运行
- **本地方法栈**：与虚拟机栈类似，但是它为**本地方法**服务，本地方法是用其他语言（C/C++）编写的，与Java进行交互，需要独立的栈来支持
- **堆**：线程共享，也是整个虚拟机**最大**的一块内存空间，而此区域的职责就是**存放和管理对象和数组**，垃圾回收机制也是主要作用于这一部分内存区域
- **方法区**：线程共享，包含**类信息**、**运行时常量池**、即时编译器的代码缓存

什么是内存溢出？内存泄漏？

- **内存泄漏**：申请的内存空间没有被正确释放，导致内存被白白占用
- **内存溢出(OOM)**：申请的内存超过了可用内存，内存不够了
- 两者关系：内存泄露 ---可能会---> 内存溢出

★如何快速排查OOM错误?

首先，**OOM的原因**：（对应的方法）

- 一次性资源申请的太多：更改申请对象数量
- 申请的资源未正确释放：找到未释放的对象进行释放
- 本身资源分配不够：jmap -heap 查看堆信息然后进行相应调整

对于前两种情况，则需要进行**OOM定位**：

- **导出dump文件**：系统已经挂了就设置当OOM时自动导出dump文件，还在运行中就 jmap -dump导出
- **借助java visualVM查看dump文件进行调试并定位**：查看跟业务有关的最多的对象 -> 找到GC Root -> 查看线程栈

★垃圾回收

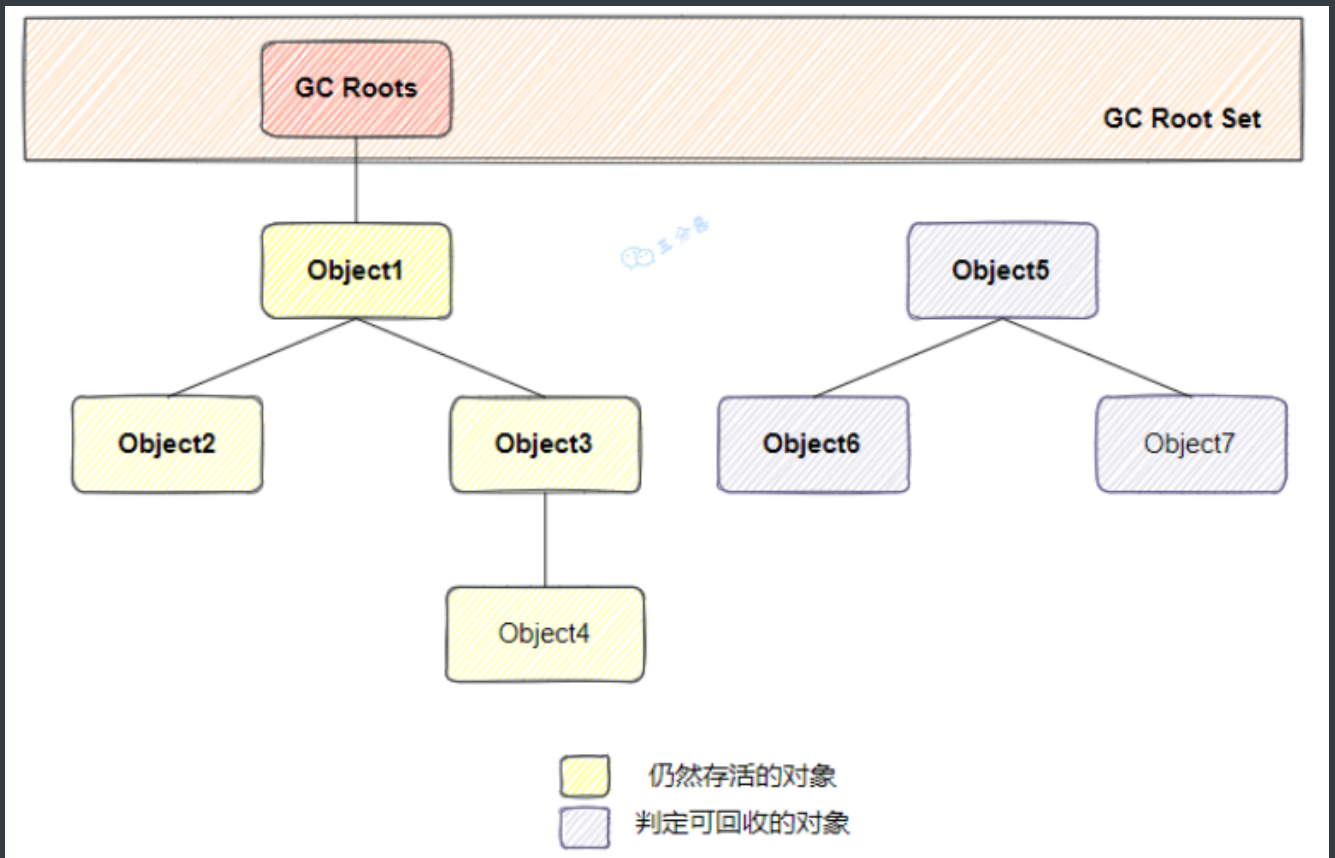
★介绍一下垃圾回收机制?

首先，先介绍**对象存活判定算法**：

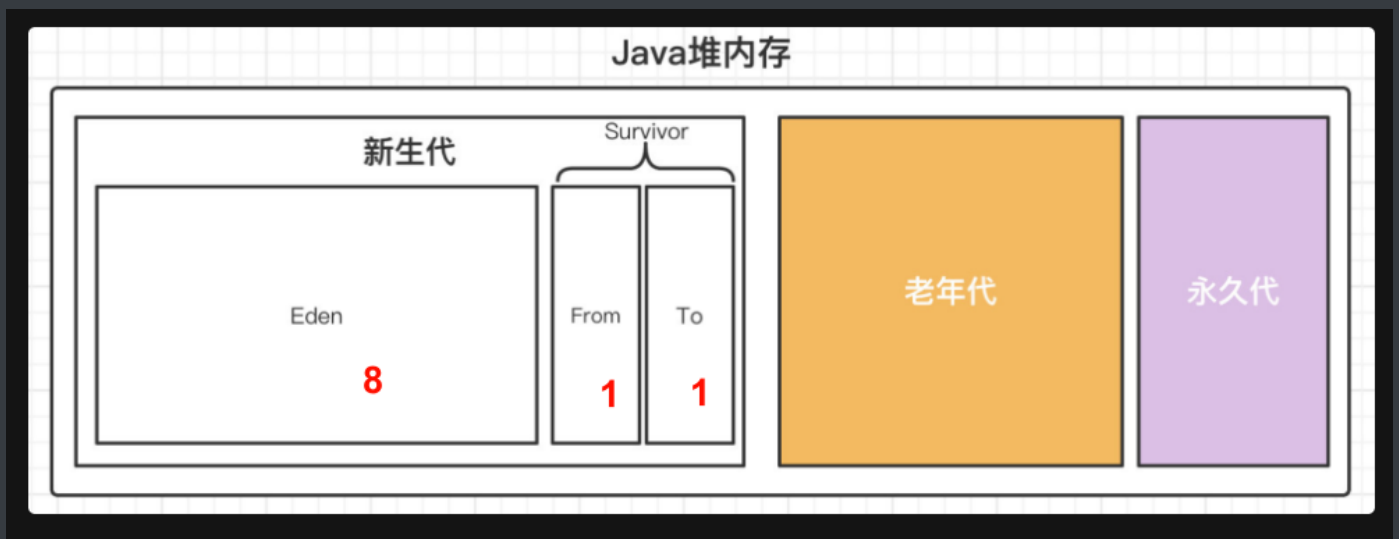
- 引用计数法
- 可达性分析算法：

GC Roots可以是

- 虚拟机栈（栈帧中的本地变量表）中引用的对象
- 方法区中类静态属性引用的对象，也就是static修饰的变量
- 方法区中常量引用的对象，final修饰变量，或者String常量池里的引用等
- 被同步锁synchronized持有的对象
- JVM内部的引用



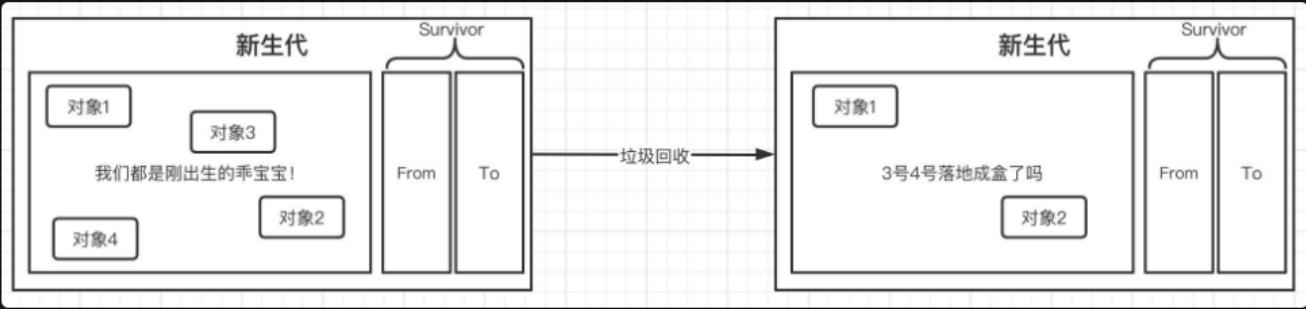
其次是，分代回收机制：Java虚拟机将堆内存主要划分为新生代、老年代和永久代(JDK8之后改为元空间实现，直接在堆外的本地内存)



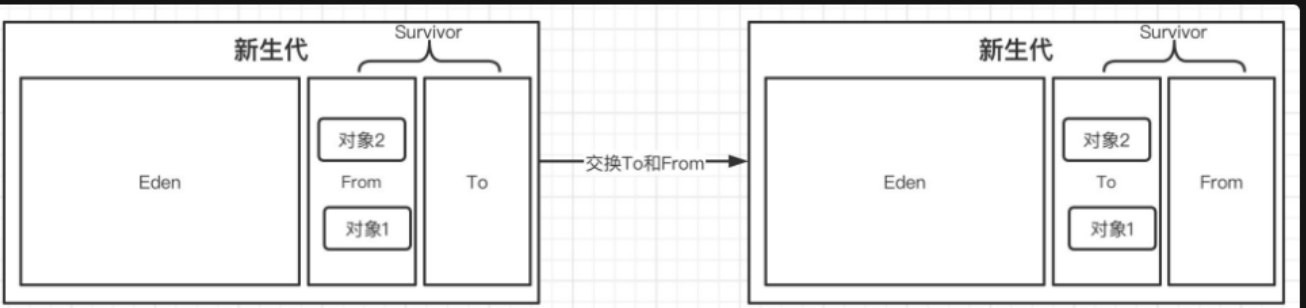
***具体是如何运作的呢？

■

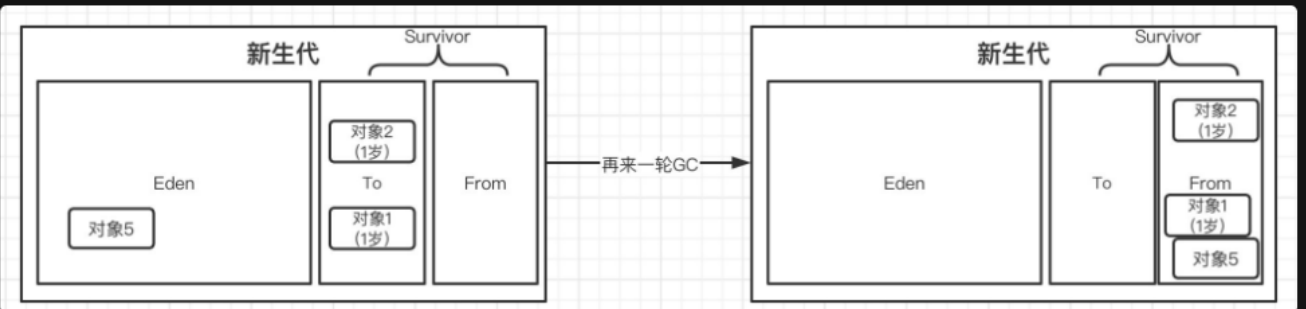
首先，所有新创建的对象，在一开始都会进入到新生代的Eden区（如果是大对象会被直接丢进老年代），在进行新生代区域的垃圾回收时，首先会对所有新生代区域的对象进行扫描，并回收那些不再使用对象：



接着，在一次垃圾回收之后，Eden区域没有被回收的对象，会进入到Survivor区。在一开始From和To都是空的，而GC之后，所有Eden区域存活的对象都会直接被放入到From区，最后From和To会发生一次交换，也就是说目前存放我们对象的From区，变为To区，而To区变为From区：



接着就是下一次垃圾回收了，操作与上面是一样的，不过这时由于我们From区域中已经存在对象了，所以，在Eden区的存活对象复制到From区之后，所有To区域中的对象会进行年龄判定（每经历一轮GC年龄 +1，如果对象的年龄大于默认值为15，那么会直接进入老年代，否则移动到From区）



最后像上面一样交换To区和From区，之后不断重复以上步骤。

垃圾回收分为哪些？

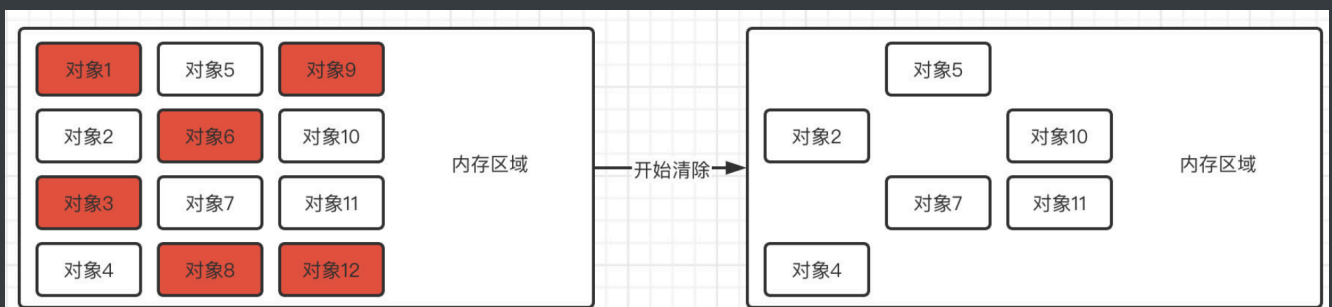
- **Minor GC** - 次要垃圾回收，主要进行新生代区域的垃圾收集。
 - 触发条件：新生代的Eden区容量已满时。
- **Major GC** - 主要垃圾回收，主要进行老年代的垃圾收集。
- **Full GC** - 完全垃圾回收，对整个Java堆内存和方法区进行垃圾回收。
 - 触发条件1：每次晋升到老年代的对象平均大小大于老年代剩余空间
 - 触发条件2：Minor GC后存活的对象超过了老年代剩余空间
 - 触发条件3：永久代内存不足（JDK8之前）
 - 触发条件4：手动调用 `System.gc()` 方法

对象什么时候回进入老年代？

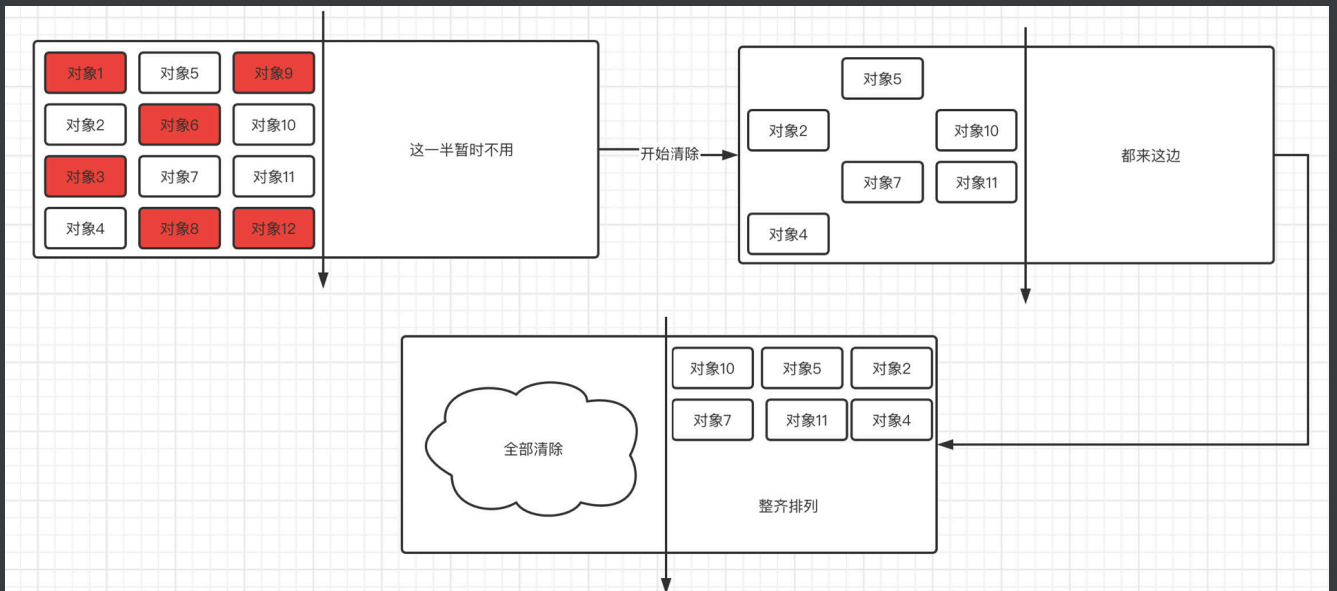
- 长期存活（默认GC年龄大于15）
- 大对象（直接进入）
- 空间分配担保：Minor GC后新生代仍有大量对象存活，那么就需要老年代进行担保，将Survivor区无法容纳的对象直接送入老年代（本身不到年龄 但新生代无法存下了）

垃圾回收算法了解吗？

- 标记-清除：标记所有需要回收的对象，然后再依次回收



- 执行效率不稳定（标记和清除的执行效率随对象数量增长而降低）
- 标记-清除之后会产生大量不连续的内存碎片，使得内存空间利用率降低
- 标记-复制：将内存区域划分为大小相同的两块区域，每次只使用其中的一块区域，每次垃圾回收结束后，将所有存活的对象全部复制到另一块区域中，并一次性清空当前区域
 - 一部分空间没有使用，存在空间浪费

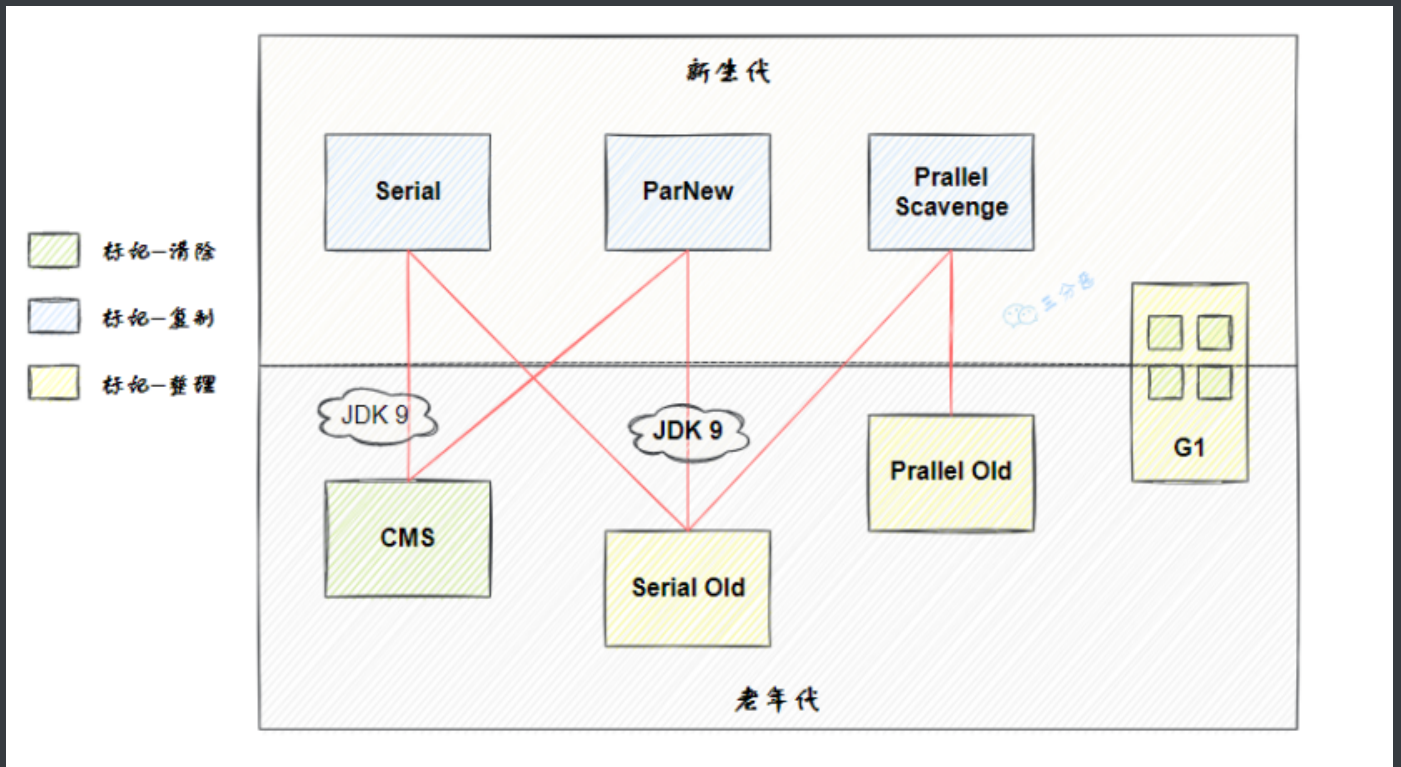


- **标记-整理：** 标记，但先不回收，而是按顺序整理好，再按顺序一次清理



🌟垃圾回收器都知道哪些？

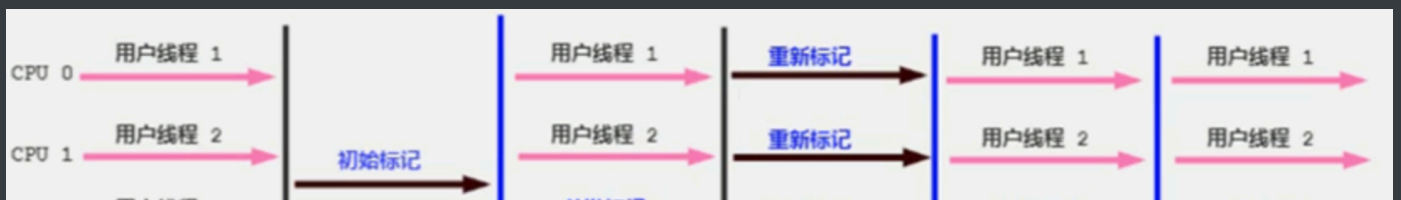
- 串行回收器：Serial Serial Old
- 并行回收器：ParNew Parallel Scavenge Parallel Old
- 🌟 并发回收器：**CMS G1**



CMS回收器

并发收集、停顿时间短，但是由于采用**标记-清除**，还是无法避免容易产生内存碎片

采用**标记-清除**算法：



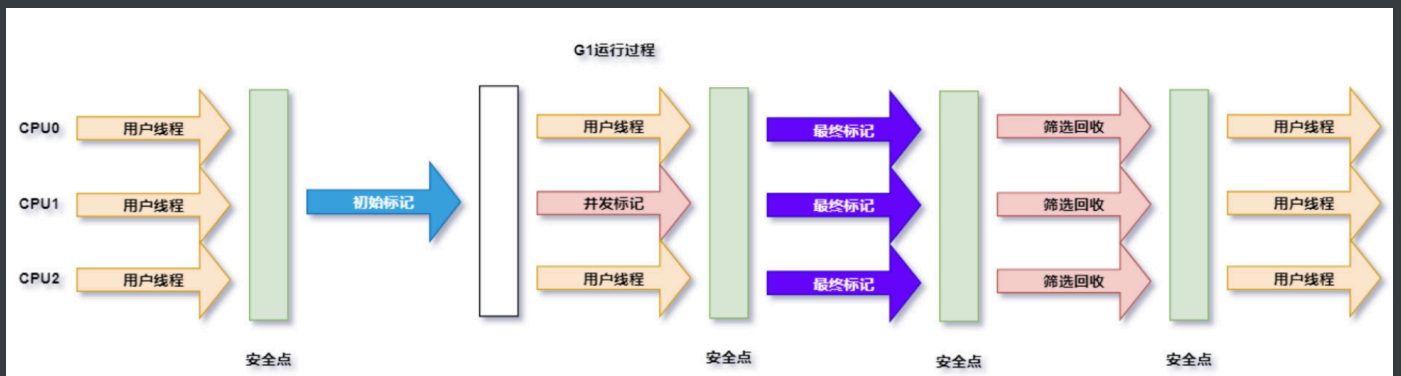
分为4个阶段：

- **初始标记**（需要暂停用户线程）：仅标记出GC Roots能直接关联的对象，速度比较快
- **并发标记**：从GC Roots的直接关联对象开始遍历整个对象图的过程
- **重新标记**（需要暂停用户线程）：并发标记阶段可能某些用户线程会导致标记产生变动，因此需要重新标记
- **并发清除**：最后就可以直接将所有标记好的无用对象进行删除

G1处理器

G1相比CMS优势：内存碎片问题缓解、停顿时间更短、处理大对象的能力、并发性能优化

开创了局部收集的设计思路和基于Region的内存布局形式



运行过程还是四个阶段（类似CMS）：

- **初始标记**（暂停用户线程）：仅仅只是标记一下GC Roots能直接关联到的对象
- **并发标记**：从GC Root开始对堆中对象进行可达性分析，递归扫描整个堆里的对象图，找出要回收的对象
- **最终标记**（暂停用户线程）：对用户线程做一个短暂的暂停，用于处理并发标记阶段漏标的那部分对象
- **筛选回收**（暂停用户线程）：负责更新Region的统计数据，对各个Region的回收价值和成本进行排序，根据用户所期望的停顿时间来制定回收计划

类与类加载

类的生命周期

类的生命周期的7个步骤

类加载器

- 启动类加载器 (**Bootstrap ClassLoader**): 也称为根类加载器, 是Java虚拟机的一部分, 负责加载Java的核心类库
- 扩展类加载器 (**Extensions ClassLoader**): 它用来加载 Java 的扩展库

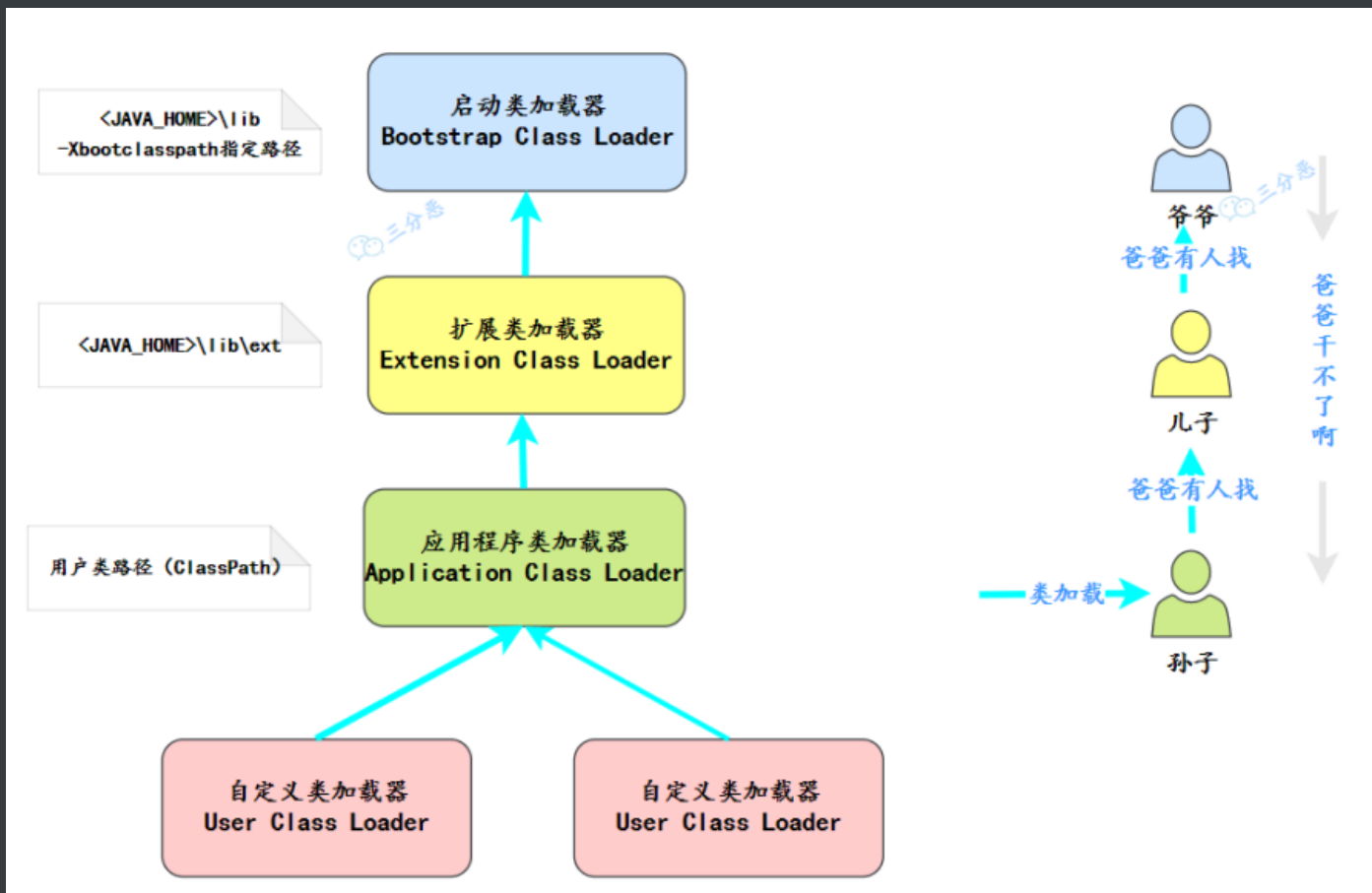
- 系统类加载器 (**AppClassLoader** 或 `SystemClassLoader`: 它根据 Java 应用的类路径 (CLASSPATH) 来加载Java 类



★双亲委派机制

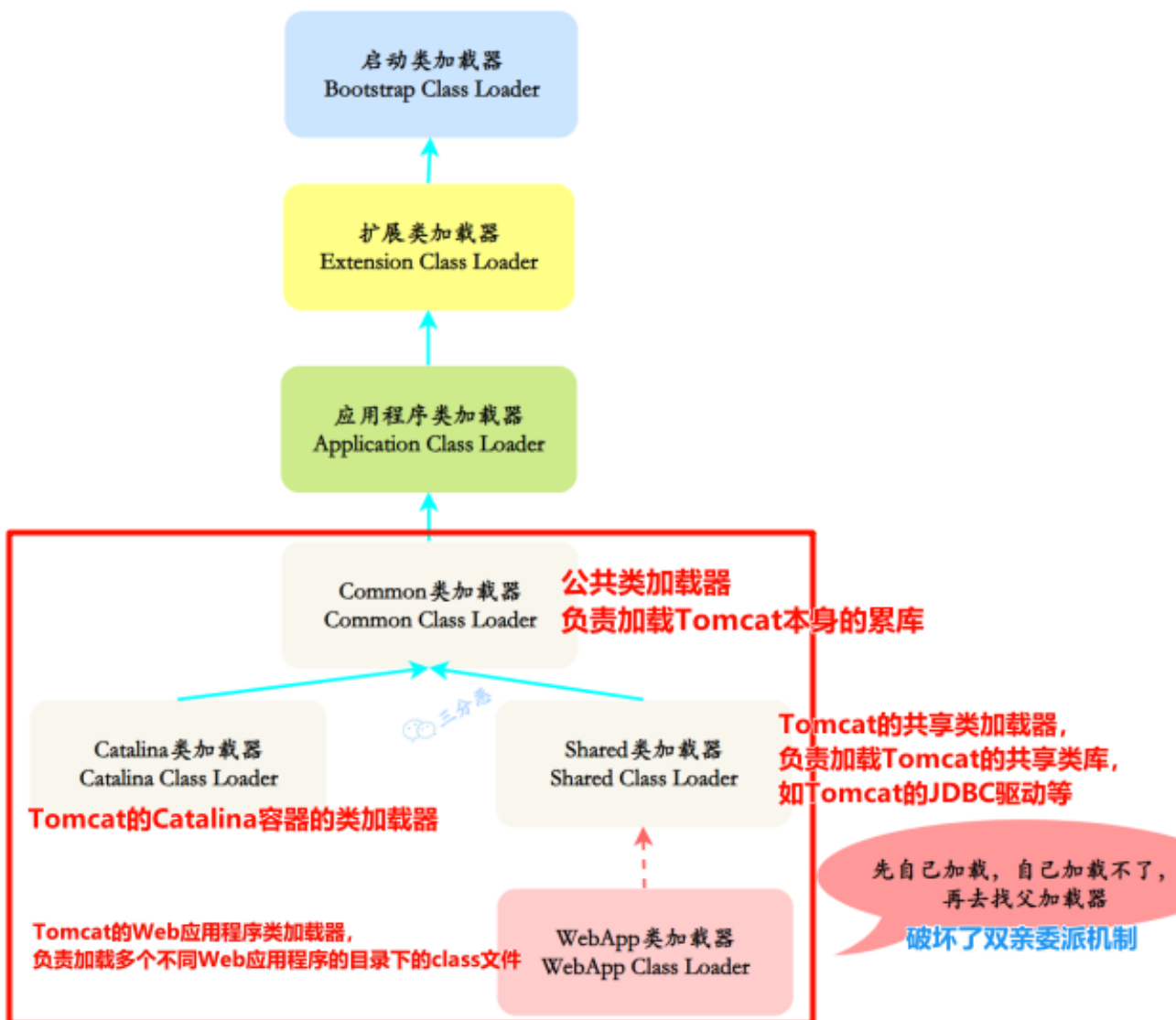
一个类加载器收到加载请求时，先给父类加载，加载不出来再自己加载

作用：避免类的重复加载、确保类的安全性和一致性、防止类的篡改、方便类加载器的扩展和自定义



★ 破坏双亲委派例子 (Tomcat)

Tomcat类加载器如下：



为什么Tomcat要破坏双亲委派机制?

- **隔离不同Web应用程序的类空间：** Tomcat容器中，可能会部署多个独立的Web应用程序，如果共享一个类加载器只能加载一个版本的类库，因此为每个web应用程序提供独立的**WebApp类加载器**
- **支持Web应用的动态更新：** 当开发者修改并重新部署某个Web应用的部分类文件时，如果不打破双亲委派，新的类定义无法被重新加载，除非重启整个Web容器，因此破坏之后可以进行热部署和热更新

Java并发

如何创建线程？

不管哪种方法，都依赖于 `new Thread().start()`

- 继承 Thread 类
- 实现 Runnable 接口
- 实现 Callable 接口
- 使用线程池
-

守护线程是什么？

- Java线程分为**守护线程**和**用户线程**（例如GC线程就是守护线程）
- 当**所有用户线程**执行完毕，程序终止（无论是否有守护线程还在执行）

ThreadLocal了解多少？用过吗？

线程本地变量，操作自己本地内存里面的变量，起到线程隔离作用

ThreadLocal内存泄漏是怎么回事？

- ThreadLocalMap<**ThreadLocal**, value>中的key 为 ThreadLocal 的弱引用
- 弱引用容易被GC回收，回收之后，ThreadLocalMap生命周期和Thread一样，所以它不被回收就会出现key没了，value还在，就会造成**内存泄漏**
- 解决办法：使用完ThreadLocal后，及时调用remove()方法释放内存空间

Java内存模型知道吗？

主内存+线程本地内存实现

原子性、可见性、有序性？

- 原子性：要么都做、要么都不做
- 可见性：一个线程对共享变量修改，其他线程能够立即看到这个修改
- 有序性：是指程序执行的顺序与代码的顺序一致

锁的分类

乐观和悲观

- 悲观锁：访问共享资源前悲观地认为总有人也要用，所以每次先拿锁（`synchronized` 和 `ReentrantLock`）
- 乐观锁：乐观地认为并发修改冲突少，因此直接访问共享资源，在使用资源时再进行冲突检测决定是否加锁（实现方法：版本号 / CAS）

可重入和不可重入

- 可重入锁允许同一个线程在持有锁的情况下再次获取该锁，而不会发生死锁
- `Synchronized`和`ReentrantLock`都是可重入锁

公平和非公平

- 公平锁指 按照 等待锁的进程按顺序发放锁（也就是先来先服务），非公平就不按顺序
- `ReentrantLock`可以指定
- `Synchronized`只能是非公平锁

重量级和轻量级

🌟`volatile`和`synchronized`知道吗？原理呢？

`volatile` 关键字主要用于解决变量在多个线程之间的可见性，而 `synchronized` 关键字解决的是多个线程之间访问资源的同步性

`volatile`

- 保证可见性：一个线程操作`volatile`变量时，不会把值缓存而是直接刷新回主内存，并且其他线程读取时会主动获取最新值而不是使用当前线程本地内存中的值

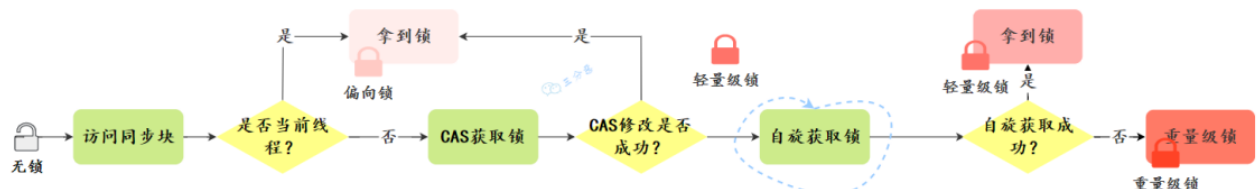
- **保证有序性**：volatile会**禁止指令重排**。若用volatile修饰共享变量，在编译时，会在指令序列中插入内存屏障来禁止特定类型的处理器重排序

synchronized

具体原理可以**类比操作系统中的信号量机制**，每次只能由一个线程进入同步代码块，释放锁之后会计数器+1并唤醒处于等待队列里的下一个线程（但是不保证顺序），获取到锁之后就计数器-1

- 属于**重量级锁**和**非公平锁**，因为它的监视器锁是依赖于底层操作系统的互斥锁实现的，但是JDK6以后引入了许多优化减少了锁的开销
- 每个对象都有一个内置锁，synchronized锁住的就是**基于对象的内置锁**，只有获取了才能进入同步代码块或者同步方法
- **保证原子性**：用synchronized关键字修饰代码块的时候，会使用 `monitorenter` 和 `monitorexit` 两个指令实现同步（相当于获取锁、释放锁）
- 锁升级：无锁 --> 偏向锁 ---> 轻量级锁 ----> 重量级锁

大体上省简的升级过程：



ReentrantLock（可重入锁）

依赖于AQS实现，使用方面和synchronized差不多，但是有一些方面不同

- 锁的实现：synchronized基于JVM实现，而ReentrantLock基于JDK的API层面实现
- **功能特点**：ReentrantLock相比synchronized有以下特点
 - ReentrantLock可以指定是公平锁还是非公平锁，而synchronized只能是非公平
 - ReentrantLock需要手动声明加锁和释放锁，而synchronized不需要手动释放

什么是CAS?

- 全称Compare And Swap 比较与交换
- 是一种无锁算法，它并不会为对象加锁，而是在执行的时候，看看当前数据的值是不是我们预期的那样，如果是，那就正常进行替换，如果不是，那么就替换失败。
- CAS有什么问题？经典三大问题
 - ABA问题（用版本号解决）
 - 循环性能开销
 - 只能保证一个变量的原子操作

AQS了解多少？

- AQS，是Java中用于实现同步器的抽象基类，它是Java并发包中许多同步工具的基础，如ReentrantLock、Semaphore、CountDownLatch等
- AQS提供了一种基于FIFO等待队列的同步器框架，允许开发者实现自定义的同步器。它的设计思想是将同步状态抽象成一个整型变量，并通过原子操作来控制该状态的获取和释放
- 使用了**模版方法模式**的设计模式

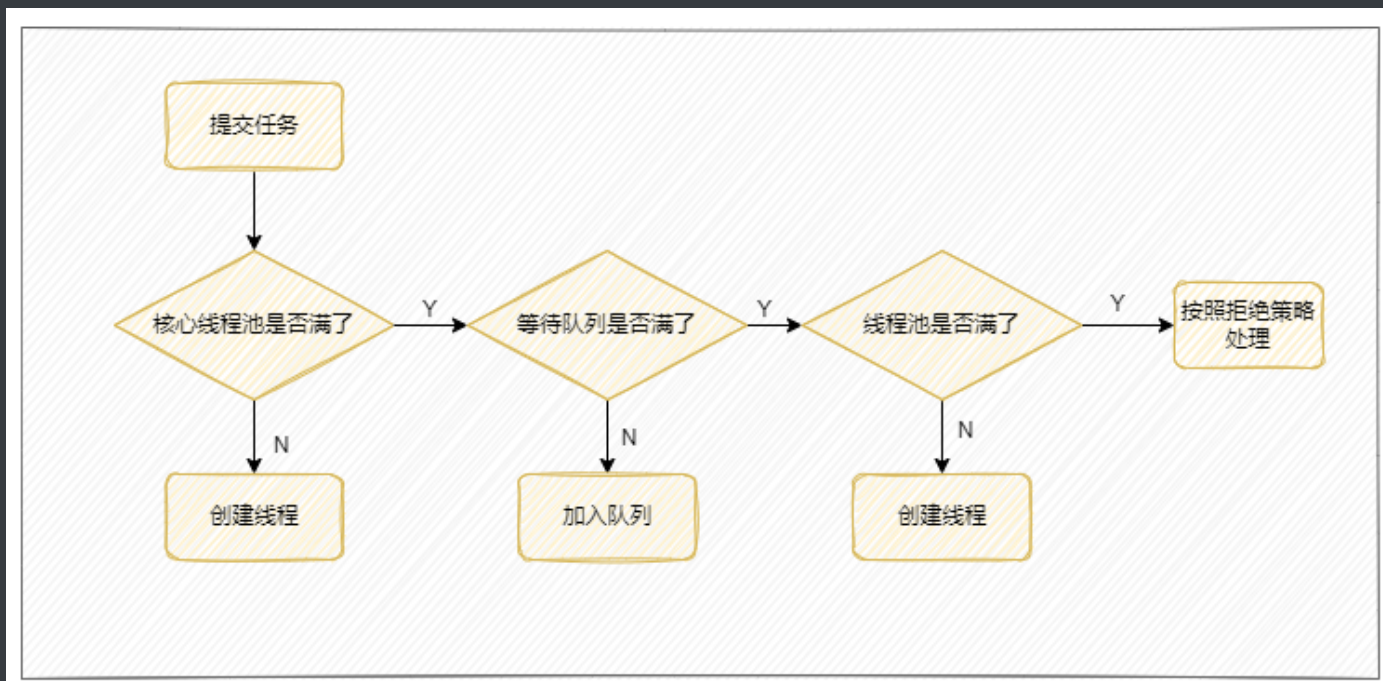
★线程池知道吗？有没有用过？

为什么要用线程池？

简单说，就是**管理线程的池子**

- 帮助管理线程，避免增加创建线程和销毁线程的资源损耗
- 提高响应速度，相对于创建新线程执行任务，从池子里拿线程更快
- 重复利用线程，节省资源

简单说一下线程池的工作流程？



通俗的比喻：营业厅办业务：总共六个窗口 现在只开放了三个窗口可以办业务

小明去办业务，可能会遇到以下情况：

1. 有空着的在营业的窗口，直接办理业务
2. 没有空闲窗口，但是等待区还有座，坐着等
3. 没有空闲窗口，等待区也满了，经理就赶紧让没开的窗口也开放了，工作一段时间后空闲了可以休息
4. 所有窗口都满了，等待区也满了，小明无法办理业务，经理怎么对待小明？（线程池拒绝策略）
 1. 营业厅瘫痪：AbortPolicy（默认策略，抛出异常）
 2. 谁叫你来办的你找谁去：CallerRunsPolicy（由提交任务的线程来执行该任务）
 3. 看你比较急，你去加个塞：DiscardOldestPolicy（丢弃队列中最早的任务，然后尝试重新提交该任务）
 4. 今天没办法，改一天再办：DiscardPolicy：（默默地丢弃该任务，不做任何处理）
 5. 除此之外，还可以自定义处理策略

以上流程跟JDK线程池业务大致类似：

- 营业厅三个开放的窗口对应 核心线程池数：**corePoolSize**

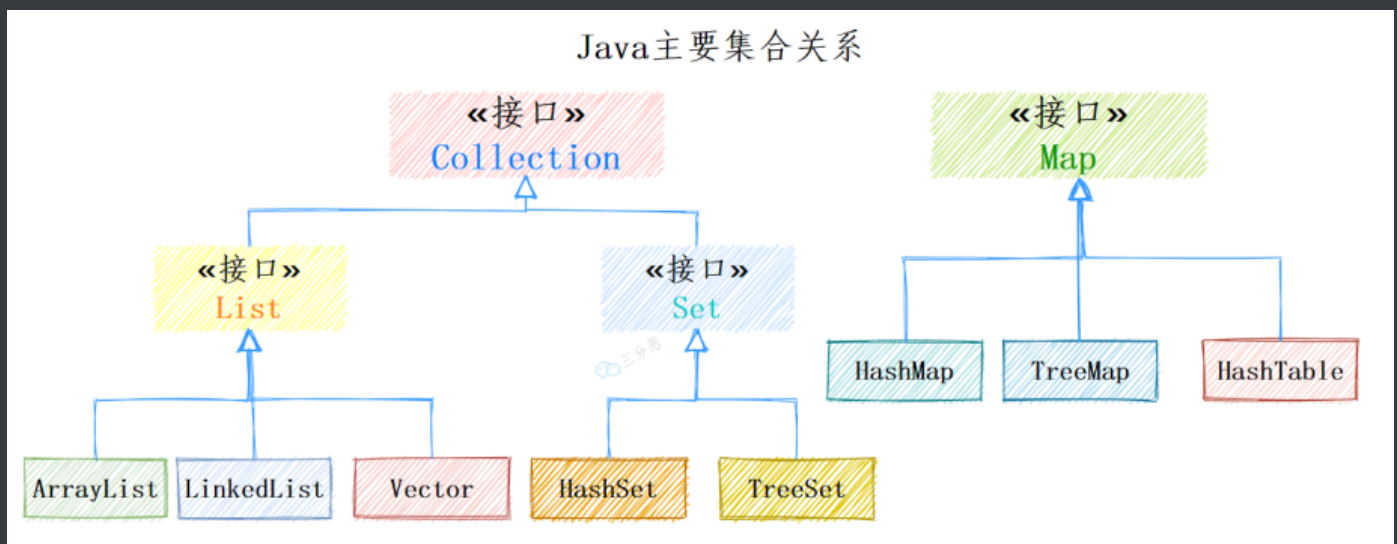
- 总的营业窗口对应：**maximumPoolSize**
- 打开的临时窗口在多长时间无人办理则自动关闭：**keepAliveTime**
- 等待区就是等待队列：**workQueue**
- 无法办理的时候经理给出的解决办法就是：**RejectedExecutionHandler**
- threadFactory在JDK中是线程工厂，用来创建线程对象，一般不动

***那你实际有没有用过呢？

有的，在**校园论坛**中，在实现定时实现将Redis中的点赞收藏数据写入MySQL功能时，利用线程池来完成这个**定时任务**，用的是**ScheduledThreadPool**

- 这样就不会每次执行定时任务都创建一个新线程，加叙述线程池的优点

Java集合



***ArrayList和LinkedList的区别？

- 数据结构不同（前者**动态数组**，后者**双向链表**）
- 前者利于查找、后者利于增删
- 是否支持随机访问
- 内存占用（是否连续）

★快速失败 和 安全失败?

两种“失败”源于在遍历集合的同时有对集合做修改，尤其是在面对多线程环境下的并发修改时所采取的不同策略

快速失败

位于 `java.util` 包下的 `ArrayList` / `LinkedList` / `HashMap` 等都采用快速失败

- **现象**：如果在迭代过程中有线程（包括本身）对集合进行了修改（添加/删除/更新），那么会立刻检测到这个修改并**迅速抛出异常**
- **原理**：通过**维护一个内部计数器**（如 `modCount`）来跟踪集合结构的变化。每次对集合进行结构修改时，这个计数器会递增，迭代器在每次调用 `next()` 或 `remove()` 方法前检查当前计数器值与遍历开始时记录的预期值是否一致。如果不一致，说明集合已被并发修改，于是立即抛出异常
- **如何解决？**
 - **删除元素**：使用迭代器的 `remove()` 方法 / 使用集合自带的 `removeIf()` 方法
 - **多线程**：加锁修改，**使用并发集合**（如 `CopyOnWriteArrayList` / `ConcurrentHashMap` 等）

安全失败

位于 `java.util.concurrent` 包下的 `CopyOnWriteArrayList` / `ConcurrentHashMap` 等都采用安全失败

- **现象**：在遍历时不是直接在集合内容上访问的，而是先复制原有集合内容，在拷贝的集合上进行遍历
- **原理**：由于迭代时是对原集合的拷贝进行遍历，所以在遍历过程中所作的修改并不能被迭代器检测到，所以不会触发**快速失败**而抛出异常
- **缺点**：迭代器并不能访问到修改后的内容（开始遍历的那一刻就一直拿的是原始集合）

讲讲HashMap?

数据结构

数组+链表+红黑树

红黑树特点

- **节点颜色**：每个节点或者是黑色，或者是红色
- **根节点和叶子节点**：根节点和叶子节点（NIL）是黑色的
- **颜色规则**：如果一个节点是红色的，则它的父节点/子节点必须是黑色的
- **路径规则**：从任意节点到其每个叶子节点的路径都包含相同数量的黑色节点

HashMap哈希函数怎么设计的？

这样设计达到高位低位混合的效果，增加哈希值随机性，降低哈希冲突概率

- 先通过hashCode获得key的哈希值（32位）
- 再将哈希值右移16位，与哈希值本身异或

HashMap和Hashtable的区别？

都实现了Map接口，但是有以下不同：

- HashMap可以接受 null 的key和value，Hashtable不行
- HashMap是非线程安全的，HashTable是线程安全的
- Hashtable很多方法是同步方法，在单线程环境下它比HashMap要慢
- 哈希值的使用不同，HashTable直接使用对象的hashCode，而HashMap重新计算hash值

LinkedHashMap和TreeMap是如何保证有序的？

前者双向链表，后者红黑树

ConcurrentHashMap了解多少？

ConcurrentHashmap线程安全在jdk1.7版本是基于分段锁(Segment数组, ReentrantLock)实现，在jdk1.8是基于 CAS + Synchronized实现

JDK1.7

采用分段锁机制，一个Segment数组，相当于每个Segment都是一个HashMap（但是**没有红黑树** 只有链表），Segment之间相互不会受到影响

▪ put 流程

1. 计算hash 定位到segment 为空就初始化
2. 使用**ReentrantLock**加锁，获取锁失败旧尝试自旋，自旋超过次数旧阻塞获取保证一定成功
3. 遍历HashEntry，和HashMap一样，尝试插入进node

▪ get 流程

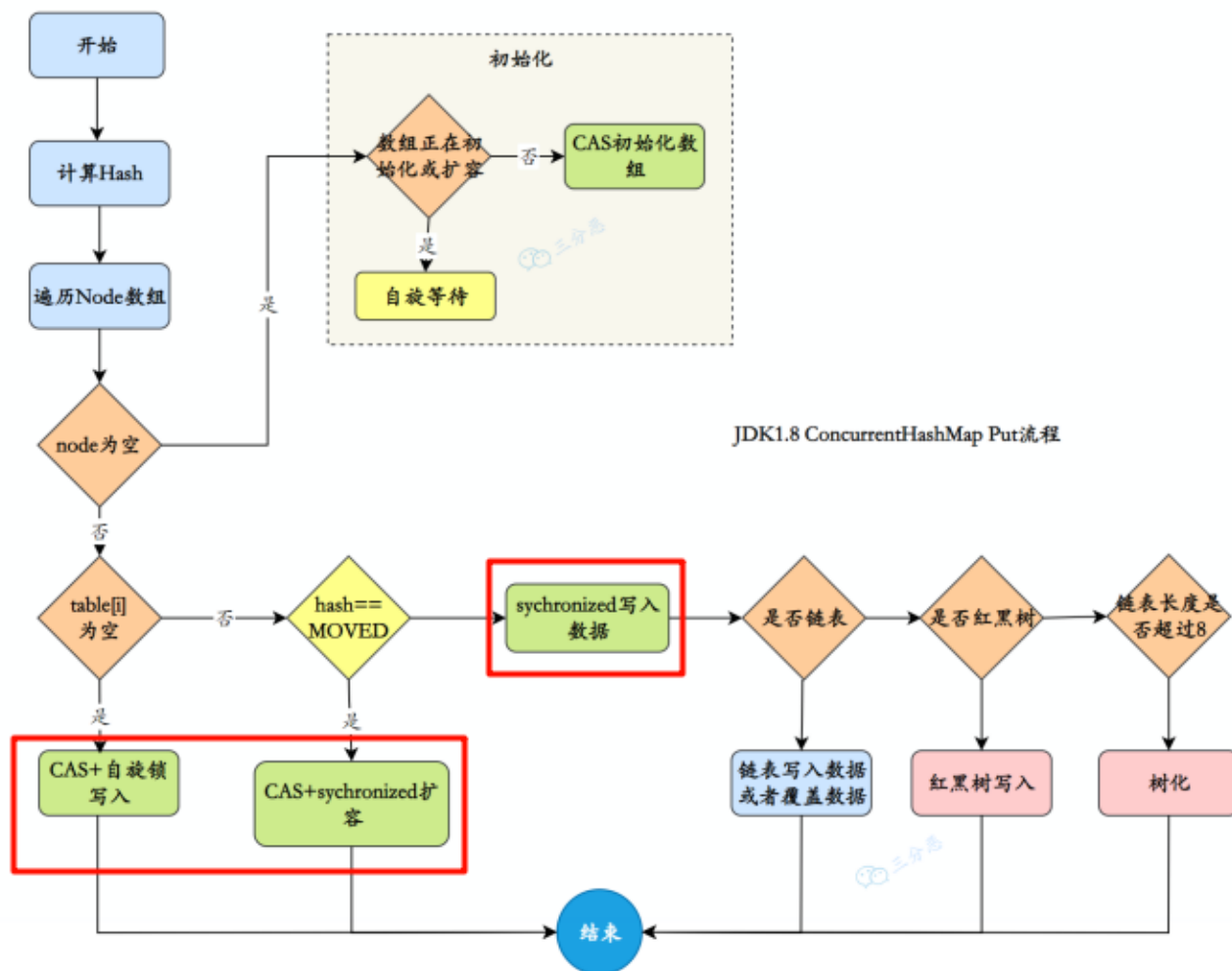
get很简单，key通过hash定位到segment，再遍历链表定位到具体元素（注意：value是volatile的 所以get无需加锁）

JDK1.8

数据结构和HashMap是一样的，**数组+链表+红黑树**。它实现线程安全的关键点在于put流程

▪ put 流程

1. 首先计算hash，遍历node数组，如果node是空的话，就通过 CAS+自旋 的方式初始化
2. 如果当前数组位置是空则直接通过 CAS自旋 写入数据
3. 如果hash==MOVED，说明需要扩容，执行扩容
4. 如果都不满足，就使用synchronized写入数据，写入数据同样判断链表、红黑树，链表写入和HashMap的方式一样，hash一样就覆盖，反之就尾插法，链表长度超过8就转换成红黑树

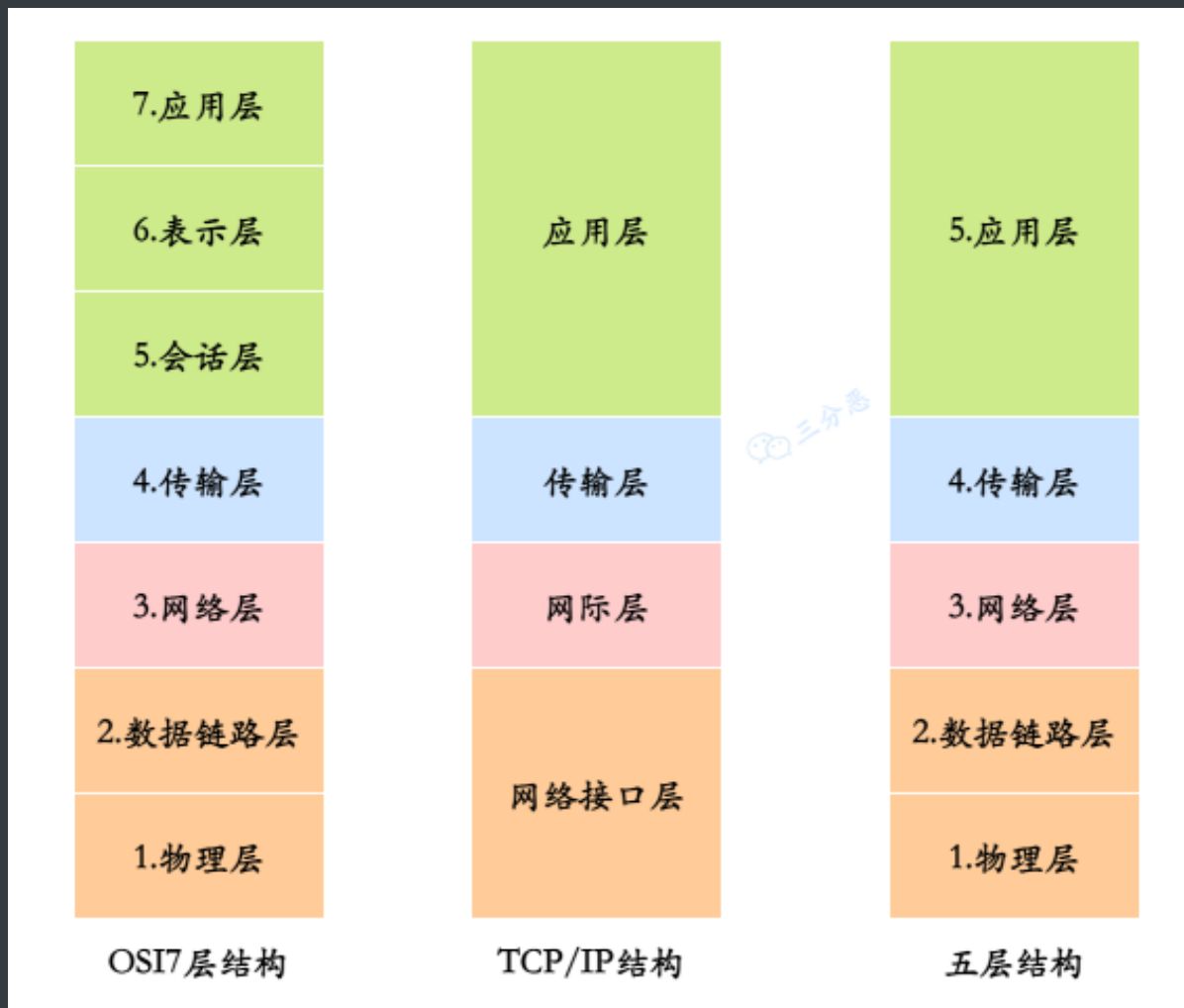


get 流程

get很简单，无锁化读取，和HashMap的get流程基本相同

★计算机网络

说说各层结构



★从浏览器地址栏输入URL到显示页面都发生了什么？

1. DNS解析：将域名解析成对应的IP地址（可以从本地缓存中获取，如果没有再去请求）
2. TCP连接：与服务器通过三次握手，建立TCP连接
3. 向服务器发送HTTP请求
4. 服务器处理请求，返回HTTP响应
5. 浏览器解析并渲染页面
6. 断开连接：TCP四次挥手，连接结束

说说Socket和WebSocket有什么区别？你的项目为什么要用WebSocket？

- **★★★WebSocket** 是一种在单个 TCP 连接上进行全双工通信的协议，它是应用层协议
 - **全双工通信**：客户端和服务端之间可以同时发送和接收数据，实现真正的双向通信
 - **低延迟**：使用单个 TCP 连接，减少了建立连接和发送数据的延迟，实现更快的数据传输速度

- **持久连接**：WebSocket 连接是持久性的，与 HTTP 连接不同，它不需要在每次通信时重新建立连接，可以在连接保持的情况下连续发送数据
- **与HTTP兼容**：WebSocket 握手过程与 HTTP 兼容，可以通过标准的 HTTP 端口（通常是 80 和 443）进行通信，并且可以在现有的基础设施上轻松部署
- 与WebSocket不同，**Socket套接字**，是位于**应用层和传输控制层**之间的一组**接口**，它是为了方便使用TCP/UDP而抽象出来的，相当于使用了设计模式中的**外观模式**把复杂的TCP/IP协议族隐藏，用它即可
- 总的来说，WebSocket 是一种高层次的协议，专门用于实时通信的 Web 应用程序，而 Socket 则是一种通用的底层网络通信接口，可以用于各种网络通信场景。WebSocket 可以看作是在 Socket 的基础上实现的一种高层次的应用层协议

Socket通信的流程？函数？

以下是TCP Socket通信的基本流程

对于UDP套接字，由于它是无连接的，所以没有“连接”和“监听”步骤，客户端可以直接进行数据发送和接受

服务端流程

1. **创建套接字**：socket，指定使用的**通信协议**（IPv4 / IPv6）、**传输层协议**（TCP / UDP）
2. **绑定**：bind，将套接字与本地IP地址和一个端口号关联
3. **监听**：listen，服务端调用 listen() 将套接字设置为监听状态，指定等待连接请求的最大队列长度（此时服务器进入被动等待状态，等待客户端的连接请求）
4. **接受连接**：客户端发起了连接请求，服务端 accept() 被唤醒并返回一个新的套接字（称为已连接套接字，这个新的套接字专用于与该客户端的后续通信），原监听套接字继续等待其他客户端的请求连接
5. **数据传输**：使用新套接字上的 send() 和 recv() 进行数据的发送和接受，服务端与客户端通过这个新套接字进行TCP通信
6. **关闭连接**：当通信结束或出现异常时，服务器调用 close() 函数关闭已连接套接字，释放相关的系统资源。监听套接字通常保持打开状态，继续接受新的连接请求

客户端流程

1. 创建套接字
2. **连接**：客户端调用 `connect()` 函数，指定要连接的服务器的IP地址和端口号。该操作发起一个连接请求到服务器，直到服务器端响应并建立连接
3. **数据传输**
4. 关闭连接

★HTTP

GET / POST请求有什么区别？

GET 的语义是从服务器获取指定的资源，POST 的语义是根据请求负荷（报文 body）对指定的资源做出处理

- **传参方式不同**：GET 请求将信息放在 URL，POST 将请求信息放在请求体中
- **幂等和安全性不同**：GET 符合幂等性和安全性，而 POST 请求不符合；
 - 「安全」是指请求方法不会「破坏」服务器上的资源
 - 「幂等」意思是指多次执行相同的操作，结果都是「相同」的
- **可被缓存不同**：GET可被缓存（缓存在浏览器本身上或者代理上(nginx)），POST不能被缓存

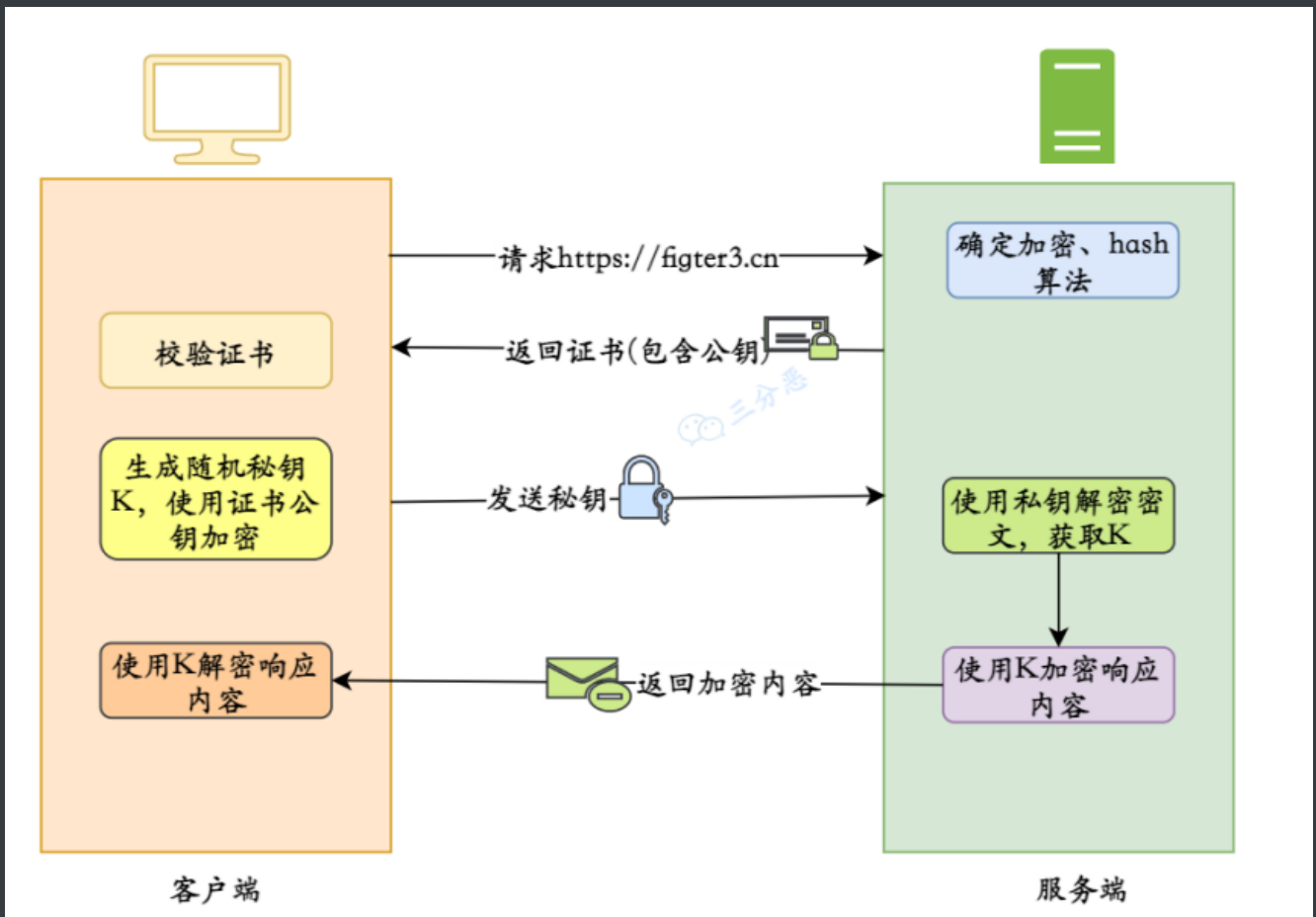
HTTP和HTTPS有什么区别？

- HTTP 是超文本传输协议，信息是明文传输，存在安全风险的问题。**HTTPS 则解决 HTTP 不安全的缺陷**，在 TCP 和 HTTP 网络层之间加入了 SSL/TLS 安全协议，使得报文能够加密传输
- **HTTP 连接建立相对简单**，TCP 三次握手之后便可进行 HTTP 的报文传输。而 HTTPS 在 TCP 三次握手之后，还需进行 SSL/TLS 的握手过程，才可进入加密报文传输
- 两者的**默认端口不一样**，HTTP 默认端口号是 80，HTTPS 默认端口号是 443
- HTTPS 协议需要向 CA（证书权威机构）申请数字证书，来保证服务器的身份是可信的

HTTPS工作流程是怎样的？

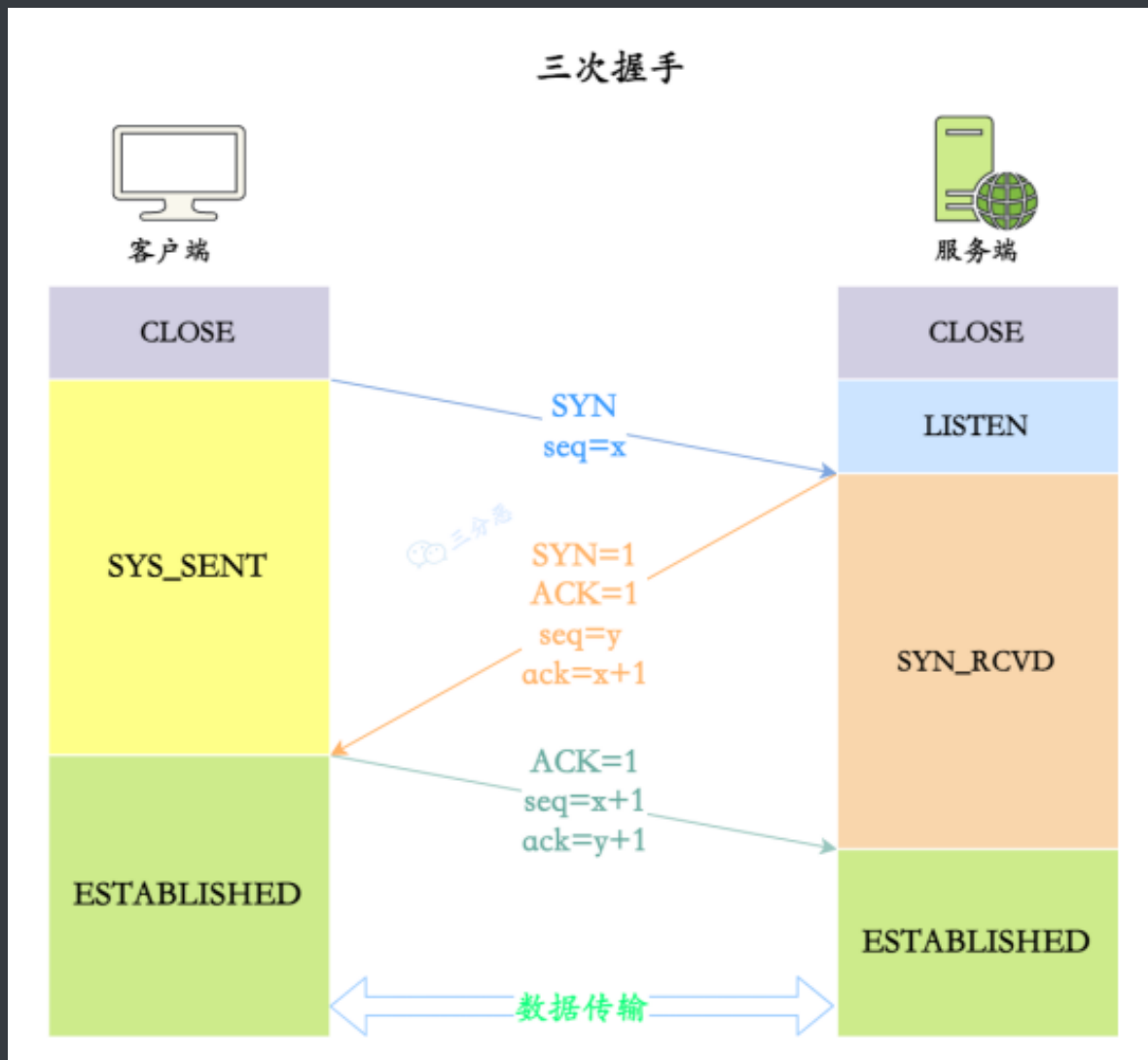
要点：公钥私钥、数字证书、加密、对称加密、非对称加密

- 非对称密钥（服务端的公钥和私钥）用于安全地交换对称密钥
- 对称密钥则用于加密和解密实际的通信数据
- 先用非对称加密得到密钥保证安全 后续使用对称加密保证效率



★TCP

★详细说一下TCP三次握手机制？

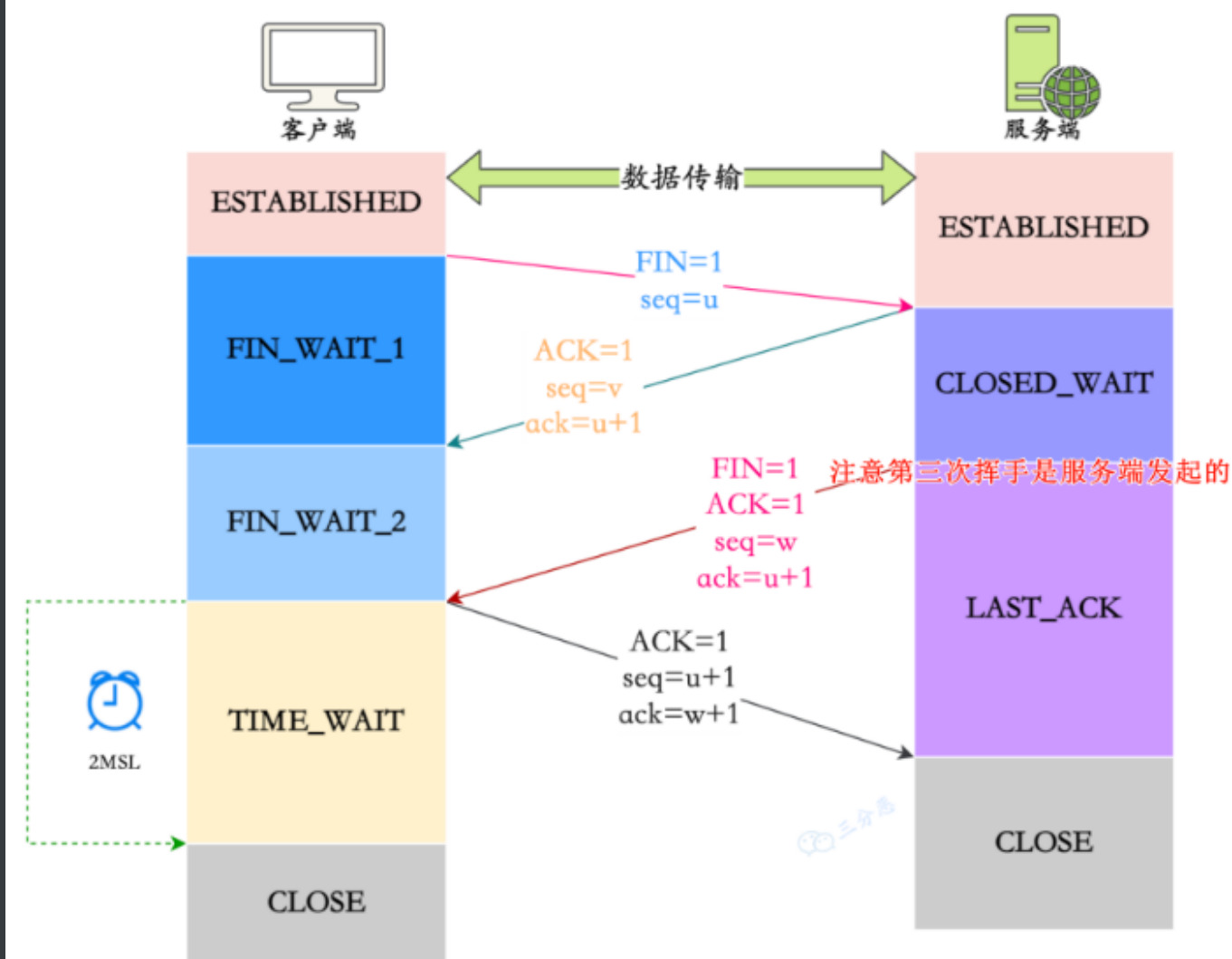


为什么TCP三次握手不能是两次？

- **确认双方的发送和接受能力：**防止第二次握手成功后，服务端这时无法接收数据了，而客户端不知道，因此客户端叽里呱啦说了一大堆，服务端都没收到，浪费资源
- **防止已失效的连接请求建立：**如果一个已经失效的连接请求最终到达了服务器，而服务器误认为这是一个新的连接请求，那么它可能会再次建立连接。引发不必要的连接

★ 说说TCP四次挥手的过程？

四次挥手



为什么TCP挥手需要四次呢？（重点在FIN）

- 关闭连接时，客户端向服务端发送**FIN**，表示客户端没数据要发了，但是还能接收数据
- 服务端收到FIN后，先回一个ACK，但服务端可能还有数据要发送，等服务端不再发送数据再给客户端发FIN表示同意现在关闭连接（因为服务端通常都需要有数据还要发送，所以服务端第二次挥手和第三次挥手的**ACK**和**FIN**分开发送，导致比三次握手多了一次）

TCP四次挥手的最后一次挥手后 为什么客户端需要等待2MSL才关闭？

MSL：报文最大生存时间 它是任何报文在网络上存 在的最长时间，超过这个时间报文将被丢弃

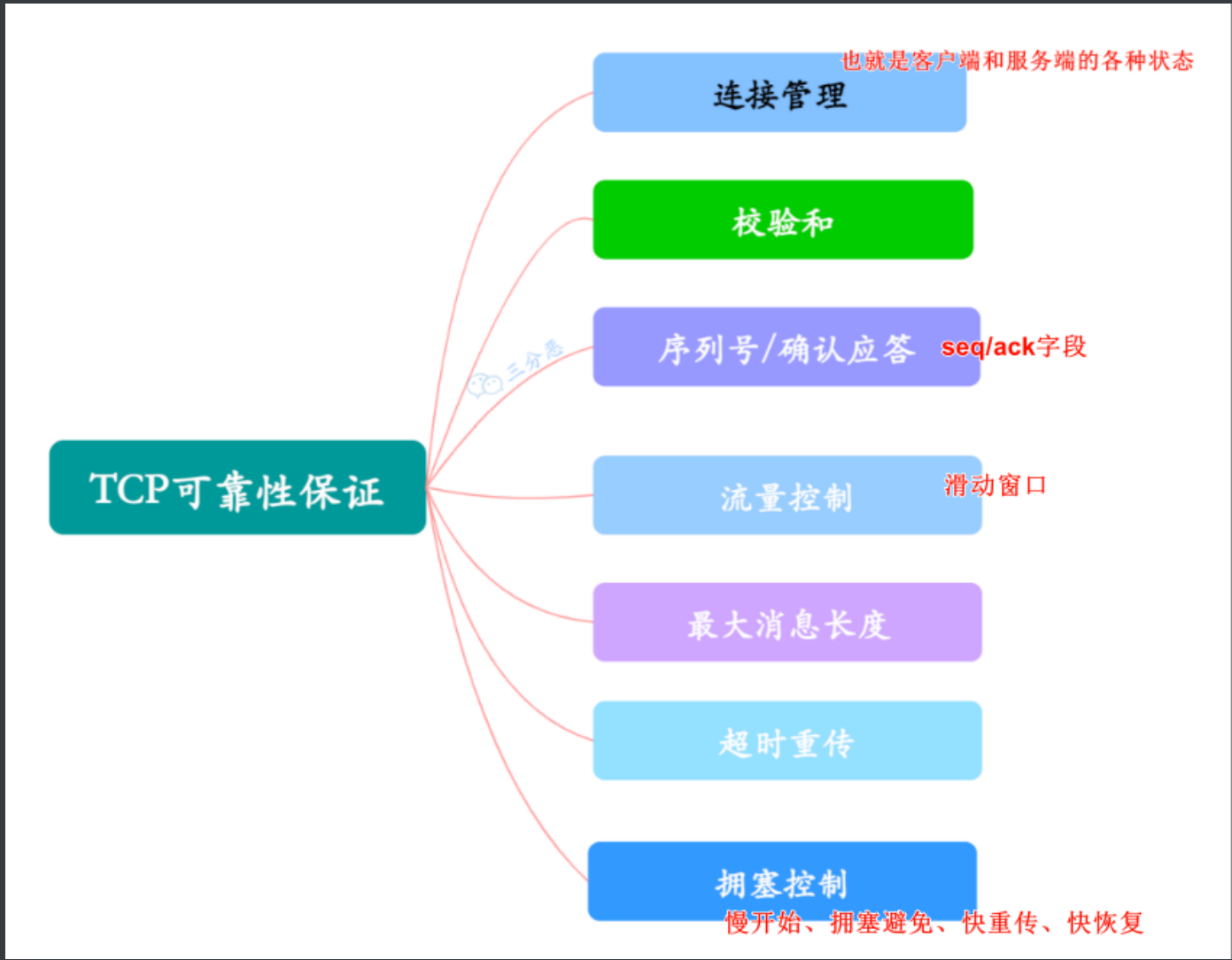
1. 保证客户端发送的最后一个ACK报文能够到达服务端

2. 防止已经失效的连接请求报文出现在本连接中

🌟 close-wait 和 time-wait 状态有什么意义?

- close_wait
 - 服务端收到客户端的关闭连接请求并确认之后就进入该状态
 - 此时，服务端可能还有一些数据没有传输完毕，因此不能立即关闭连接
 - close_wait就是为了保证服务端在关闭连接之前将待发送的数据处理完
- time_wait（也就是为什么为什么需要等待2MSL后才关闭）
 - 发生在第四次挥手，当客户端向服务端发送最后一次ACK确认报文后进入time_wait状态
 - 防止旧连接的数据包
 - 保证连接正确关闭

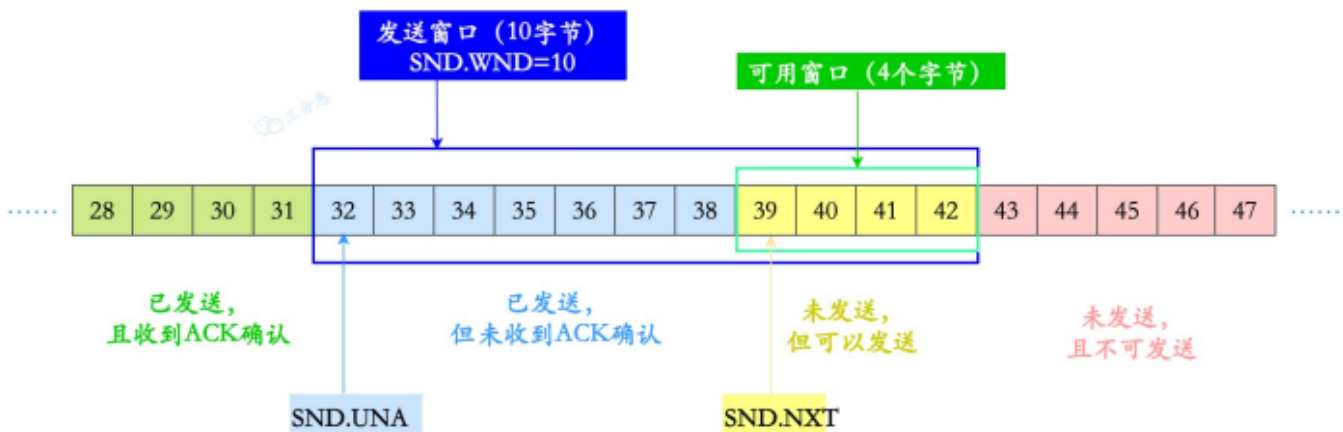
TCP如何保证连接可靠?



✨详细说说流量控制（滑动窗口）？

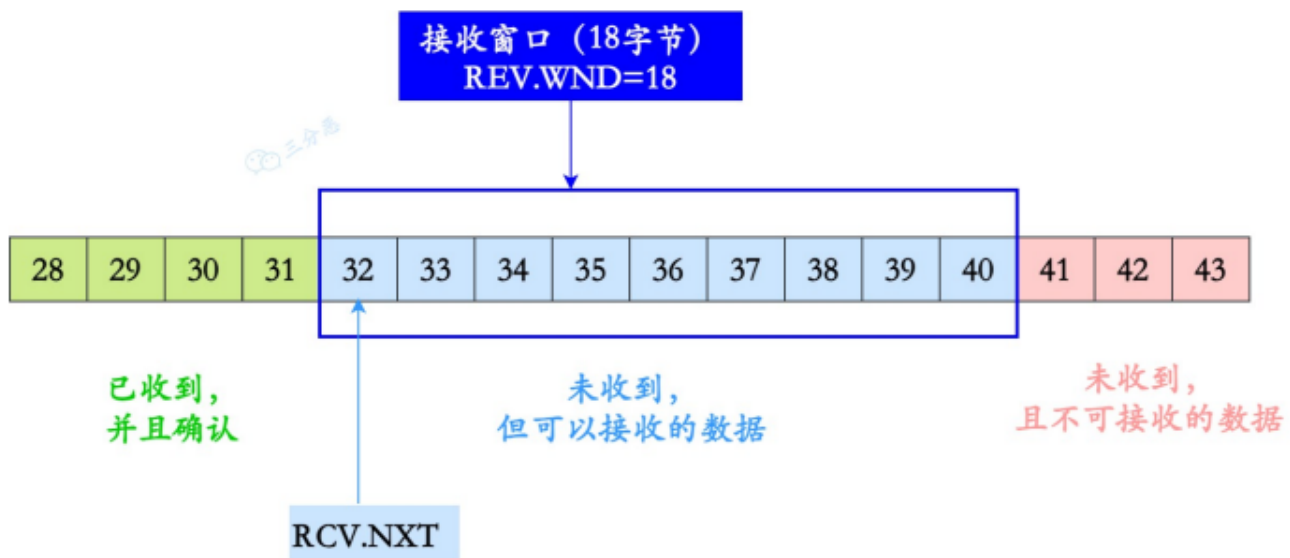
- 已发送且已收到 ACK 确认
- 已发送但未收到 ACK 确认
- 未发送但可以发送
- 未发送也不可以发送

发送窗口



- 已成功接收并确认
- 未收到数据但可以接收
- 未收到数据并不可接收的数据

接受窗口

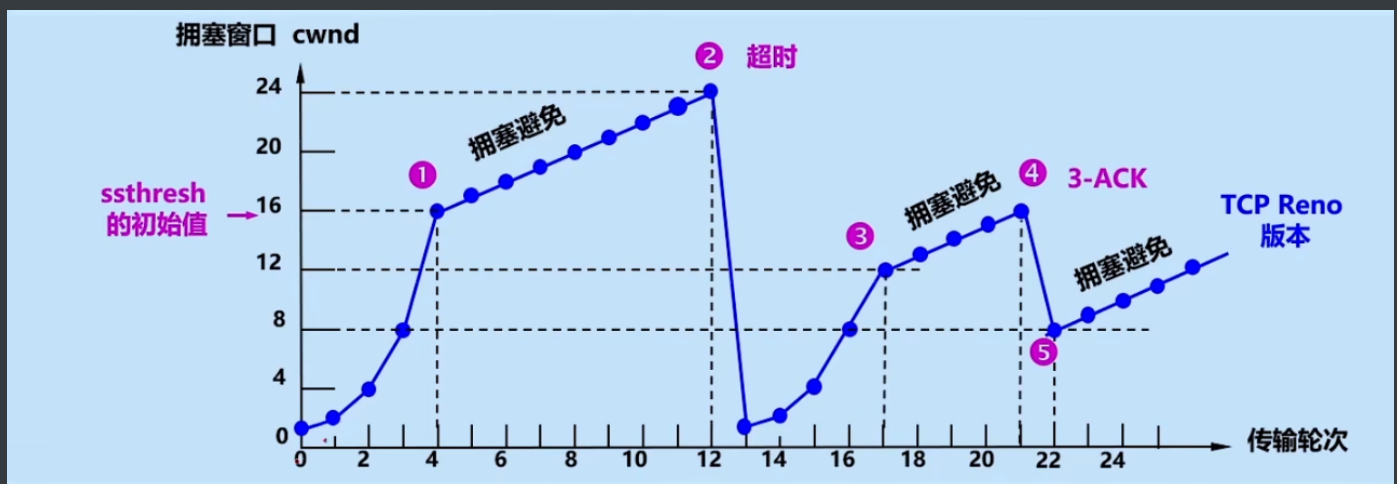


★详细说说拥塞控制?

拥塞控制是相对于整个网络拥堵状况来说的, 避免整个网络水管接的太慢发的太快

几个核心概念:

- **ssthresh**: 慢开始门限
- **cwnd**: 拥塞窗口



拥塞控制分为几个阶段:

- **慢开始**：每轮cwnd翻倍，直到 $cwnd \geq ssthresh$ 就结束慢开始
- **拥塞避免**：慢开始结束后，开始执行拥塞避免，每轮 $cwnd + 1$
- **拥塞发生**：发生**超时重传**情况，则执行**拥塞发生**，设 $cwnd = 1$ ， $ssthresh = \text{超时时的 } cwnd / 2$
- **快重传 / 快恢复**：发生收到 (3-ACK) **重复确认**情况，则执行**快重传/快恢复**，设 $cwnd /= 2$ ， $ssthresh = cwnd$ （也就是立刻开始拥塞避免）

TCP / UDP 区别？

最根本区别：**TCP**面向连接、可靠，而**UDP**是无连接、不可靠

- **UDP效率高、速度快、占用资源少、但是不保证可靠**（适用即时性强但准确要求低的场景，如即时通讯）
- **TCP效率慢、占用资源多、但是保证可靠性**（适用于准确率要求高但是速度可以相对慢，如收发邮件、文件传输）

计算机操作系统

基础

操作系统的功能？

四大管理+用户接口

- 处理机管理
- 存储器管理
- 设备管理
- 文件管理
- 用户接口

操作系统的主要特性？

- 并发（不是并行，注意区分）
- 共享
 - 互斥共享（如打印机 不可同时 必须互斥）
 - 同时访问（如磁盘可以同时读取）
- 虚拟
 - 时分复用
 - 空分复用
- 异步
 - 并发环境下，程序的执行顺序、开始/执行时间 不确定

☆☆☆进程与线程

☆什么是进程和线程、协程？



- **进程**：计算机中正在运行的一个程序实例。（如：打开的微信就是一个进程）
- **线程**：多个线程可以在同一个进程中同时执行，并且共享进程的资源（如：打开的微信里有个线程专门监控别人发给你的消息）
- **协程**：是一种用户态的轻量级线程，协程的调度**完全由用户控制**（也就是在用户态执行）

进程和线程的区别？

- 线程是进程划分成的更小的运行单位,一个进程在其执行的过程中可以产生多个线程。
- 线程和进程最大的不同在于基本上各进程是独立的，而各线程则不一定，因为同一进程中的线程极有可能会相互影响。
- 线程执行开销小，但不利于资源的管理和保护；而进程正相反。

★为什么有进程还要线程？

- 进程切换开销太大，线程切换成本低
- 多线程利于高并发性能高
- 线程之间可以共享进程的内存和文件，无需像进程之间通信需要调用内核

进程有哪些状态？

- 创建
- 就绪
- 运行
- 阻塞
- 结束

什么是互斥和同步？

- 互斥：间接制约关系，源于**资源共享**，如：打印机资源，不可同时使用
- 同步：直接制约关系，源于**进程/线程间的合作**，如：A、B合作完成一件事，A做完了B才能接着做
- **同步机制应该遵循的原则：**
 - 空闲让进
 - 忙则等待
 - 有限等待
 - **让权等待**：当进程/线程无法获取自己所需资源时，应立即释放处理机，以免进程陷入“忙等”

🌟🌟🌟进程/线程间如何同步？

同步的主要任务是对多个相关进程在**执行次序**上进行协调，以使并发执行的诸进程之间能有效的共享资源和相互合作，从而使程序的执行具有可再现性

- 互斥锁：加锁解锁，挨个访问共享资源。例如Java中的 `synchronized`

- 🌟信号量

- 整型信号量（和CAS自旋锁思想相同）

会造成“忙等”，未遵循“让权等待”准则

思想如下：

- 定义一个用于表示资源数目的整型量S V操作就S++
 - 除初始化外，仅能通过两个标准的原子操作wait(S)/P操作和signal(S)/V操作来访问S
- P操作先检查资源数，大于0就减一
如果资源数小于等于0就 一直wait

- 记录型信号量

- 记录型信号量定义如下：
value 大于等于0 代表可用资源的数目；小于0 则它的绝对值代表由于申请资源而阻塞等待的进程数量

```
struct semaphore {  
    int value;           // 资源数目  
    struct semaphore *list; // 等待进程链表指针，用于链接所有等待进程  
}
```

- 记录型信号量的wait(S)和signal(S)

P操作

```
wait(semaphore *S) {  
    S.value--;           // 申请资源，减量操作  
    if(S.value < 0) block(S.list); // 资源分配完成，自我阻塞  
}
```

V操作

```
signal(semaphore *S) {  
    S.value++;           // 释放资源，增量操作  
    if(S.value <= 0) wakeup(S.list) // 唤醒list中的第一个等待的进程  
    // 唤醒后可以直接使用资源，不用再进行P操作，但是用完资源要V操作  
}
```

🌟进程通信方式有哪些？

每个进程的用户地址空间都是独立的，一般而言是不能互相访问的，但内核空间是每个进程都共享的，所以**进程之间要通信必须通过内核**

管道

- 匿名管道：用完即销毁（只能用于存在父子关系的进程）

```
$ ps auxf | grep mysql
```

Linux命令里的“|”就是一个匿名管道，功能是将前一个命令的输出作为后一个命令的输入

- 命名管道：FIFO，需要使用前通过 `mkfifo` 命令创建并指定名字（可以在不相关之间的进程）

```
$ mkfifo myPipe # myPipe就是管道的名字
$ echo "hello" > myPipe # 将数据写进管道
$ cat < myPipe # 读取管道里的数据
```

- 好处：简单、同时很容易得知管道里的数据已经被另一个进程读取了
- 坏处：效率低、只能单向通信、不适合进程间频繁交换数据

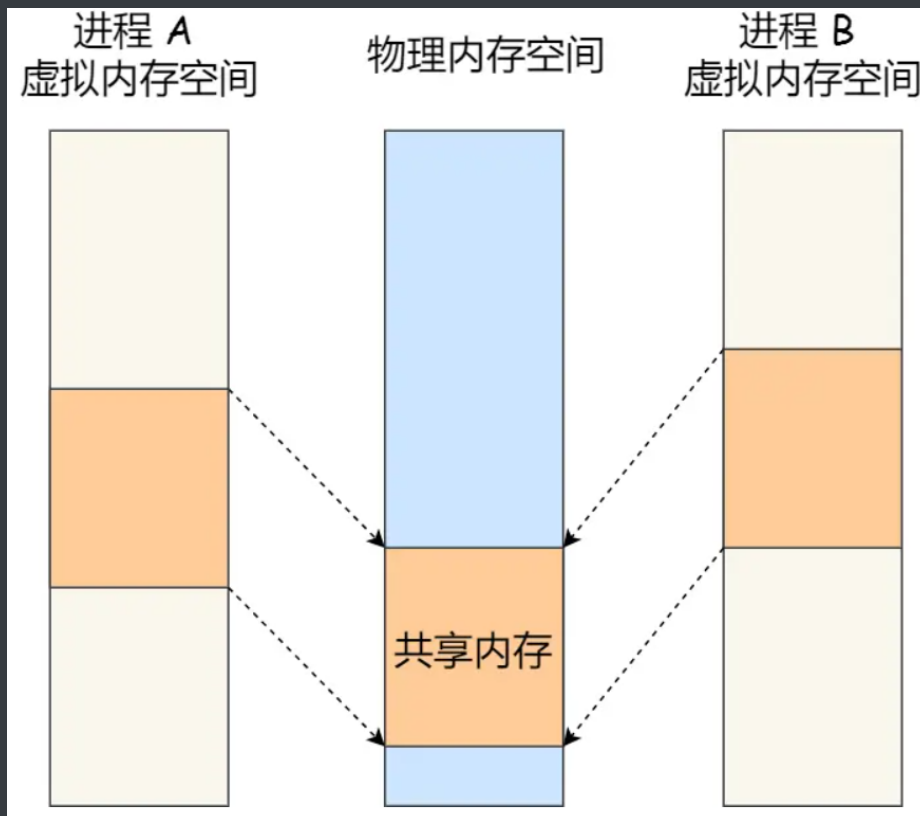
消息队列

消息队列是保存在内核中的消息链表

- 好处：解决管道不适合频繁交换数据的问题
- 坏处：通信不及时、消息附件也有大小限制、不适合比较大数据的传输、通信过程中存在用户态与内核态之间的数据拷贝开销

共享内存

共享内存的机制，就是拿出一块虚拟地址空间来，映射到相同的物理内存中



- 好处：解决了（消息队列中的）用户态与内核态之间的数据拷贝开销、**大大提高了进程间通信的速度**
- 坏处：多进程同时访问共享内存时候，需要进行**同步**，造成额外开销

信号量

为了解决多进程竞争共享资源的问题，信号量就实现了这个**同步机制**

信号量的主要目的是**确保进程同步和互斥**，但在某些情况下，它也可以作为一种基本的通信机制

Socket

前面提到的都是在同一台主机上进行进程间通信，

跨网络与不同主机上的进程之间通信，就需要Socket了

进程调度算法有哪些？

- 先来先服务

- 短作业优先
- 时间片轮转
- 多级反馈队列（目前公认较好的一种）
- 高响应比优先

什么是僵尸进程和孤儿进程？

- 僵尸进程：一个进程已经完成执行，但是其父进程尚未对其进行善后处理，仍在继续占用系统资源
- 孤儿进程：一个进程，它的父进程已经退出或被终止，但它仍在系统中运行

如何查看是否有僵尸进程？

- **top命令**查看是否有僵尸进程

```
top - 15:08:36 up 14:07, 6 users, load average: 0.00, 0.01, 0.07
Tasks: 182 total, 2 running, 178 sleeping, 0 stopped, 2 zombie
Cpu(s): 0.0%us, 0.3%sy, 0.0%ni, 98.7%id, 0.3%wa, 0.0%hi, 0.7%si, 0.0%st
Mem: 1035140k total, 1014952k used, 20188k free, 5376k buffers
Swap: 1048568k total, 32644k used, 1015924k free, 801856k cached
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1	root	15	0	2072	620	532	S	0.0	0.1	0:01.43	init
2	root	RT	-5	0	0	0	S	0.0	0.0	0:00.00	migration/0
3	root	34	19	0	0	0	S	0.0	0.0	0:00.00	ksoftirqd/0
4	root	RT	-5	0	0	0	S	0.0	0.0	0:00.00	watchdog/0
5	root	10	-5	0	0	0	S	0.0	0.0	0:00.60	events/0
6	root	10	-5	0	0	0	S	0.0	0.0	0:00.00	khelper
7	root	10	-5	0	0	0	S	0.0	0.0	0:00.00	kthread
10	root	10	-5	0	0	0	S	0.0	0.0	0:00.39	kblockd/0

- zombie数量不为0时，**ps命令**和**grep**定位僵尸进程（查询状态为Zz的进程）

```
ps -A -ostat,ppid,pid,cmd | grep -e '^[Zz]'
```

- **kill命令**杀死僵尸进程或者父进程即可

***死锁

什么是死锁？

- 多个进程/线程同时被阻塞，它们中的一个或者全部都在等待某个资源被释放。由于进程/线程被无限期地阻塞，因此程序不可能正常终止

产生死锁的必要条件？

- 互斥
- 请求和保持
- 不可抢占
- 循环等待

如何解决死锁？

解决死锁从多个角度分析：预防、避免、检测和解除

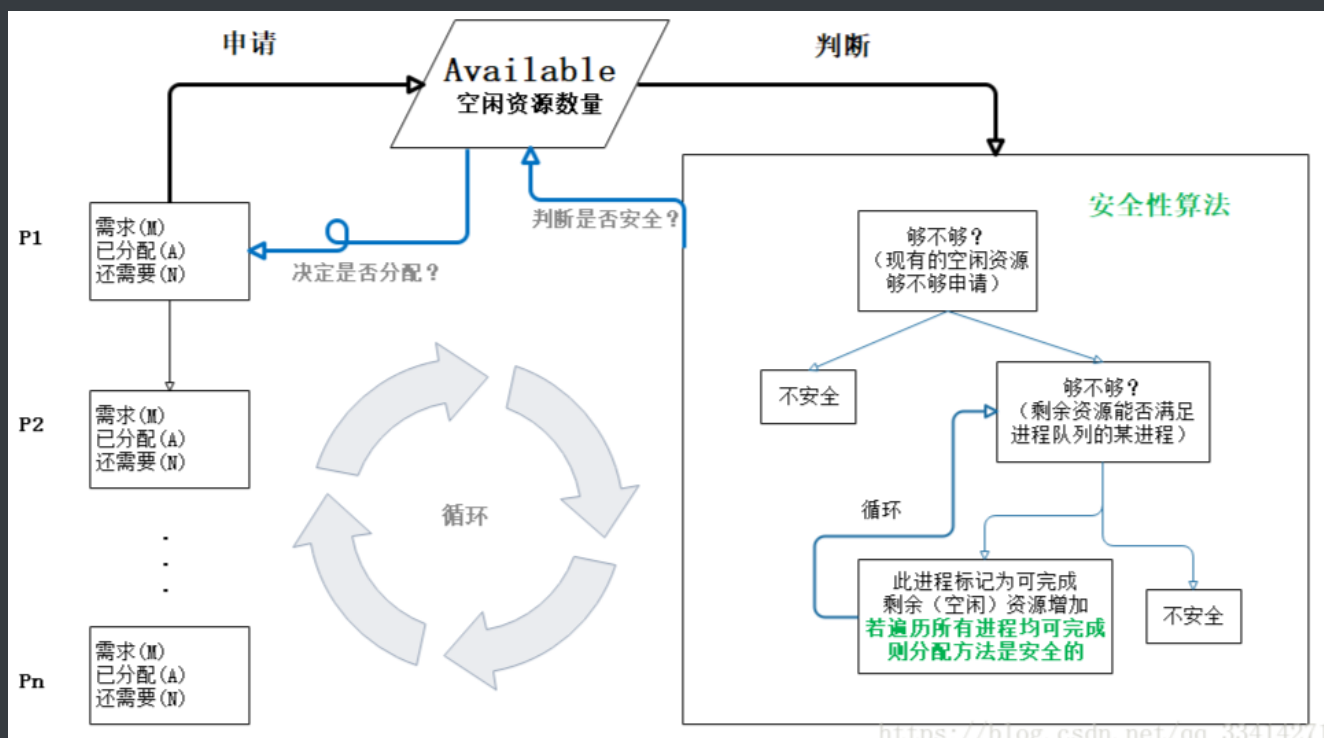
- **预防**：限制并发进程对资源的请求，使得任何时候都不可能满足死锁的必要条件
- **避免**：系统在分配资源时，根据资源的使用情况**提前做出预测**，从而**避免死锁的发生**
- **检测**：系统内有**专门的机构**，死锁发生时该机构能检测到，并确定死锁相关的进程和资源
- **解除**：是与检测相配套的一种措施，用于**将进程从死锁状态下解脱出来**。

如何避免死锁？

预防死锁，本质上就是破坏四个必要条件之一（**互斥**不能破坏）

- 破坏**不可抢占**：也就是采用 **剥夺式调度算法**（不适用所有类型的资源，且导致资源利用率下降）
- 所以一般考虑破坏 **请求和保持** 和 **循环等待**
- **☆☆☆大名鼎鼎的银行家算法**（安全分配资源）（避免死锁算法而非解除！）

当一个进程申请使用资源的时候，银行家算法通过先 **试探** 分配给该进程资源，然后通过安全性算法判断分配后的系统是否处于安全状态，若不安全则试探分配作废，让该进程继续等待



内存管理

内存管理主要做了什么?

- 内存分配
- 内存保护：硬件检查越界，软件中断处理
- 地址映射：逻辑地址 -> 物理地址
- 内存扩充：虚拟存储技术

有哪些内存分配方式?

- 连续
 - 单一（只分系统区和用户区）
 - 固定（内存可用区分为若干个大小固定的，一个分区就装一道作业）
 - 动态（事先不划分，待需要时再按需分配）（因此需要如下的算法决定如何分配）
 - 首次适应
 - 循环首次适应
 - 最佳适应
 - 最坏适应

连续分配会形成“碎片”，尽管有算法“紧凑/拼接”也会有浪费，因此产生离散分配

- 离散

页是信息的**物理单位**，**段**是信息的**逻辑单位**

- 分页：内存分为固定大小的物理页（按物理大小分）
- 分段：内存分为若干个段（一个段是相同逻辑的信息，如主程序、子程序等）（按逻辑分）
- 段页式：段和页各取所长，内存先分为若干段，再把每段分为若干页

虚拟内存？

是指具有**请求调入功能和置换功能**，**能从逻辑上**对内存容量加以扩充的一种存储系统

- 基本原理

- 部分装入
- 请求调页
- 页面置换

- 存储管理方式

- 请求分页（相比普通分页多了**缺页中断机构**，下同理）
- 请求分段

- 🌟页面置换算法

- 最佳置换算法OPT
 - 一种理论上的算法，实际不可能实现
 - 选择以后永不使用的或者未来最长时间不再使用的页面淘汰
- 先进先出FIFO
 - 选择在内存中驻留时间最长的页面淘汰
- 最近最久未使用LRU
 - 选择最近最久未使用的页面进行淘汰——用“最近的过去”作为“最近的将来”的类似

文件管理

文件管理做了什么？

- 存储管理
- 文件管理
- 目录管理
- 文件访问控制

磁盘调度算法？

- 先来先服务
- 最短寻道时间优先
- 扫描算法
- 循环扫描算法

☆☆☆I/O多路复用

简单说说什么是I/O多路复用？

一个单独的线程或进程可以同时处理多个I/O操作，不必耗费时间和资源创建多个线程或进程，提高了程序的并发性和效率

概念： I/O多路复用通过使用特定的**系统调用**（如select、poll、epoll等）可以同时监视多个I/O事件，一旦其中任何一个I/O事件就绪（例如，可以读取数据或写入数据），程序就会得到通知，然后可以针对这个就绪的事件进行相应的处理。

可以先举例说明：

假设你是一个老师，让30个 学生解答一道题目，然后检查学生做的是否正确，你有下面几个选择

1. 按顺序逐个检查，这中间如果有一个学生卡住，全班都会被耽误（**同步阻塞**）
2. 按顺序逐个检查，这中间如果有一个学生卡住，直接跳过，去看后面的（**同步非阻塞**）

3. 你站在讲台上等，谁解答完谁举手，谁举手你下去检查谁的答案，然后再回讲台等着（多路复用）

有哪些实现方式？

原理比较难，不详细叙述，就用上面 老师-学生 的模型来看

- **select / poll**：学生写完了作业会举手，但是你不知道是谁举手，需要一个个的去询问
二者区别在于（没有本质区别，但是有小区别）
 - select 的 文件描述符（也就是所有学生）用**long[1024]数组**存储，而poll是基于**链表**
 - select 对 学生个数作了限制（内核限制为1024个），而 poll无限制
- **★epoll**：学生写完了作业会举手，你知道是谁举手，你直接去收作业

简单原理在于：epoll 用**红黑树**来跟踪存储所有待检测的文件描述符（所有学生），并维护了一个链表来记录就绪事件（也就是举手的学生）

epoll的**巨大优点**在于：

- 没有最大并发连接的限制
- **★**不采取轮询所有连接的方式，效率高，只会处理活跃的连接，与连接总数无关

设计模式

面向对象设计原则

1. 单一职责原则：一个对象应该只包含单一的职责，并且该职责被完整地封装在一个类中
2. 开闭原则：软件实体（如类/模块/函数等）应当对扩展开放，对修改关闭
3. 里式替换原则：所有使用父类的地方都必须能够无缝地替换为子类，而不会引发任何错误或异常
4. 依赖倒转原则：高层模块不应依赖于底层模块，它们都应该依赖抽象。抽象不应依赖于具体实现，具体实现应该依赖于抽象。

5. 接口隔离原则：客户端不应该依赖于它们不需要的接口。接口应该尽可能地小，精确地满足客户端的需求。
6. 合成复用原则：实现代码复用应该优先使用对象组合，而不是通过继承来达到复用的目的。
7. **迪米特法则**（最少知识原则）：一个类/模块对其他的类/模块有越少的交互越好

创建型

关注的是如何更有效地**创建对象**，**封装对象**的创建过程，以及降低系统中对象之间的耦合度

1. **🌟工厂方法模式**：定义一个用于创建对象的接口，但由子类决定实例化哪个类。工厂方法模式将对象的实例化延迟到其子类（相当于生产XX手机的工厂）
2. 抽象工厂模式：提供一个创建一系列相关或相互依赖对象的接口，而无需指定它们的具体类。（相当于生产手机的工厂，不特定生产的是什么手机）
3. 建造者模式：相比直接去new一个新的对象，建造者模式的重心更加关注在如何完成每一步的配置。适用于需要构建多个步骤的对象，例如构建复杂的产品对象
4. 单例模式：确保一个类只有一个实例，并提供全局访问点。单例模式通常用于管理共享资源，例如数据库连接、线程池等
5. 原型模式：用原型实例指定创建对象的种类，并通过拷贝这些原型创建新的对象

结构型

结构型设计模式关注如何**将现有的类或对象组织在一起形成更加强大的结构**

1. 类/对象适配器模式：将一个类的接口转换成客户端所期望的另一个接口，以解决接口不兼容的问题（就像USB-typec等转接头）
2. 桥接模式：将抽象部分与它的实现部分分离，以便它们可以独立地变化（例如点奶茶先问你要大杯小杯、加不加芋泥啾啾等条件）
3. 组合模式：将对象组合成树形结构以表示“部分-整体”的层次结构。通过组合模式，客户端可以统一对待单个对象和组合对象（例如电脑的文件夹和文件系统）
4. **装饰模式**：动态地给对象添加额外的功能，而不需要通过继承来实现。装饰模式通过将对象包装在装饰器对象中来实现功能的动态添加。（装饰器模式）

5. 代理模式：为其它对象提供一个代理或占位符，以控制对这个对象的访问。与装饰模式区别是，A代理了B，访问B就是访问A，A装饰了B，访问B还是访问B本身。
6. **外观模式**：为复杂的子系统提供一个简化的接口，以方便客户端使用。（相当于加一个代办处，不用去各个部门自行办理不同的业务）
7. 享元模式：通过共享对象来减少系统中对象的数量，从而节省内存和提高性能。

行为型

行为型设计模式是一类解决**对象之间交互、协作和责任分配**的设计模式。它们关注的是**对象之间的行为和职责分配**，以及如何**有效地管理和组织对象之间的交互**

1. 模版方法模式：定义一个算法的骨架，将具体步骤的实现延迟到子类中。模板方法模式使得子类可以重新定义算法的某些步骤，而不需要改变算法的结构
2. **★ 责任链模式**：为了解耦请求发送者和接收者之间的关系，将多个处理器组成一个链，每个处理器都有机会处理请求，直到请求被处理完成（例如Security的过滤器链）
3. 命令模式：将一个请求封装成一个对象，从而使得请求的发送者和接收者之间解耦。（相当于有一个遥控器给各个模块发请求）
4. 中介者模式：（相当于搞一个中介，想找别人先找它，将原本的复杂多对多简化为一对多）
5. 备忘录模式：在不破坏封装性的前提下，捕获一个对象的内部状态，并在该对象之外保存这个状态，以后可以将该对象恢复到之前保存的状态（相当于写了个备忘录记录当前的状态，以后可以恢复）
6. 观察者模式：定义了一种一对多的依赖关系，使得当一个对象的状态发生改变时，所有依赖于它的对象都会得到通知并自动更新（监听的事件触发，就通知订阅的人）
7. 状态模式：允许对象在其内部状态发生改变时改变其行为，使得对象看起来像是修改了其类
8. **策略模式**：定义一系列算法，将每个算法封装起来，并使它们可以相互替换。策略模式使得算法可以独立于客户端而变化，提高了系统的灵活性（例如线程池拒绝策略）
9. 访问者模式：用于在不改变已有类的前提下，为类增加新的操作（访问者不能修改被访问元素，只能访问）

